

SCRIPTUM

* WHITEPAPER V14 *

DrFunkenstein — May 2026

“Then let us leave behind a chain worthy of remembrance, as history forgets most men. The chain will not.”

— DrFunkenstein —

Table of Contents

PART I — WHY	14
Introduction	14
What Scriptum Is	14
Executive Summary	15
Heimdall — The Safety Control Window (SCW)	18
What SCW Does Not Protect	18
Kavaca — Post-Quantum Cryptographic Security	20
Real Problems. Real Solutions.....	20
Key Compromise.....	20
Malicious or Unintended Transactions	20
Automated Draining Attacks	20
Coercion and Forced Transactions	21
Quantum Threats	21
Definitions	21
Protocol vs. Unit	21
Core protocol vocabulary.....	21
State vocabulary	22
Named mechanisms	22
Economic vocabulary.....	26
Privacy and Disclosure Vocabulary	27
Parameter references	28
System Intent.....	28
Core Principle.....	30
Problem Statement.....	30
Structural Weakness of Immediate Execution.....	30
Consequences.....	30
Limitation of Existing Mitigations.....	30
The Core Problem.....	31
Design Requirement.....	31
Scriptum Approach.....	31
Core Transition.....	31
Design Philosophy.....	31
The Hard Layer — Ownership and State.....	31

The Soft Layer — Value and Fees	32
PART II — HOW	33
Ilmarinen — System Architecture.....	33
Block Parameters.....	33
Monetary Policy	34
Coinbase — The Miner Reward.....	35
Emission Formula.....	35
Coinbase Transaction.....	35
Maturity	36
Supply Accounting.....	36
Constraints	36
Mining Algorithm — SHA3-512	36
Node Tiers.....	37
Multi-Miner Protocol.....	38
Post-Quantum Cryptography — Kavaca.....	38
Why a Single Primitive, Not Pluggable Crypto.....	39
Address Format.....	40
Public Key Binding at First UNLOCK.....	41
Mechanism.....	42
Validation	43
Design Properties	43
Rationale.....	44
Temenos — The Parcel Model.....	44
Attribute Model	44
CLASS	44
ROLE.....	45
STATE	46
Migration States	47
Migration Authority.....	47
State Definitions.....	48
Default State.....	48
LOCKED	48
NEXUS (formerly UNLOCKING).....	48
ACTIVE	49

PERMA_LOCKED (MARKER).....	49
OPEN (VAULT).....	50
ARC.....	50
TIME_GUARDED and VESTING (TREASURE) — Bastet.....	50
State Transitions.....	50
CIRCULATING Lifecycle	50
TREASURE Lifecycle — Bastet.....	51
VAULT Lifecycle	52
Lifecycle Reference Diagram	52
SCW — The Safety Control Window and NEXUS.....	53
NEXUS — The State-Transition Crucible.....	53
CANCEL — Two Authorisation Paths	55
ACTIVE State — Refresh and Default-to-LOCK.....	55
Ma’at — The Transaction Model.....	56
Transaction Definition.....	56
User-Initiated Transactions	56
UNLOCK	56
CANCEL	56
SPEND(x).....	57
Coin Selection and Destination Rules	58
LOCK(x).....	59
BURN(x).....	59
DONATE(x)	59
REFRESH	59
PACK.....	59
HORKOS_AGREE.....	60
HORKOS_LOAD_A.....	60
HORKOS_LOAD_B.....	60
HORKOS_CONFIRM	60
HORKOS_ATTEST.....	60
Protocol-Driven Transactions.....	60
EJECT (formerly ACTIVATE).....	60
EXPIRE.....	61
COINBASE	61

VEST	61
RELEASE.....	61
RESEAL.....	61
SWEEP	61
DANA ISSUE	62
BOARD	62
DISEMBARK.....	62
Horkos Resolution Sequence.....	62
Value Transfer Sequence.....	63
No Hidden Execution	63
The Covenant — Fees and Miner Incentives	63
Fee Philosophy.....	63
Fee Model: Operation-Based, Not Input-Based	64
Fee Schedule	65
f_S — Progressive Tier Model	65
Examples	66
Design Objectives	67
Dust Threshold	67
Miner Reward.....	67
Menuha — Sunday Rest.....	67
Urðarbrunnr — Valuation Integrity and Temporal Pricing.....	68
Valuation Coefficients.....	68
Economic Consequence	69
The UIS — Three Temporal Layers	69
The Four Mechanisms.....	69
Hugin and Munin — V_A Anomaly Detection.....	70
Detection Without Intervention.....	70
Protected Mode — Aurum.....	71
Mining Speed and Hardware	71
What Determines Mining Speed.....	71
51% Attack at Low Hashrate — Economics	71
Block Validation by Hardware.....	72
Minimum Block Time.....	72
Block Architecture and Capacity Model.....	72

The Capacity Formula.....	72
Transaction Sizes.....	73
Genesis Block Parameters	74
Block Size — The Single Governance Variable	75
Protocol-Driven Transitions and Limbo.....	75
Storage Formula.....	76
The Scaling Path.....	76
Horkos	76
Privacy Model.....	77
The Privacy Stack.....	78
Architectural Note: Validity Versus Anonymity.....	79
What the Chain Provides and Does Not Provide.....	80
Adoption Is Privacy.....	82
Ratatoskr — Chain-Traversal Enumeration	82
Audit via Ratatoskr	83
Wallet Recovery via Ratatoskr.....	83
Selective Disclosure	83
Epoch-Bounded View-Key Disclosure	84
Master Rotation.....	85
What the Holder Sees, What the Chain Sees, What the Delegate Sees.....	85
Boundary Honesty.....	86
Forward Reference to CM	86
What This Replaces.....	86
Wallet-Layer Encryption	87
Address Space and Cluster Behaviour.....	88
Node Types — Genesis Requirements	89
Home Mining Node	89
Dedicated Mining Node.....	89
Full Validation Node.....	89
Sanctum — Archive Node	90
Consensus and Finality	91
Chain Selection	91
Confirmations.....	91
Fork Handling.....	92

Terminal Block Height — RAGNAROK_HEIGHT	92
Open Source and Community Review	92
Network Behaviour and Mempool Policy	93
PART III — USE	95
Aurum — Wallet and User Experience	95
Core Flow	95
UX Requirements (Summary)	95
Markets and Aurum	96
Scriptum as a Traded Asset	96
The Exchange Model	97
What the SCW Prevents	97
What the SCW Does Not Prevent	97
Menuha and Volatility	98
Menuha Sweep (SWEEP / Frau Holle) — Cross-Address Consolidation	98
Scope	99
Eligibility	99
Operation	100
Binding Rules	100
Fees and Inclusion	100
Constraints	100
Why Three (Two Fuses)	100
Horkos	101
Participants	101
Negotiated Terms and Structural Floors	102
The Five-Stage Lifecycle	102
Outcomes	103
Menuha Behaviour	103
Why This Design	104
Ás-brú — Fast-Rail Exchange Bridge	105
Participants	105
Fixed Parameters and Trade Terms	106
The Five-Stage Lifecycle	107
Menuha Interaction	108
Edge Cases and Failure Modes	108

Why This Design	109
Kiva — The Discovery Hall.....	110
Participation Model	111
The Hall — Presence	111
The Board — Ás-brú Offers	112
Handoff to Settlement.....	112
Privacy Posture.....	113
Why This Design	113
Price Discovery — Aurum Market	114
Phase 1 — Aurum Market.....	114
Phase 2 — Community Exchange Listing.....	115
Phase 3 — Global Exchange Listing.....	115
Operational Modes and Practical Use	115
Secure Storage — The Default	115
Immediate Use — The ACTIVE State.....	116
Market Interaction and Liquidity	116
System Coherence	117
Operational Principle.....	117
PART IV — INSTITUTION	118
Governance — WOC, Founders, and Curia	118
Constitutional Roles	118
Decision Tiers — Four-Level Multi-Sig.....	118
Level 1 — Curia Operational.....	118
Level 2 — Founders Strategic	118
Level 3 — Cross-Cohort Significant.....	118
Level 4 — Constitutional	119
Emergency Operations.....	119
Decision Classification.....	119
No-Quorum Rules	120
WOC Emergency Authority	120
WOC Succession	120
Fountain Governance.....	120
The Upgrade Ceremony	120
Cut and Splice, Not Branching.....	121

The Ten Gates.....	121
Procedural Commitments.....	122
Welcomed Forkability	123
Limits of Procedural Protection	123
The Tithe — Protocol Revenue.....	124
The Split — Stage One	124
The Tithe Allocation — Stage Two.....	124
Cap-Overflow Rule	125
What Funds Pays For	125
What the RESERVE Holds	125
What the Charity Pot Holds.....	126
Tithe at Genesis	126
Fountain — Genesis Liquidity Provisioning	126
Allocation	126
Price.....	127
Mechanism — Horkos covenants via Aurum	127
Eligibility — public	127
Authority	127
Fiat proceeds	128
Closure.....	128
Exchange-Choice Architecture.....	128
Protocol Hard Caps — Curia Capture Defence.....	130
What Fountain is Not	130
The Donation Mechanism — Dana	130
Aurum Implementation	131
Supply Accounting.....	131
Bastet — The Guardian of Treasure	131
PART V — GENESIS	133
Rebirth and the Founding Manifest.....	133
Block 0 — The Rebirth Block	133
Block 0 — Deterministic Construction.....	133
DISEMBARK — Protocol-Driven Instantiation.....	134
ARC 0 — The Founding Manifest.....	134
Allocation	135

Vesting Schedules.....	136
PART VI — LONGEVITY.....	138
The Hard Ceiling — Supply, Scarcity, and the Answer to Everything	138
The Coin and its Unit.....	138
The River of Forgetting — Lethe and the Lost Coins.....	138
The Sacred Fire — Burn Economics and Gehenna.....	139
The Mayflower Parcel.....	140
The Centennial Renewal — Ragnarök, the Arc, and the Successor Phoenix	140
TREASURE — Automatic Boarding	141
Protocol Enforcement.....	141
The Arc Procedure — Enrollment and Boarding.....	141
Enrollment.....	141
Scheduled Boarding.....	141
Cancellation.....	141
PACKING Phase.....	142
STRAGGLER_WINDOW.....	142
BOARDING	142
ANOINTED and CONDEMNED	142
After Boarding.....	143
Ragnarök and Departure	143
ARRIVAL — Disembarkation in the Successor Phoenix	143
Finality.....	144
The Number That Matters	144
Canonical Continuity and Forks	144
The Oldest Truth.....	144
Ain Soph — System Philosophy	144
Scriptum is not:	144
Scriptum is:.....	145
PART VII — SECURITY & REGULATORY.....	146
The Security Model.....	146
Structural Safety vs.Economic Safety.....	146
What a 51% Attacker Can Do	148
What a 51% Attacker Cannot Do	149
The Three Attack Scenarios	149

Key Theft Scenario	150
Oracle Manipulation Scenario	150
Timestamp Gaming Scenario.....	151
NIST Alignment Statement.....	151
Regulatory Position	152
Invariants and Assurance	153
Conclusion — A System Built for Failure	154
Final Statement.....	154
Appendix A. Privacy Architecture	154
The Three-Layer Model	154
Stealth Wallets — How They Work.....	154
Sending to a Stealth Address	155
Receiving — Chain Scanning.....	155
The Audit Key.....	155
Protocol Integration	156
Appendix B. Aurum UX Reference.....	156
Aurum UX Reference.....	156
A.1.1. Warning Levels	157
A.1.2. Warning Placement.....	158
B.1. Boarding Status Display	158
B.2. UIS Protected Mode	159
Extended Protected Mode (Prolonged Oracle Failure)	159
B.3. Parcel State Display	159
B.4. Fee Preview	160
Changelog.....	Error! Bookmark not defined.
v12.5 changes from v12.4	Error! Bookmark not defined.
v13 changes from v12.5.....	Error! Bookmark not defined.
v13.4 changes from v13.3	Error! Bookmark not defined.
v13.5 changes from v13.4.....	Error! Bookmark not defined.
Genesis VERSION parameter.....	Error! Bookmark not defined.
CM, SPX, IA, Highlights status.....	Error! Bookmark not defined.
Scope explicitly excluded from v13.5	Error! Bookmark not defined.
v13.6 changes from v13.5	Error! Bookmark not defined.
v13.7 changes from v13.6	Error! Bookmark not defined.

v13.8 changes from v13.7	Error! Bookmark not defined.
v13.9 changes from v13.8	Error! Bookmark not defined.
v14 — Pass E (May 2026)	Error! Bookmark not defined.
v14 — Pass D (May 2026)	Error! Bookmark not defined.
v14 changes from v13.9 (corpus unification milestone)....	Error! Bookmark not defined.
Scope explicitly excluded from v14	Error! Bookmark not defined.
Colophon	160

PART I — WHY

Introduction

Scriptum is a general-purpose settlement chain architected around custody-grade safety assumptions.

It is general-purpose in scope: a chain that handles small everyday payments, fast-rail exchange settlement, attested off-chain delivery, private peer-to-peer transfers, institutional bulk settlement, and zero-knowledge anonymous transitions being planned (Scriptum V2, post-Genesis) — all through the same consensus rules, the same fee schedule, the same primitives. It is not a specialised privacy coin, not a specialised payments coin, not a specialised settlement layer. It is a settlement chain that accommodates the full range of legitimate use.

It is custody-grade in posture: every consensus rule, every state transition, every primitive is designed first for the case where the value at stake matters and the holder cannot tolerate loss to theft, key compromise, or quantum decryption. The Safety Control Window treats every transaction as if it might be unauthorised until the 24-hour reversal window closes. Post-quantum signatures and key exchange are baseline from genesis, not a future migration. Selective disclosure is cryptographically bounded, not policy-bounded. Custody discipline shapes the chain's defaults; everyday use inherits the discipline at no operational cost.

This is the reconciliation of two design priorities that other chains treat as opposed. A holder transferring €10 of household value operates the same chain a custodian transferring €200 million operates. Both inherit the same safety assumptions. The chain does not have separate modes, separate fee schedules, separate cryptographic baselines for different holder profiles. The chain has one set of rules and one set of safety assumptions, and the safety assumptions are tuned for the case where the holder cannot afford to be wrong.

What Scriptum Is

You walk into the bank.

Someone is already there robbing it. He takes you into the safe. Your credentials are valid. But the safe has a time lock. It does not open because you asked. It opens when the mechanism has confirmed — after the observation window has passed — that no alarm was raised.

That is Scriptum's Safety Control Window.

Every other blockchain is a bank where a valid key opens the safe immediately. Scriptum is the bank where a valid key starts the clock.

Bitcoin's security rests on hashrate. The more miners compete, the more expensive an attack becomes. This works. But at low hashrate — in the early life of any chain — the cost of a 51% attack falls, and with it the cost of reversing recent settlements.

Scriptum adds a second layer. The Safety Control Window means that reversing a finalised settlement requires not merely a short reorganisation but sustained majority control for the full 24-hour window. An attacker who achieves 51% hashrate for a short burst can attempt to reorganise recent Bitcoin history. To reverse a finalised Scriptum settlement, that attacker needs 51% for a full day — continuously, without interruption.

Hashrate is the vault door. The SCW is the time lock. Both must be defeated. Neither alone is sufficient.

Scriptum builds on Bitcoin's proven foundation — proof-of-work, Parcel accounting, fixed supply — and adds what Bitcoin lacks: protocol-level time separation between authorization and settlement, post-quantum cryptography from genesis, and a fee model that is fair at every scale. From the person sending €10 to the institution settling €200 million. The same rules. The same chain.

Executive Summary

Scriptum's design rests on two core architectural commitments: post-quantum cryptography (Dilithium3 signatures, Kyber key exchange) and the Safety Control Window — the protocol-enforced delay between transaction instruction and irreversible settlement during which an unauthorised movement can still be cancelled. From these commitments flow the chain's structural properties — view-key delegation for holder-controlled audit, address binding for efficient per-address operations, atomic settlement primitives, transparent reserve verifiability — and the applications the design enables: regulated CASP custody, institutional treasury operations, peer-to-peer settlement, atomic bilateral trades via Horkos, payments, exchanges, donations, and Foundation reserve management within a single coherent protocol. Settlement-grade custody is one application of the design, not its purpose. The chain is general-purpose; the post-quantum + SCW foundation gives every application access to properties that other chains cannot offer. Five structural layers: a cancellable Safety Control Window (the NEXUS dwell-state) on protected paths; post-quantum cryptography on every address and signature from genesis; mandatory stealth-compatible output construction (every SPEND derives a fresh one-time output via Kyber/ML-KEM key encapsulation against the recipient's published view key) with delegated-audit view keys; sealed sender memos for per-transaction recipient-visible provenance disclosure and selective audit; and a weekly Menuha rest that closes execution on Sundays. Two constants: honest supply accounting including acknowledged Lethe, and centennial Ragnarök renewal via Arc migration. **Scriptum is built for safety, security, controlled transfer, usability, auditability, fairness, and societal reroute.** It provides reasonable privacy as a property of its architecture — privacy in the bank-wire sense, where ordinary commercial transactions are not subject to broadcast public surveillance — but not stealth as a chain-level commitment. Stronger privacy postures are achievable through wallet-side composition of the chain's existing primitives (multiple home addresses, stealth-on-receive, view-key delegation with epoch-bounded scope; cryptographic anonymity through 7D arrives in Scriptum V2 as a planned post-Genesis extension); the chain provides the cryptographic primitives without claiming maximum stealth as a default property. The chain is designed to outlive its founders, operate under

any price regime, and meet institutional compliance requirements without surrendering reasonable user privacy.

Every existing blockchain assumes that keys remain secure, that users act without error, and that authorization always reflects intent. When those assumptions fail — and they do fail — the result is immediate and irreversible loss. There is no recovery window. There is no intervention point. The system was not designed to survive failure.

Scriptum was. It separates authorization from execution by protocol. Value must be explicitly unlocked, observed, and confirmed over time before it can move. Compromise does not equal immediate loss. Error does not equal permanent loss. The window exists precisely to survive failure — not to prevent it.

In a Bitcoin attack, a valid signature moves value immediately and irreversibly.

In Scriptum, a valid signature begins an observation window. An attacker with a stolen key cannot extract value without also holding sustained hashrate control long enough to suppress the holder's CANCEL across the full 8,640-block Safety Control Window — approximately twenty-four hours of continuous dominance in plain sight.

Every coin in motion returns to LOCKED if the attack fails at any point. Every ACTIVE coin expires back to LOCKED by protocol-enforced EXPIRE. The attacker who holds the key but cannot sustain control for a full day extracts nothing; the attacker who holds sustained control but lacks the key extracts nothing.

This is not a claim that theft is impossible. It is a claim that theft requires BOTH cryptographic compromise AND sustained mining dominance, detectable and cancellable throughout.

Bitcoin spends tens of billions of dollars a year on hashrate to raise the cost of an instant attack. Scriptum raises it structurally, by replacing instant finality with a cancellable observation window and requiring any attacker to maintain both prerequisites across the window.

As Scriptum accumulates adoption, it accumulates hashrate the same way every successful proof-of-work chain does — on top of the structural protection it already has.

Bitcoin has hashrate. Scriptum has hashrate and SCW, and will for as long as the chain runs.

The architectural decision that creates this advantage must be made at block zero. Scriptum made it. Every address is post-quantum from genesis. Stealth addressing and holder-controlled audit keys are live at launch, built on NIST-standardised CRYSTALS primitives. The cryptographic legacy problem that every existing blockchain will eventually face — the day quantum computers can break ECDSA — Scriptum does not have. The chain is born current.

Scriptum introduces two foundational security innovations with no direct equivalent in existing blockchain systems.

The first is the Safety Control Window (SCW). In every existing blockchain, authorization and execution are effectively the same act. A valid signature moves value immediately and irreversibly. Scriptum separates these two events by protocol. Value must be explicitly unlocked, enter a mandatory 8,640-block observation window, become Active, and remain valid before it can move. This introduces time as a fundamental security dimension. Compromise does not equal immediate loss. The window exists precisely to survive failure, not to prevent it.

The second is post-quantum cryptography. Scriptum uses CRYSTALS-Dilithium3 as its native signature scheme, standardised by NIST in 2024. Every address, from the genesis block forward, is secured against quantum attack. **The chain is born post-quantum**, with constitutional upgrade capacity through the Upgrade Ceremony if cryptographic primitives need amendment in future.

Beyond security, Scriptum introduces a monetary model of genuine novelty. The maximum supply is **protocol-fixed under current canonical rules at 42,000,000 Scriptum** — the answer to the ultimate question of life, the universe, and everything — and is **amendable only through the full Upgrade Ceremony** (a constitutional hard fork requiring all ten gates of governance ratification). Under canonical operation the cap is hard; the chain itself enforces it on every block. Amendment is a constitutional act, not an algorithmic adjustment. The base unit is the Nugget: one Scriptum equals one hundred million Nuggets. The block reward is a continuous function of net circulating supply: as supply grows toward 42 million, the reward asymptotically approaches zero. It never reaches it. The Scriptum system continues through successive Phoenix cycles

Burning is formally supported and economically designed. All burns are directed to Gehenna, a protocol-defined address provably unspendable by mathematical construction. Burned Scriptum reduces net circulating supply, expands the mineable space below the 42M ceiling, and increases the block reward — attracting miners, strengthening security, and restoring equilibrium without intervention. Burning is the network breathing.

Scriptum draws deliberately on mythological names from many cultures and traditions. Gehenna from the Abrahamic faiths. Lethe and Aegis from ancient Greece. Ragnarök from the Norse. Heimdall, the watchman who never sleeps, guards the Safety Control Window. Kavaca — divine armour from the Hindu tradition — names the post-quantum security layer. Ilmarinen, the eternal Finnish smith who forged the sky, names the system architecture. Ma'at, the Egyptian goddess of truth and balance, names the transaction model. Ain Soph, the Kabbalistic infinite, names the system philosophy. Temenos, the sacred Greek enclosure, names the vault. Urðarbrunnr, the well of fate from Norse cosmology, names the valuation integrity system. Irkalla, the Sumerian land of no return, anchors the final chapter. This is not decoration. These names carry weight. They give Scriptum a language that anyone, anywhere, across any tradition, can feel before they understand the mathematics.

Scriptum acknowledges what other chains ignore: that coins are lost permanently through accident, death, and circumstance. These lost coins — designated Lethe — Lost Scriptum —

are categorised, acknowledged, and explicitly excluded from all protocol calculations. Scriptum does not pretend to know what it cannot know.

At Year 100 plus one day, the protocol triggers Ragnarök — the centennial renewal. A new chain is born. Every holder must actively board the Arc within a defined migration window. One hundred years of accumulated Lethe is left behind. The new chain departs clean, with known supply, purified circulation, and a fresh 42 million ceiling.

Scriptum is a centennial institution — designed to outlive everyone who builds it, everyone who mines it, and everyone who holds it, and to begin again for those who come after.

Heimdall — The Safety Control Window (SCW)

Scriptum introduces the Safety Control Window (SCW) as a protocol-level control mechanism for value movement. Value is not immediately spendable. It must first be explicitly unlocked, which opens a defined control window during which the intended action becomes visible, verifiable, and reversible.

The SCW allows detection of unintended or malicious actions, user intervention and cancellation, and controlled execution of value transfer. At completion of the SCW, the asset becomes ACTIVE and may be spent, locked, or burned.

The SCW is mandatory at the entry to ACTIVE state, enforced at the protocol level. Every UNLOCK passes through the NEXUS dwell window before value reaches ACTIVE. Once a holder has actively brought value into ACTIVE state, they may choose to keep it there through periodic REFRESH actions; if they do so, they have continuously and deliberately reaffirmed their consent to hot status, and may use that hot value through direct SPEND. **Direct SPEND from ACTIVE is not a bypass of safety — it is the operational use of value the holder has actively maintained in hot state through deliberate ongoing action.** When ACTIVE expires through REFRESH inaction, the value returns to LOCKED automatically, returning to SCW-protected status without holder intervention. The chain's safety architecture is the SCW at entry to ACTIVE, the REFRESH discipline that requires active maintenance to preserve hot status, and the automatic return to LOCKED when active maintenance lapses. Holders also have the option of private SPEND modes (from ACTIVE or directly from LOCKED) that re-enter NEXUS for an additional cancellation window when they want it.

What SCW Does Not Protect

The SCW is a powerful safety primitive but it is bounded. Honest framing matters: holders, custodians, and regulators reading this Whitepaper must understand the boundary between what the SCW protects against and what it does not.

The SCW protects: value moving from LOCKED to ACTIVE (every UNLOCK passes through the 8,640-block NEXUS dwell window during which the holder may CANCEL); value at rest in LOCKED state (theft requires UNLOCK, which surfaces the attack in the dwell window); recovery of value from LOCKED state when a private key has been compromised (the

holder retains CANCEL authority during dwell, independent of whether the attacker has the spend key).

The SCW does not protect: value already in ACTIVE state. Once a holder has actively brought value into ACTIVE through completed UNLOCK, that value may be SPENT directly without re-entering NEXUS. A direct SPEND from ACTIVE is final at consensus inclusion; there is no dwell window, no CANCEL opportunity, no reversal. This is the operational trade-off the holder accepts when choosing to keep value in ACTIVE through REFRESH: hot value is operationally ready and spendable immediately, but it is not SCW-protected against compromise during the period it remains hot. Holders who want SCW protection on outward movement from ACTIVE choose private SPEND modes, which re-enter NEXUS for an additional cancellation window. The chain provides both options; the holder selects per transaction based on the value at stake and the operational urgency.

The SCW does not protect: value in VAULT state under multi-sig governance authority (FUND, RESERVE, CP). These are governed by multi-sig per GOV — L1 (5-of-9 Curia) for routine FUND disbursements and CP group allocations, L4 (WOC + Founders + Curia) for RESERVE access. The SCW dwell does not apply to them. The governance multi-sig is the protection layer for VAULT-held value; the SCW is the protection layer for holder-held LOCKED value.

The SCW does not protect: off-chain value commitments. Horkos covenants, Ás-brú trades, and other settlement primitives have their own protection structures (multi-sig agreement, stake-at-risk, attestor verification, window timeouts). The SCW is not a substitute for these; it operates only on the Scriptum side of value movement.

The SCW does not protect: value moved by holder intent. If a holder genuinely intends to SPEND directly from ACTIVE, the chain executes the SPEND; the SCW does not second-guess intent. A holder who is tricked into signing a SPEND transaction (phishing, social engineering, hostage scenarios) has a Dilithium3 signature on the transaction; consensus treats this as authorisation. The SCW protects against unauthorised key use; it does not protect against deceived holders.

The SCW does not protect during Limbo. CANCEL validity is height-based: CANCEL is valid only while `current_block_height < eject_height`. Once the SCW dwell window has technically closed (block height has reached `eject_height`), CANCEL is rejected at consensus validation. The protocol-driven EJECT transition then executes in the next block under normal load, completing the lifecycle within one block. Under congested load (mempool fuller than block capacity), EJECT may not be included immediately; the Parcel enters *Limbo* — the gap between the formal end of the cancellation window and the actual EJECT inclusion. During Limbo the SCW provides no further protection: CANCEL no longer works (height-based rule), and EJECT has not yet executed. This is a deliberate consensus design choice. Limbo is typically one block under normal load and rare in practice; under sustained congestion it may extend. Holders should be aware that the final block of the SCW dwell is not the latest moment a CANCEL can be submitted — CANCELS intended to be honoured should be submitted with at least one or two blocks of margin before `eject_height`.

The honest framing: **the SCW is the chain's structural defence against unauthorised UNLOCK, not a universal safety guarantee for every transaction.** The holder's responsibility is to choose the operational mode (LOCKED with full SCW protection, or ACTIVE with operational readiness and direct-SPEND availability) appropriate to the value at stake and the use pattern. The chain provides the primitives; the holder chooses the discipline.

Kavaca — Post-Quantum Cryptographic Security

Scriptum uses CRYSTALS-Dilithium3 (ML-DSA, NIST standard 2024) as its native signature scheme. This provides resistance against quantum attacks, publicly verifiable security, and long-term robustness. Every address from genesis is post-quantum secured. **The chain is born post-quantum, with constitutional upgrade capacity through the Upgrade Ceremony if cryptographic primitives need amendment in future.**

Post-quantum signatures are substantially larger than classical signatures (~5,400 bytes per input vs ~250 bytes in Bitcoin). Block capacity at the genesis 2 MB block size yields approximately around 25 settlements per second (UNLOCK + EJECT + SPEND cycle), well above the 5/sec design minimum. This is the deliberate trade-off for quantum security. Scriptum is not designed for high-frequency small transactions; it is settlement-grade infrastructure where deliberateness is the design property, not a limitation.

Real Problems. Real Solutions.

Key Compromise

In conventional systems, possession of a private key enables immediate and irreversible transfer. In Scriptum, an attacker can initiate an unlock but cannot immediately transfer value. The SCW creates a visible and controllable period during which the legitimate owner can detect and cancel.

Malicious or Unintended Transactions

User errors or manipulated transactions typically result in irreversible loss. In Scriptum, value must pass through the SCW before transfer. Unintended actions can be identified and reversed before execution.

Automated Draining Attacks

The most devastating attacks on existing wallets succeed because they execute in milliseconds. Smart contracts drain entire holdings before any human can respond. In Scriptum, instantaneous draining is impossible by protocol; any automated attack must survive the 8,640-block SCW before value leaves the chain. Even with valid signing capability, value cannot move without surviving 8,640 blocks of observable time. Speed is not a weapon here.

Coercion and Forced Transactions

Forced actions still require entry into the SCW, creating a temporal buffer for intervention or cancellation.

Quantum Threats

Scriptum mitigates future cryptographic weakness through Dilithium3. Authorization alone is insufficient — execution requires surviving the SCW.

Definitions

Scriptum uses precise terms. This section defines them once, for quick reference.

Protocol vs. Unit

Scriptum — the name of the protocol and the chain. *Scriptum* is the name of the system as a whole: the consensus rules, the SCW architecture, the governance Charter, the economic structure, the body of work documented in the WP, CM, SPX, and the rest of the corpus.

FUNK — the unit and ticker symbol of the chain's monetary asset. *FUNK* is the unit in which amounts are denominated and the ticker symbol used on exchanges. Bitcoin's analogue: *Bitcoin* is the protocol, *BTC* is the ticker; for Scriptum, the protocol is *Scriptum* and the ticker is *FUNK*. When the WP refers to "5 FUNK," it means five units of the protocol asset; when it refers to " $f_FUNK = f_EUR / V_A$," it means the fee amount denominated in FUNK. The chain's hard cap is $42,000,000 \text{ FUNK} = S_MAX$. Where corpus prose uses *Scriptum* as a unit ("0.1% of principal in Scriptum"), the intended meaning is the unit FUNK; usage will tighten over time to use FUNK consistently where unit denomination is meant.

Core protocol vocabulary

Parcel — Unspent Transaction Output. A discrete unit of value, identified by (txid, output_index), that has been created but not yet consumed as input to a subsequent transaction. Every Parcel carries three attributes: CLASS, ROLE, and STATE.

CLASS — the economic identity of a Parcel. Four classes exist: CIRCULATING (ordinary user value), TREASURE (governed allocations with vesting), VAULT (protocol-controlled institutional value), and MARKER (non-economic identity records).

ROLE — the semantic purpose within a class. Examples: WIZARD, FOUNDER, and CURIA under TREASURE; WFC, FUNDS, RESERVE, and CP under VAULT; MAYFLOWER and DANA under MARKER.

STATE — the current position in the lifecycle. The principal states are LOCKED, NEXUS, ACTIVE, TIME_GUARDED, VESTING, SEALED, OPEN, PERMA_LOCKED, PACKING, and ARC.

State vocabulary

LOCKED — the default state of every CIRCULATING Parcel. Protected by the Safety Control Window. Cannot be spent without passing through UNLOCK and EJECT.

NEXUS (formerly UNLOCKING) — a Parcel in the controlled-transfer dwell-state. Entered by UNLOCK or SPEND. The Safety Control Window is open during dwell; the holder can CANCEL the transition via signature or via hash-preimage submission. Eject fires at `unlock_height + 8,640`, executing the entry's pre-committed destination state transition. Still not spendable during dwell.

ACTIVE — post-EJECT state for self-spends following UNLOCK. The Parcel is now spendable via SPEND, BURN, DONATE, or LOCK. The ACTIVE window is bounded; the holder may REFRESH to extend it; if no action is taken, EXPIRE returns the Parcel to LOCKED.

PERMA_LOCKED — the state of MARKER-class Parcels. Cannot be unlocked, spent, or burned. Persists across every Ragnarök.

Named mechanisms

SCW — Safety Control Window. The dwell period within NEXUS during which CANCEL remains valid. The SCW has a fixed dwell duration of 24 hours (8,640 blocks) defined by the protocol (see SPX-2) and is not subject to dynamic adjustment. The core protective primitive of Scriptum.

NEXUS — the state-transition crucible. The protocol-level machinery that provides cancellable controlled-transfer windows for the chain's three protected paths: UNLOCK to HOME, private SPEND from hot, private SPEND from cold. Value enters NEXUS at the source action; protocol-driven EJECT delivers it to its committed destination at `unlock_height + 8,640`; CANCEL during dwell returns the value to source. Direct SPEND from ACTIVE operates outside NEXUS as the chain's fast operational path — the value has already passed through NEXUS at UNLOCK and the holder has been actively maintaining its hot status through periodic REFRESH; direct SPEND is the operational use of actively-maintained ACTIVE Scriptum, not a bypass of safety. NEXUS is not a mixer, churn pool, or provenance-fog layer; it is the named architecture of the Safety Control Window.

Heimdall — the semantic name for the SCW. The watcher who never sleeps.

Kavaca — the post-quantum cryptographic protection, combining Dilithium3 signatures and Kyber key encapsulation, active from genesis.

Ilmarinen — the node software that runs Scriptum. The mythic Finnish smith who forged the Sampo; here, the consensus engine that maintains the chain's state machine, validates transactions, and propagates blocks. Ilmarinen runs on every participating node — home miners, dedicated miners, full validators, and Sanctum archives.

Hayyānu — the standalone Scriptum miner. Named for the Ugaritic divine craftsman Hayyānu the Skilled (*ḥyn d-ḥrš yd-m*, “Hayyān of the deft hands”). Performs SHA3-512

proof-of-work, assembles candidate blocks, includes Ás-brú API confirmations, and submits the block to the network when a valid hash is found. Architecturally a separate binary from Aurum: Hayyānu connects to a running Ilmarinen node over TCP (default localhost:8333) and exchanges mining jobs and solutions via the scriptum-rpc wire protocol. The separation is structural — App Store and Play Store policies prohibit mining in consumer wallet applications, so Aurum ships mining-free across all distribution channels; Hayyānu is direct-download only from dfqr.org. The mining engine is the same whether invoked as a standalone binary at industrial scale or surfaced via Aurum’s home-mining UI on platforms where mining is permitted (the UI shell wraps the same Hayyānu binary as a subprocess). One engine, two surfaces, three distribution paths.

Aurum — the reference Scriptum wallet. Bundles Ilmarinen internally so every Aurum user runs a validating node (depth varies by device tier — see §Node Tiers). Provides the user-facing layer for all chain operations: receiving, spending, NEXUS routing, Horkos negotiation, Ás-brú trade construction, view-key delegation, Sealed Sender Memo handling, Frau Holle Menuha sweeps, Aurum Market, and Kiva (the discovery hall — see §Kiva). 7D denomination decomposition is a planned Scriptum V2 surface in Aurum and is not present at v14. Aurum is mining-free by design — Hayyānu is a separate binary (see §Hayyānu) so Aurum can ship cleanly through App Store and Play Store distribution channels. The “golden wallet” — the bright, refined surface over the chain’s machinery. Open-source under Apache 2.0; the full Aurum specification is published as a separate technical document. Other wallet implementations conforming to WP / CM / SPX are welcomed and unprivileged.

Kiva — the discovery hall above the settlement primitives. Named for the Pueblo ceremonial chamber where members of a community become visible to one another for collective business — distinct from private dwellings, distinct from the central plaza. Opt-in and symmetric: a holder enters Kiva by explicit choice, becomes visible to other members by their home Scriptum address, and is invisible to and from the hall without entering. Two surfaces operate inside Kiva — the hall (presence list with optional Buy/Sell tags and on-chain reputation indicators inline) and the board (signed Ás-brú offers with 24-hour TTL ceiling, active withdrawal at any time, maximum five active offers per address). Kiva does not execute trades, does not hold funds, does not become a counterparty, does not adjudicate disputes. After two holders find each other through Kiva, the interaction becomes direct Aurum-to-Aurum and settlement proceeds through the chosen primitive — Aurum Market, Horkos, or Ás-brú. A public hall by intent: holders are identified by home address so that on-chain reputation is the trust layer. Holders who require privacy of the fact-of-trading do not enter Kiva and use the chain’s private primitives directly. Kiva is not consensus — no transaction validity depends on it; it is an Aurum-side overlay using the existing Scriptum P2P network for gossip transport. The full Kiva specification appears in §Kiva (this document) and in the Aurum Technical Document.

Horkos — the bilateral atomic settlement primitive (Greek Ὁρκος, the personification of sworn oath in Hesiod’s *Works and Days*; son of Eris, punisher of perjurers). Three-party covenant: A commits Scriptum to ESCROW, B delivers off-chain (fiat, goods, services), C attests to the delivery. The chain enforces structural floors (stake $\geq 0.1\%$ of principal as a nominal protocol minimum — commercial use cases will typically negotiate substantially

higher stakes; window ≥ 360 blocks, window $<$ remaining time to Ragnarök) and procedural rules; within those, parties shape the contract. Resolution flows through standard UNLOCK $\rightarrow$ EJECT $\rightarrow$ SEND. Earlier specifications named this primitive Delayed Atomic Settlement (DAS); the v12 rename unifies the chain's mythological vocabulary.

Ás-brú — the fast-rail exchange bridge primitive (Old Norse for Bifröst, the bridge of the Æsir connecting Midgard to Asgard). Two-party covenant: A locks Scriptum in ESCROW, B transfers off-chain on a centralised exchange. Window fixed at 60 blocks (~10 minutes); no third-party attester; miners run **Crowglass** — the exchange observation daemon — which queries the named exchange's public API for the committed off-chain transfer fingerprint and includes ASBRU_RELEASE in the building miner's block when a matching transfer is confirmed. Half-atomic by design — cryptographic enforcement on the Scriptum side, miner-consensus + reputation backing on the off-chain side. The bridge that connects the on-chain economy to exchange-rail liquidity without requiring formal listing.

The Crowglass naming is intentionally exchange-agnostic. The daemon observes whichever exchange the trade's commit fingerprint specifies; the protocol does not maintain an admitted-exchange list, and miners choose which exchanges they observe. The Foundation publishes a recommended Crowglass operational configuration (see GOV §12.1 and §11.3); the recommendation is informational, not protocol-binding. The corvid imagery (Odin's ravens Huginn — "thought" — and Muninn — "memory" — fly out across the world and return with what they have seen) names the function: distributed observation returning structured testimony. Glass is the window onto external state.

7D — the Seven Dwarves. **Status at v14: architectural commitment; consensus-rule specification deferred to Scriptum V2 (post-Genesis).** The fixed-denomination cryptographic anonymity layer planned for V2. The design envisages seven independent commitment trees corresponding to the seven denominations: Doc (100), Grumpy (200), Happy (500), Bashful (1,000), Sleepy (2,000), Sneezzy (5,000), Dopey (10,000). Each tree maintains its own member set and nullifier registry. SPEND from a dwarf is a STARK ring-membership proof — proves the spender holds an authorised leaf in the tree without revealing which leaf, with the destination address and a fresh nullifier bound into the proof. Wallet-side decomposition handles arbitrary amounts: a holder spending 13,000 routes through (Bashful + Sleepy + Sleepy + Sleepy + Sleepy + Sleepy + Sleepy) or any other valid integer combination, transparently to the user. Cryptographic anonymity scales with denomination collision — the more holders deposit at the same denomination, the larger each tree's anonymity set. **The CM does not at v14 specify the STARK proof system, the commitment tree construction, the nullifier-derivation rule, or the consensus-validation rules for 7D-mode SPEND. These are V2 deliverables; the Foundation's research-and-standardisation track post-Genesis will produce the concrete cryptographic specification. The v14 chain ships pseudonymity + recipient unlinkability (stealth addressing) as its cryptographic-privacy baseline; cryptographic anonymity through 7D is the planned extension, not a current capability.**

Bastet — the vesting enforcement mechanism for TREASURE. Watches allocations through their TIME_GUARDED and VESTING states and releases portions at each monthly milestone.

Menuha — the weekly rest. A 24-hour window each Saturday 23:00 UTC to Sunday 23:00 UTC during which user-initiated outward value transactions (SPEND, BURN, DONATE, LOCK, and all user-initiated Horkos stages including agreement, loading, confirmation, and HORKOS_ATTEST) are invalid. Protocol-driven transactions continue, including the protocol-driven resolution sequence of Horkos contracts whose attestation finalised before the rest day.

Frau Holle — the Menuha Sweep. **Cross-stealth-address consolidation, wallet-driven and Aurum-mandatory.** Consolidates LOCKED Parcels across the holder's stealth-derived addresses into a single LOCKED Parcel at the holder's HOME address when the holder's address count (number of distinct addresses currently containing the holder's Parcels, computed by the wallet via local Ratatoskr scan) exceeds three. The chain's consensus cannot identify which addresses belong to which holders without view-key access; only the holder's wallet, with the holder's view key, can compute the address set and construct the multi-input SWEEP transaction. The wallet signs the transaction with the relevant addresses' Dilithium keys; the chain validates signatures like any other transaction. Aurum's wallet implementation constructs and submits cross-address SWEEPs at the next Menuha when the threshold is crossed without holder opt-out — for users of compliant wallets, Frau Holle is automatic and unavoidable. There is no per-address SWEEP scope; per-address consolidation is performed naturally by UNLOCK as a side effect of normal operation (smallest-first coin selection consumes the holder's per-address Parcels as part of the consolidation to HOME).

Ratatoskr — the chain-traversal enumeration primitive run **wallet-side and auditor-side, not as protocol consensus machinery**. Given a holder's key pair (or a holder-delegated view key), Ratatoskr traverses the chain and reports the address set derivable from those keys (i.e. the set of stealth-derived addresses currently containing Parcels the holder controls). Has one function: report. The primitive is computationally identical to what every wallet does when scanning the chain for its holder's Parcels; Ratatoskr names this scanning operation as a first-class concept and specifies its interfaces (wallet-internal scan, archive-served scan, auditor-delegated scan via view-key handoff). Used by wallets, auditors, and the wallet-side construction of cross-address SWEEP transactions. The squirrel running up and down Yggdrasil carrying messages.

PI — Personal Identity, the operational sovereign in the Identity Tree architecture. The holder's persistent on-chain identity where value lives, where counterparty interactions accumulate, where the on-chain behavioural pattern develops.

Sealed Sender Memo — the encrypted address disclosure carried by every SPEND transaction. Encrypts the sender's published address (and any optional additional content within the size budget) to the recipient using a key derived from the same Kyber/ML-KEM-encapsulated shared secret that derives the stealth output address. Recipient decrypts via their Kyber view key. External observers cannot read the memo. Mandatory in form (every

SPEND carries a well-formed encrypted memo); content is holder-chosen but the minimum is the sender's published address. The Sealed Sender Memo provides **recipient-visible sender-provenance disclosure** as a deliberate trade-off: recipient-side unlinkability is not absolute (the recipient learns which published address sent them value), but external observers without view-key access still cannot link senders to recipients. The chain's per-transaction audit primitive — holders can selectively disclose individual memos to auditors or counterparties to prove specific transactions without revealing their broader on-chain footprint.

Hugin and Munin — the V_A anomaly detection pair. Hugin observes per-block V_P/V_A deviation; Munin holds the cumulative drift and raises the alarm. Odin's two ravens, thought and memory.

Gehenna — the burn address. Provably unspendable. All BURN transactions send value here.

Chesed — the semantic name for voluntary donation to the Vaults (DONATE).

Dana — the permanent MARKER issued on confirmation of a donation.

Mayflower — the permanent MARKER issued at Genesis to Founders and WOC, re-instantiated unchanged at every Rebirth.

Phoenix — a single complete chain from Genesis to Ragnarök; one hundred years of chain life.

Arc — the manifest that carries state across Ragnarök to the successor Phoenix.

Ragnarök — the terminal block of every Phoenix. Block RAGNAROK_HEIGHT.

Economic vocabulary

V_A — the valuation coefficient. Anchors fees to euro value. Defined and protected by Urðarbrunnr (UIS).

f_U, f_A, f_S, f_refresh — the four fee components. f_U is paid at UNLOCK (calculated on summed unlocked value). f_A is reserved at UNLOCK using the V_A at the UNLOCK block, and paid to the miner at the EJECT block (flat instruction fee, irrespective of amount). f_S is the progressive tier fee paid at SPEND (calculated on amount) and at Horkos load operations (HORKOS_LOAD_A on principal plus attestation fee, HORKOS_LOAD_B on stake). f_refresh is the small fee paid for ACTIVE-state REFRESH (governance-set, expected near-zero).

Tithe — the protocol's share of fees. 90% goes to the miner who includes the transaction; 10% is the Tithe, split internally as 48% WFC (compensation pool for the fifteen constitutional position-holders), 32% Funds (Foundation operational allocation), 10% RESERVE (strategic long-term reserve), 10% CP (Charity Pot, the structural humanitarian allocation, additionally receiving all WFC cap-overflow). See §15 for full mechanics.

UIS — the Urðarbrunnr Integrity System. The machinery that maintains V_A honestly in the presence of oracle noise, network churn, and attempted manipulation.

Horkos — the protocol primitive for bilateral trade settlement with third-party attestation. The name (Greek Ὅρκος, the personification of sworn oath in Hesiod) reflects the contract's nature: a sworn bilateral covenant whose breach incurs cryptographic forfeit. Earlier specifications referred to this primitive as “Delayed Atomic Settlement” (DAS); the rename in this version reflects mature naming consistent with the chain's mythological vocabulary (Heimdall, Kavaca, Hayyānu, Ilmarinen).

Privacy and Disclosure Vocabulary

The chain uses six related but distinct terms when describing what recipients, observers, and delegates can and cannot see. These terms have specific technical meanings in this corpus and must not be used interchangeably.

Pseudonymity. The default property of every Scriptum address. An on-chain address is a 64-byte hash; it is not bound to a legal identity at chain level. An observer can see what an address holds, sends, and receives, but cannot derive the legal-person counterparty from the chain alone. Pseudonymity is the baseline; it is what every cryptocurrency provides. *Bank-wire-grade pseudonymity* is the operational standard Scriptum claims: the transaction record exists, is observable to participants in the transaction and to legitimate investigators with proper authority, and is not visible to the general public except where parties choose to publish.

Recipient unlinkability. The property that distinct outputs sent to the same recipient address are not visibly tied to a common recipient by an on-chain observer. Stealth derivation provides recipient unlinkability by default: every SPEND output lands at a fresh, single-use stealth address; an observer cannot determine that two outputs go to the same recipient without auxiliary information.

Graph unlinkability. The stronger property that the *sequence* of transactions involving a holder — the chain-graph connecting their outputs over time — cannot be reconstructed by an on-chain observer. Stealth addressing alone does not provide graph unlinkability; sender-recipient pairs are inferable from broadcast and spend timing. The 7D path is the chain's mechanism for graph unlinkability; non-7D transactions do not claim it.

Cryptographic anonymity. The property that an observer cannot identify the sender or recipient of a transaction at all. This is a stronger property than pseudonymity or unlinkability. Scriptum provides cryptographic anonymity *only* via the 7D path (zero-knowledge transitions hiding sender, recipient, and amount behind STARK proofs). Standard SPEND does not provide cryptographic anonymity. *Never use “anonymous” to describe a non-7D transfer.*

Audit visibility (selective disclosure). The property that a holder can deliberately reveal a bounded portion of their own activity to a chosen delegate (auditor, accountant, custodian, heir, legal counterparty) without revealing anything outside the bound. Scriptum provides audit visibility per UTC calendar year through the epoch-bounded view-

key construction (CM-16): the holder exports a specific year's epoch view secret; the delegate decrypts that year's outputs and nothing else. The boundary is cryptographic; the holder does not depend on the delegate's good behaviour.

Privacy (umbrella term). When used without qualification, "privacy" in this corpus refers to *the holder's control over what is revealed to whom, when, and how*. It is the aggregate effect of pseudonymity, recipient unlinkability, audit visibility, and (where chosen) cryptographic anonymity. "Privacy" is the noun for the design property; the specific mechanisms above are the means by which it is realised. "*Reasonable privacy*" in operational prose means bank-wire-grade pseudonymity plus recipient unlinkability, not cryptographic anonymity.

Discipline for cross-document use. Other corpus documents (CM, SPX, GOV, CON, IA, EA, Highlights, ATD) defer to these definitions. Claims framed as "private" in body prose mean pseudonymous + unlinkable, never cryptographically anonymous, unless the 7D path is explicitly named. Claims framed as "anonymous" or "cryptographically anonymous" reference the 7D path only. Claims framed as "audit visibility" or "selective disclosure" reference the CM-16 epoch-bounded view-key construction only.

Parameter references

Specific numeric values — block time, Safety Control Window duration, fee tier percentages, genesis allocation amounts, emission constants — are defined in the Scriptum System Parameters document (SPX). This whitepaper references named parameters where relevant without reproducing their values, so that governance changes to SPX do not require whitepaper revisions.

System Intent

Scriptum is a general-purpose settlement chain architected around custody-grade safety assumptions. It serves the full spectrum of value-transfer needs — from small everyday payments to large institutional settlements — with the same chain, the same primitives, and the same fee schedule that scales fairly across all of it. The chain does not specialise; it provides the foundation, and the user picks the appropriate primitive for each transaction. The safety assumptions are constant across the spectrum: every transaction inherits the Safety Control Window, post-quantum cryptography, and the consensus disciplines that custody-grade use requires.

Trading in both directions, on-chain primitives across the whole rail spectrum.

Scriptum provides bidirectional liquidity through cryptographically enforced primitives without requiring custodial intermediaries. Aurum Market handles direct peer-to-peer trades with reputation as the trust layer. Horkos handles attested off-chain settlement for fiat wires, physical goods, services, and complex deliveries — three-party covenant with stake-backed enforcement. Ás-brú handles fast-rail exchange bridges — two-party covenant, miner-monitored, no exchange cooperation required. Standard SPEND (in three modes — direct, private-from-hot, private-from-cold) handles peer-to-peer transfers across all sizes. Holders move Scriptum in or out of fiat, crypto, and goods/services through

whichever primitive fits the rail; the protocol is versatile by composition rather than by special-casing.

Tiered privacy with opt-in cryptographic anonymity. Bank-wire-grade pseudonymity is the default for every transaction — stealth-derived receive addresses via Kyber/ML-KEM, view-key-delegated audit, sealed sender memos. This is sufficient for ordinary commercial settlement: chain-only adversaries cannot link senders to recipients, and holders retain full audit control through view-key disclosure. Holders who require cryptographic anonymity for specific transfers — journalist payments, sensitive donations, confidential business — route through 7D's fixed-denomination ring proofs, where STARK-based proof of leaf membership in a denomination tree provides genuine credential-set unlinkability. The privacy is opt-in per transaction, not chain-wide policy. The graduated stack (HOME SPEND / NEXUS / 7D) lets holders pick the tier appropriate to each transaction; the chain provides all three and is honest about what each delivers.

Safety as a default, usability as a discipline. Every cancellable path passes through NEXUS — value is observable for 24 hours and recoverable via CANCEL until eject. Theft of a key does not mean theft of value; the holder, an observer, or an automated monitor can submit CANCEL during the dwell window. The Aurum wallet streamlines safety into the workflow rather than against it: one-click sends from ACTIVE for fast operational use, automatic cross-address consolidation via Frau Holle on Menuha, in-flight transaction monitoring that surfaces unauthorised UNLOCKS immediately. Safety is not a friction tax. The protocol enforces structural protection; the wallet makes the protection invisible to compliant users while remaining mandatory at the consensus layer.

Institutional-grade auditability with cryptographic boundaries. View-key delegation gives holders cryptographic control over who can see their on-chain history. Custodians, accountants, regulators, and counterparties can be given precisely the audit scope the holder chooses — per-transaction (via Sealed Sender Memo disclosure), per-epoch (via epoch-bounded view-key handoff per CM-16), or full-portfolio (via multi-epoch handoff covering the periods of interest) — with the chain's primitives enforcing the boundary cryptographically. Selective disclosure is the design pattern. Subpoena power is acknowledged as outside the chain's threat model; holders compelled to disclose can do so with cryptographic proof of completeness, and parties not in the audit scope cannot read the disclosed material. **At v14 the chain's compliance posture is pseudonymity + bounded selective disclosure; this makes the v14 chain compatible with MiCA, FATF Travel Rule, and equivalent frameworks while preserving holders' privacy against parties outside the audit scope. The planned 7D layer (Scriptum V2, post-Genesis; see §7D glossary) will add cryptographic-anonymity transactions that fall outside the standard view-key audit boundary because their sender is hidden by STARK ring-membership proof rather than pseudonymous on-chain identity. When 7D ships, regulated CASPs serving Scriptum holders may be required to apply enhanced due diligence to 7D-derived flows under prevailing AML/CFT regulation; the chain's role at v14 is to commit to the architectural separation between pseudonymous-with-selective-disclosure operations (v14 baseline) and cryptographic-anonymity operations (V2 extension), so that the regulatory framing remains tractable as the chain evolves.**

Every movement of value in Scriptum is intentional, observable, and — until confirmed — reversible. This is not a limitation; it is the design. Scriptum is built for the secure storage and deliberate movement of value in environments where failure is expected and must be survivable. It is general-purpose; the post-quantum and Safety Control Window foundation gives every application access to structural properties — cancellable settlement where useful, audit-scoped disclosure where required, and the planned route to cryptographic anonymity through 7D when V2 ships — that other chains cannot offer.

Core Principle

Ownership does not imply immediate control. Control requires intent, security, and time.

Problem Statement

Existing blockchain systems are built on the assumption that value transfer should be immediate upon authorization. This model assumes private keys remain uncompromised, users act without error, authorization is always intentional, and cryptographic primitives remain permanently secure. These assumptions do not hold in adversarial conditions.

Structural Weakness of Immediate Execution

In current systems, authorization and execution are coupled into a single step. Any valid signature results in immediate exposure of value. There is no separation between intent and execution, no opportunity to observe or intervene. Security depends entirely on preventing failure — not surviving it.

Consequences

Compromised keys result in instantaneous and irreversible loss

Phishing or user error leads to permanent loss of funds

Automated exploits can drain wallets in seconds

Coercion scenarios offer no recovery window

Future cryptographic breakthroughs may invalidate existing protections without mitigation

Limitation of Existing Mitigations

Timelocks, multisignature schemes, smart contract vaults, and custodial withdrawal delays share a common limitation: they are optional, locally defined, externally enforced, and can be bypassed or misconfigured. None alters the fundamental property of the system: that valid authorization enables immediate execution.

The Core Problem

The core problem is not insufficient cryptography. The core problem is that authorization is treated as equivalent to execution. This creates a system where failure is binary, recovery is impossible, and time plays no role in security.

Design Requirement

A secure value system must assume that authorization can be compromised, users can make errors, and attackers can obtain valid signing capability. Security must not rely on preventing failure, but on controlling execution.

Scriptum Approach

Scriptum separates authorization from execution. A valid signature is no longer sufficient to move value. Value must be explicitly unlocked, enter the SCW, become ACTIVE, and remain valid throughout the ACTIVE period before it can be transferred.

Core Transition

Traditional systems ask: was the transaction authorized?

Scriptum asks: did the transaction remain valid over time?

Design Philosophy

Scriptum is built on three principles:

Security over convenience

Determinism over flexibility

Control over immediacy

Through its time-gated execution model, abrupt market reactions cannot be acted upon instantaneously. This results in natural dampening of short-term volatility without imposing price control. As the ecosystem grows, progressively larger capital flows are required to produce comparable price movements — increasing market inertia as the system matures. Price changes remain possible at all times, but their immediate impact on executed transactions is reduced.

The Hard Layer — Ownership and State

The hard layer consists of Parcel ownership, state transitions, and the Safety Control Window. These are governed exclusively by deterministic consensus rules. No external system is involved.

Consensus depends only on Parcel state, SCW rules, and deterministic protocol logic

No exchange, oracle, price feed, or external system is required for protocol operation

Ownership is either correct or not — there is no partial correctness

The hard layer must always be correct. The system does not tolerate wrong ownership.

Complexity containment:

Scriptum protocol surface is broad — it includes the SCW, Limbo, ACTIVE windows, PACK/ARC/BOARD, CLASS/ROLE/STATE, V_A, and the UIS. A senior engineer will recognize this as a large state machine. The critical reassurance is this: all complexity is confined to deterministic state transitions. There is no dynamic execution, no contract logic, no Turing-complete layer, and no emergent behavior. Every valid state and every valid transition is enumerated in this document. An implementation is correct if and only if it reproduces these transitions exactly. The system is wide but not deep.

The Soft Layer — Value and Fees

The soft layer consists of the V_A valuation coefficient used exclusively for fee calculation. V_A is consensus-derived — proposed by miners and constrained by deterministic bounds — but it is not externally dictated.

V_A affects only fee magnitude — not ownership, not state, not balances

Anyone may propose V_P values: miners, reference feeds, aggregators

No source is authoritative. No oracle is hardcoded

Even if V_A is wrong or manipulated: all balances remain correct, all ownership is unaffected, system integrity is fully intact. Only fee magnitude is temporarily off — users may wait for convergence.

Manipulation of V_A yields no meaningful economic advantage. To materially influence fee calculation, an attacker would need sustained control over block production across the full smoothing window — approximately 12 hours. The only possible gain is a marginal reduction in transaction fees on a single transaction. This cost-benefit profile renders such attacks economically irrational.

Meaningful fee effects appear only at very large transaction sizes. At that scale, the network is already functioning as intended: it is being used for high-value, deliberate transfers. The cost of manipulating V_A still exceeds any achievable benefit.

Furthermore, the incentive is self-defeating in both directions. A miner who drives V_A upward increases fees, triggering the UIS warning system in Aurum. Users observe the anomaly and defer transactions. Transaction volume falls. Total fee revenue for all miners — including the attacker — decreases. A miner who drives V_A downward reduces fees, increasing volume but reducing per-transaction income. Neither direction produces net gain. The economic equilibrium is honesty.

The system tolerates wrong prices. It does not tolerate wrong ownership.

PART II — HOW

Ilmarinen — System Architecture

Scriptum is a native blockchain system with Proof-of-Work consensus, SHA3-512 proof-of-work mining, Parcel-based Parcel accounting, and a fixed maximum supply of 42,000,000 Scriptum.

Block Parameters

Block time: 10 seconds (target)

Block size: 2 MB at genesis (governance variable, scales with bandwidth)

Capacity: ~17 canonical atomic cycles per second at 2 MB genesis (2,097,152 / ~11,900 bytes per first-touch UNLOCK-NEXUS-EJECT cycle $\approx$ 176 cycles per block $\approx$ 17.6 cycles per second; for bound-address operation at ~8,800 bytes per cycle, throughput rises to ~24/sec). This exceeds the 5 cycles/second design minimum by more than three times. Approximately 1.5 million cycles per day at full block utilisation at genesis. Scriptum provides deliberate value transfer at settlement grade. A private bank, a family office, or a sovereign wealth fund does not need 1,000 TPS. It needs certainty.

Difficulty adjustment: every 2,592 blocks (~7.2 hours). The interval encodes the Great Year — the 25,920-year axial precession of Earth — at second resolution: 2,592 blocks $\times$ 10 seconds = 25,920 seconds per retarget period. Tracked by Babylonian, Egyptian, Hindu, Greek, and Mayan astronomy, the Great Year is the longest cyclic rhythm civilisations have continuously observed. Retargeting at this cadence gives 2,592 samples per period (29% more than Bitcoin's 2,016) — better noise smoothing — with adjustment fires ~3.3 times per day, frequent enough to track real hashrate changes without being twitchy.

Retargeting is asymmetric: the harder direction (target shrinks; difficulty rises) is clamped to 2 $\times$ per period, the easier direction (target grows; difficulty falls) is clamped to 4 $\times$ per period. The asymmetry is anti-timewarp defence. A coalition can poison the retarget math by drifting block timestamps backward across the retarget window (within MTP and future-bound limits), making the actual elapsed time appear longer than it really was — the math then infers low hashrate and produces an easier target (difficulty falls). With tight harder-direction clamping and looser easier-direction clamping, the attacker's tool (manufacturing a difficulty drop) is capped at 4 $\times$ per period; the chain's recovery tool (raising difficulty when honest hashrate returns) is preserved through the 2 $\times$ harder-direction clamp applied at subsequent retargets.

Block timestamps are u64 milliseconds since Unix epoch. The resolution prevents retarget direction inversion at sub-second block production rates (bootstrap phase, Phoenix renewal cycles, pathological hashrate spikes). Two timestamp consensus rules apply to every accepted block beyond genesis:

- **Future bound.** $\text{block.timestamp} \leq \text{local_clock} + 7,200,000 \text{ ms}$ (2 hours). Accommodates expected miner/validator clock skew (NTP drift, geographic distribution). A block claiming a far-future timestamp is rejected.
- **Median Time Past (MTP).** $\text{block.timestamp} > \text{median}(\text{last } 2,592 \text{ block timestamps})$. For chains shorter than the window, uses all available history. The window matches the difficulty retargeting interval — same number, same Great Year, design coherence. Prevents miners from drifting timestamps backward to game retargeting.

Absolute target bounds: MIN_TARGET floors the difficulty (prevents chain halt at unmineable difficulty) and MAX_TARGET ceilings it (sanity bound at trivial difficulty). Retargeting math is overflow-safe — multiplication that would overflow u128 clamps to MAX_TARGET rather than producing wrong-direction results.

Coinbase reward bound: the coinbase output must exactly equal $R_0 \times (1 - S_{\text{net}} / S_{\text{max}})$ per the emission formula (subject to block-fee additions). Miners cannot claim arbitrary rewards; the chain enforces the exact protocol formula on every block.

Block size and transaction count limits: BLOCK_SIZE_MAX is the Curia-governance variable (2 MB at genesis); MAX_TX_PER_BLOCK is a consensus-enforced upper bound preventing resource-exhaustion attacks even if block size were governance-raised. Both are enforced at block validation; violations reject the block.

Ragnarok block height: RAGNAROK_HEIGHT = 315,360,000. Defined in blocks, not calendar time: 8,640 blocks/day x 365 days x 100 years. Leap years are not counted. Block count is the protocol primitive. This is a deliberate choice — the chain counts blocks, not seconds.

Monetary Policy

Maximum supply: 42,000,000 Scriptum (S_{max}). The cap is firm in normal operation but is amendable through the Upgrade Ceremony if governance and the community ratify such a change. The cap is a constitutional commitment, not algorithmic immutability. See §[Cap Amendability] below.

Formula constant: $R_0 = 1$ (fixed forever; no Phoenix-specific recalibration)

Block reward: $r = R_0 \times (1 - S_{\text{net}} / S_{\text{max}})$, where $S_{\text{net}} = S_{\text{mined}} - S_{\text{burned}}$ (all quantities in Scriptum)

S_{net} at Phoenix 1 genesis: 6,720,000 Scriptum (16% of S_{max} — the founding allocations created at Block 0; see §Allocation in §21).

Genesis emission of Phoenix 1: $r_1 = 1 \times (1 - 6,720,000 / 42,000,000) = 1 \times 0.84 = \mathbf{0.84 \text{ Scriptum/block}}$.

Reward asymptotically approaches zero within each Phoenix and never reaches it. Phoenix 1 mines approximately 35.26 million Scriptum across its 100-year lifespan, ending at 99.95% of S_{max} .

Over time within each Phoenix, transaction fees become the primary miner incentive. By Year 50 of Phoenix 1, block rewards are economically marginal compared to settlement-fee revenue. All successor Phoenixes operate as predominantly fee-driven economies from genesis. This transition from mining-driven to fee-driven economics is by design — the chain's institutional revenue mechanism, the 10% Tithe routed to WFC, Funds, RESERVE, and CP, is designed to scale with settlement volume, not with mining emission.

The Scriptum system continues through successive Phoenix cycles. Each Phoenix begins with whatever S_{net} disembarks from the Arc; R_0 remains fixed at 1.

Coinbase — The Miner Reward

The COINBASE transaction is the protocol-driven mechanism by which new Scriptum enters circulation. Exactly one COINBASE output is created per block, paid to the address specified by the successful miner. COINBASE has no signature, no inputs, and is enforced at the consensus layer: any block whose COINBASE output violates the emission formula is rejected by all nodes.

Emission Formula

The block reward r is computed deterministically from current chain state:

$$r = R_0 \times (1 - S_{\text{net}} / S_{\text{max}})$$

$R_0 = 1$ (formula constant, fixed forever, identical across all Phoenixes)

$S_{\text{max}} = 42,000,000$ (cap on circulating supply; amendable only through the Upgrade Ceremony)

$$S_{\text{net}} = S_{\text{mined}} - S_{\text{burned}}$$

At Phoenix 1 genesis, $S_{\text{net}} = 6,720,000$ (the 16% founding allocations). Genesis emission $r_1 = 1 \times (1 - 6,720,000 / 42,000,000) = 0.84$ Scriptum/block.

The formula is asymptotic: as S_{net} approaches S_{max} , r approaches zero but never reaches it. r is always strictly positive, however small. Mining never exhausts within a Phoenix; it slows to asymptote, and BURN events reopen headroom by reducing S_{net} . Successor Phoenixes inherit a different starting S_{net} (whatever disembarks from the Arc manifest), but R_0 does not change. Each successor Phoenix has its own emission curve shaped by the same formula and the same R_0 , scaled to whatever inherited S_{net} is.

Coinbase Transaction

For each block, the protocol creates:

COINBASE:

inputs: none

outputs: one Parcel — amount = r , state = LOCKED, class = CIRCULATING,

role = NONE, address = miner address

fee: none (COINBASE does not pay fees)

The miner claims r + any block-space fees (f_U , f_A , and the 90% miner share of f_S) in the same block.

Maturity

COINBASE outputs must reach 100 confirmations before they can enter the standard CIRCULATING lifecycle (UNLOCK → NEXUS dwell → EJECT → ACTIVE → SPEND). During this maturity period the COINBASE Parcel exists on-chain but cannot be unlocked. This matches the Bitcoin convention and protects against coinbase-reorg exploitation: a miner cannot spend a reward that could be orphaned by a longer competing chain.

Supply Accounting

Every confirmed COINBASE output increments S_{mined} by its amount. S_{mined} is unbounded over the lifetime of a Phoenix — it continues to grow as long as mining continues. $S_{\text{net}} = S_{\text{mined}} - S_{\text{burned}}$ is the quantity subject to the 42M circulating cap; S_{mined} itself is not capped.

At the start of every successor Phoenix (following Ragnarök and Arc migration), S_{mined} and S_{burned} reset to zero for fresh accounting. R_0 remains fixed at 1; the formula $r = R_0 \times (1 - S_{\text{net}} / S_{\text{max}})$ continues to apply unchanged. The starting S_{net} of the successor Phoenix is the total Scriptum value disembarked from the Arc manifest — typically a substantial fraction of S_{max} , since most of Phoenix 1's mined supply has survived as committed Parcels that boarded the Arc. Successor Phoenixes therefore mine smaller absolute quantities than Phoenix 1, with emission curves shaped by the same formula and the inherited starting state. This is the chain's mature steady-state: mining bootstraps Phoenix 1; subsequent Phoenixes are predominantly fee-driven economies, with the 10% Tithe (routed to WFC, Funds, RESERVE, and CP) providing the chain's institutional revenue.

Constraints

A block is invalid if it omits COINBASE, creates more than one COINBASE output, pays an amount other than the protocol-defined r , pays COINBASE to a non-miner address, or attempts to spend the COINBASE output before 100 confirmations. These checks are consensus-enforced and not subject to miner discretion.

Mining Algorithm — SHA3-512

Scriptum uses SHA3-512 as its proof-of-work algorithm. SHA3 (Keccak) is the NIST-standardised successor to SHA-2, designed with a different internal structure (sponge construction) that provides defense in depth against structural cryptanalytic advances. SHA3-512 produces a 512-bit output and is computationally efficient on commodity CPUs, GPUs, and dedicated hardware alike.

This is deliberate. Scriptum does not pursue ASIC-resistance through algorithmic obscurity. The chain accepts that mining will professionalise over time as the asset gains value — that is the normal trajectory of any proof-of-work cryptocurrency. The protocol's commitment is to a transparent, well-studied, standardised primitive rather than to a perpetually-shifting algorithmic moving target. Anyone with a computer can mine Scriptum at launch. As mining hardware specialises, the chain's security scales with hash-rate; the asset gains the cryptoeconomic guarantees that hash-rate buys. The Commodore 64 starts the chain. Industrial miners sustain it.

Node Tiers

Scriptum nodes are not monolithic. The protocol supports four validation tiers, each appropriate to a different operator profile, storage budget, and trust posture. Every tier validates new blocks against consensus rules; what they differ on is what historical state they retain and what queries they can serve.

Tier 1 — Light client. Verifies new block headers and follows the chain tip via SPV-style proofs. Does not maintain a Parcel set. Trusts a remote validating node for state queries (balance lookups, transaction history). Footprint: kilobytes. Suitable for very constrained environments (watch devices, embedded surfaces). The light client is the *only* tier that involves trusting a remote party for state. Aurum does not surface light-client mode by default — the architectural claim “every Aurum user is a node operator” depends on running at least Tier 2.

Tier 2 — Phone node. Maintains the full Parcel set (live Parcel equivalent), full block-header chain, and a sliding window of recent block bodies (typically the last SCW depth — 8,640 blocks $\approx$ 1 day). Validates every new block against consensus rules using the Parcel set plus headers. Cannot serve historical queries beyond the recent-bodies window — for older transactions, queries fall back to Tier 3 or Tier 4 peers. Footprint: target 1–5 GB depending on chain age and Parcel-set density. This is the Aurum default on mobile devices. The phone user is a real validator, not a light client.

Tier 3 — Desktop node. Maintains the full Parcel set, full block-header chain, and a longer pruned window of block bodies (typical: weeks to months of recent history). Validates every new block; can serve recent-history queries to lighter peers. Footprint: $\sim$ 145 GB at chain maturity, scaling with adoption. This is the Aurum default on desktop and laptop devices.

Tier 4 — Sanctum (full archival). Maintains the full Parcel set, full block-header chain, and *every block body since genesis*. Validates every new block; can serve any historical query at arbitrary depth. Required for compliance research, regulatory archival, deep audit, and protocol forensics. Footprint: grows linearly with chain age. This is the institutional / Foundation / research operator tier. Master Aurum builds and standalone Sanctum binaries run at this tier.

Tiers 2–4 are validating tiers: they enforce consensus rules independently and accept or reject every new block on their own determination. Tier 1 is a trusting tier: it follows the consensus declared by the validating peers it queries. The chain's decentralisation property

holds across Tiers 2–4 — every Aurum user on a phone or desktop is a real participant in consensus, not a client of someone else’s validation.

Pruning rules are protocol-relevant. A Tier 2 or Tier 3 node must be safe to drop block bodies older than its retention window — meaning the Parcel set must be self-sufficient for validating new blocks without re-scanning ancient history. Scriptum’s Parcel-set architecture (every output’s current state is fully represented in the Parcel set; spent outputs are removed; the set is the live consensus state) makes this safe. The full block-header chain is always retained — headers are tiny and required for re-validating chain weight under reorganisation.

Multi-Miner Protocol

The Ilmarinen node and the Hayyānu miner communicate over TCP from genesis — there is no in-process channel, no shared-memory shortcut, no Unix-domain socket. The architecture supports many Hayyānu instances connecting to a single Ilmarinen, and many Ilmarinen instances forming the chain network. The TCP-from-genesis decision is deliberate: it means the same wire protocol that scales from a single laptop miner pointed at localhost in development scales unchanged to thousands of industrial miners pointing at validator pools in production. No protocol rewrite at adoption scale.

The wire protocol (scriptum-rpc) defines six message types: JOB_REQUEST, JOB_RESPONSE, SOLUTION_SUBMIT, SOLUTION_ACCEPTED, SOLUTION_REJECTED, and ERROR. Framing is u32 length-prefix with u8 message tags. All transitions and types serialise via canonical to_bytes/from_bytes — the consensus hash commits to wire-format bytes, not to language-runtime Debug strings.

The protocol defines one architecture. Operators scale by running multiple Hayyānu instances against one Ilmarinen (the validator distributes jobs, each with its own miner-bound coinbase address; the first valid solution wins) or by building pool software on top. Pools are application-layer constructs — coordinating job distribution and reward sharing across participating miners — not protocol features. The chain does not know whether a given Hayyānu is solo, part of a small operator’s fleet, or part of an industrial pool. It validates blocks the same way regardless.

Crowglass — the exchange observation daemon used in Ás-brú trade validation — is a component of the Ilmarinen node, not the Hayyānu miner. Miners pull jobs from the validator that have already had Ás-brú considerations included by Crowglass; Hayyānu’s responsibility is solving the proof-of-work, not observing exchanges.

Post-Quantum Cryptography — Kavaca

Scriptum uses the full CRYSTALS suite as its native cryptographic foundation, both components standardised by NIST in 2024:

CRYSTALS-Dilithium3 (ML-DSA) — the signature scheme. Every user-authorized transaction is signed with Dilithium3. Every address is secured against quantum attack

from genesis. Protocol-driven transitions (EJECT, EXPIRE, RELEASE, VEST, RESEAL, SWEEP, COINBASE, DISEMBARK) are consensus-enforced and require no signature.

CRYSTALS-Kyber (ML-KEM) is the key encapsulation mechanism. Present in every Scriptum address from genesis. Kyber provides post-quantum key encapsulation — the cryptographic foundation for stealth addresses, holder-controlled audit keys, and the sealed sender memo carried by every SPEND transaction (see §Privacy Model). The sender encapsulates an ephemeral shared secret to the recipient's published Kyber view key (a single KEM operation, the post-quantum analog of a Diffie-Hellman handshake but technically a one-pass encapsulation rather than a two-party key exchange); the resulting shared secret simultaneously derives the stealth output address and keys an authenticated symmetric encryption of the sender's published address. Two independent privacy properties — receive-side stealth and address-level provenance disclosure to the recipient — are produced from a single Kyber KEM operation. All CRYSTALS components are active at launch; there is no dormant cryptography, no reserved fields awaiting activation, no coordinated upgrade to wait for. Scriptum ships complete.

CRYSTALS-Dilithium3 and CRYSTALS-Kyber eliminate all known classical and quantum attack vectors against Scriptum's signing and key-encapsulation primitives. Like any cryptosystem, they remain subject to future cryptographic advances. If a fundamental flaw is discovered in either primitive, the Governance Charter's hard-fork mechanism is available to migrate Scriptum to successor standards.

SHA3-512 is the chain's single hash function. Every chain-level hash operation — address derivation, Merkle trees, terms hashes, proof-of-work, CANCEL preimages, exchange identifier hashes in Ás-brú trade commits — uses SHA3-512. There is no second hash function, no special-case carve-out for “identifier hashes” or “auxiliary hashes.” The motivation is coherence over micro-optimisation: the byte cost of 64-byte outputs versus 32-byte outputs is rounding error against Dilithium3 signature overhead (signatures are ~3 KB each), while the security and audit story is dramatically cleaner with one function used everywhere. NIST Level 5 post-quantum collision resistance (256-bit against Grover-birthday) applies to every chain-level hash. The CANCEL hash in particular — the safety-critical hash protecting the SCW (Heimdall) against an attacker holding signing keys but not the user's CANCEL preimage — benefits most from full Level 5 security, which is exactly the surface where attack concentration is highest.

Why a Single Primitive, Not Pluggable Crypto

A reasonable reader may ask: why does Scriptum commit to a single cryptographic primitive set rather than supporting runtime-pluggable crypto? Modern systems often offer cryptographic agility — TLS negotiates cipher suites at connection time; libraries like OpenSSL and liboqs are explicitly designed to let applications swap algorithms via configuration. Scriptum deliberately rejects this pattern. The reasoning is structural, not ideological.

Consensus chains require single canonical primitives. A blockchain is a system in which thousands of independent nodes must reach the same answer about every transaction's validity. If one node verifies a signature using Dilithium3 while another

verifies using ECDSA, the two nodes disagree on whether the signature is valid — which means they disagree on whether the transaction is valid — which means they disagree on whether the block containing it is valid. Consensus fails. Pluggable crypto is appropriate for systems where parties negotiate bilaterally (TLS clients and servers establishing a shared cipher). It is structurally incompatible with a system where every party must reach the same answer without negotiation.

Cryptographic agility creates attack surface that single-primitive design eliminates.

Pluggability is sometimes framed as future-proofing — “if Dilithium3 is broken later, swap it out at runtime.” In practice, supporting multiple crypto suites enlarges the attack surface substantially. Downgrade attacks become possible: an adversary forces the system to negotiate down to a weaker algorithm. Implementation bugs multiply across each supported algorithm. The TLS ecosystem has been bitten repeatedly by exactly this — historical attacks such as FREAK, Logjam, and POODLE exploited cipher-suite negotiation rather than weaknesses in any single algorithm. Scriptum’s structural defense is the opposite: one signing algorithm, one key-encapsulation algorithm, fully committed, single audit surface, no negotiation, no downgrade.

The real upgrade path for a blockchain is governance-coordinated hard fork through Menuha, not runtime configuration.

If Dilithium3 is ever broken and Scriptum must migrate to a successor primitive, the chain does not survive by individual nodes flipping configuration flags. The chain survives because the constitutional governance specified in the Governance Charter ratifies the upgrade through Level 4 multi-sig (WOC + 3-of-5 Founders + 7-of-9 Curia), the new node software is published, and the network deploys coordinately during a Menuha window. Menuha is the chain’s weekly 24-hour rest period (§31) — no outward transactions execute, the mempool drains, the protocol performs its own housekeeping via SWEEP. It is the cleanest possible deployment moment: no transactions in flight, no consensus risk during the swap, every operator synchronized to the same boundary, 24 hours of buffer for rolling deployment. When Menuha closes and the next operational week begins, every up-to-date node enforces the new rules simultaneously; nodes that have not upgraded fall out of consensus and stop validating until they do.

This is genuine cryptographic agility — slow, deliberate, governance-mediated, explicit, and structurally appropriate for a consensus system. Pluggable crypto would let individual nodes upgrade unilaterally, which is precisely what consensus systems must prevent. Single-primitive commitment combined with the Menuha-windowed hard-fork mechanism gives Scriptum the agility it actually needs without any of the attack surface or consensus risk that runtime pluggability would introduce. The chain ships complete, with a single canonical post-quantum primitive set, and an upgrade path designed for the rare necessity rather than the routine convenience.

Address Format

Every Scriptum address contains four components:

Dilithium3 public key — active from genesis (signing)

Kyber public key — key encapsulation, active from genesis

View key slot — audit key, active from genesis

Checksum — active from genesis

Addresses are stored on chain as 64-byte address hashes — the full SHA3-512 hash of the key bundle, no truncation. NIST Level 5 collision resistance (256-bit against Grover-birthday) applies to address identity, matching the chain-wide hash function commitment per §Post-Quantum Cryptography. Users see addresses as Bech32 strings with HRP “s1q” per SPX-0 — the user-facing encoding is Bech32 wrapping the 64-byte address with the standard Bech32 BCH checksum. The full key bundle is revealed by the holder on first UNLOCK at each address and carried forward inside the Parcel lineage from that point. Subsequent state changes (NEXUS dwell → ACTIVE → SPEND) inherit the binding automatically; change outputs returning to the same address carry the binding with them; bundled UNLOCKS benefit if any input Parcel already carries the binding. The mechanism is described in §6.6.

Stealth addresses, holder-controlled audit keys, and post-quantum key encapsulation (Kyber/ML-KEM) are available from block zero. No activation event. No reserved fields. No coordinated upgrade. Scriptum ships with the full cryptographic stack live.

The address format is fixed from genesis. Any future structural change requires a hard fork under the Governance Charter.

ESCROW addresses are protocol-controlled wallets used for Horkos. They have no user private key. Funds at ESCROW addresses move only through the protocol-driven resolution sequence triggered by attestation outcome or timeout — UNLOCK followed by EJECT followed by SEND, executed by the protocol rather than by user signatures. Neither the parties (A, B), nor the attestor (C), nor the Foundation, nor Curia can move an ESCROW Parcel discretionarily. Routing is determined entirely by the outcome of the trade per the agreement signed at the start of the contract. ESCROW addresses carry full post-quantum binding structure identical to user addresses; they are first-class Scriptum addresses, differing only in that no one holds the private key.

Public Key Binding at First UNLOCK

Post-quantum cryptography has a throughput problem. NIST-standardised lattice signatures (Dilithium3) and lattice key-encapsulation (Kyber768) provide the security guarantees the next century requires, but at a cost: a Dilithium3 public key is ~1,952 bytes, a Kyber public key is ~1,184 bytes, and a Dilithium3 signature is ~3,293 bytes. A naive lattice chain that reveals the full bundle in every signed transaction pays roughly five to six kilobytes per transaction in cryptographic overhead alone — an order of magnitude more than ECDSA. Without an architectural answer, this overhead caps a 2 MB / 10-second chain at a few transactions per second; with one, it lands in the 17–24 canonical settlement cycles per second range Scriptum targets. The architectural answer is binding via address identity. It is what makes Scriptum’s choice of NIST-standard post-quantum primitives

operationally viable at settlement-grade throughput rather than a research-grade demonstration.

The binding is not caching. It is deduplication via address identity. The address is the 64-byte SHA3-512 hash of the bundle (Dilithium3 public key, Kyber768 public key, view key, flag bits); the address therefore commits to the bundle cryptographically at NIST Level 5 collision resistance. The user-facing Bech32 encoding (HRP “s1q”) wraps the same 64-byte address with the Bech32 checksum for transcription safety; the consensus address identity is the 64-byte hash itself. Every Parcel at that address shares that commitment. The binding is serialized once — at first UNLOCK — and carried by the Parcel thereafter. It is not re-serialized in subsequent transactions because the address itself is the reference. Post-quantum public keys are large. A Dilithium3 public key is approximately 1,952 bytes. A Kyber public key is approximately 1,184 bytes. Revealing the full bundle in every signed transaction would be prohibitively expensive. Scriptum avoids this by revealing the public key once per address, at the first UNLOCK, and propagating the binding through the Parcel lineage thereafter. The pattern of revealing public keys at first spend is well-established (Bitcoin, since 2009); the architectural contribution is its application to lattice cryptography at the scale where it matters — the difference between a chain that runs at single-digit transactions per second and one that runs at settlement-grade throughput. Among lattice-signature chains, Abelian reveals the full pubkey per transaction and pays the full bandwidth cost; QRL avoids the problem at the cost of stateful one-time XMSS signatures. Scriptum chooses the third path: large pubkeys, revealed once, structurally bound to the address. Any remainder output returning to the originating address carries the originating binding; remainders never re-reveal and never lose the binding.

Mechanism

At the first UNLOCK transaction consuming a LOCKED Parcel at a given address, the transaction reveals the full public key bundle. The chain verifies that the hash of the revealed bundle equals the address. The resulting NEXUS-dwelling Parcel carries the bundle as part of its data. Every state-change Parcel thereafter (EJECT → ACTIVE, CANCEL → LOCKED, EXPIRE → LOCKED, LOCK → LOCKED, REFRESH → ACTIVE) inherits the bundle by descent. Change outputs from a SPEND returning value to the sender’s own address carry the bundle forward; outputs going to a different address do not.

An UNLOCK transaction may consume input Parcels from multiple distinct addresses simultaneously. This is the consolidation property that makes mandatory stealth-compatible output construction operationally viable — without it, a holder accumulating Parcels across many one-time stealth-derived addresses would be unable to consolidate them efficiently. The protocol verifies a Dilithium signature for each distinct source address represented in the inputs, using each address’s bundle (revealed on first touch or inherited from prior history per §6.6). The transaction is valid if and only if every source address’s signature verifies against its bundle. The single output NEXUS-dwelling Parcel is destined for the holder’s HOME address. Any LOCKED remainder is also delivered to HOME. The fee is one f_U on the summed unlocked value, regardless of input count or source address count. See §Privacy Model for how this property composes with stealth-driven address fragmentation and Frau Holle’s cross-stealth-address consolidation during Menuha.

An UNLOCK transaction is valid with or without re-revealing the bundle, governed by the following rule: if at least one input Parcel already carries the bundle for the address, the transaction may omit it and reference the inherited binding; if no input carries the bundle, the transaction must reveal it. The chain verifies the signature against the bundle — whichever way it arrives — and rejects mismatches.

Every SPEND in Scriptum is stealth. The sender performs a Kyber/ML-KEM encapsulation against the recipient's Kyber public key (carried in the recipient's binding bundle, which the sender obtains from the recipient's published address), producing a fresh ephemeral shared secret. The result is a one-time output address unique to that transaction. The SPEND transaction includes the encapsulation ciphertext (~1,200 bytes of ephemeral key material) so the recipient, scanning the chain with their view key, can decapsulate the same shared secret and recognise the output as theirs. This mechanism is not optional. There is no plain-mode SPEND. Every settlement is private by default; selective disclosure is enabled only through holder-authorised audit keys (see §The Audit Key and Appendix A).

Validation

A validating node processes an UNLOCK as follows. Inspect the input Parcels and group them by source address. For each source address represented in the inputs: verify that either an input carries the bundle for that address (revealing it as first-touch) or the address already has its bundle bound on chain from a prior UNLOCK (referencing inherited binding per §6.6); verify a Dilithium signature against the bundle for that address. The transaction is valid if and only if every source address's signature verifies against its bundle and the destination address (the holder's HOME) is correctly committed in the transaction. The output NEXUS-dwelling Parcel is destined for HOME, carrying HOME's bundle (revealed at HOME's first-touch or inherited if HOME has been used before). Any LOCKED remainder is also delivered to HOME with the same bundle. No external lookup, oracle, or registry is required. Validation is deterministic from the transaction and the consumed Parcels.

The binding is secured by hash commitment. A valid binding for address X is the unique bundle whose hash equals X; any mismatch produces a different address and is rejected by consensus. Forging a binding requires finding a collision against the address hash — computationally equivalent to breaking the underlying hash function. Once a binding is revealed on chain, it is public information; security depends on the private key remaining with the holder, not on the bundle being secret.

Design Properties

Immutability. The bundle is fixed by the address hash. Any attempt to substitute a different bundle produces a different hash, a different address. Ownership cannot be transferred by binding substitution.

Scope. Binding is per address. Each address has exactly one valid binding — the bundle whose hash equals it. The binding lives in Parcels descended from the first UNLOCK at that address; there is no global registry, no chain-wide pubkey table, and no cross-address linkage.

Path propagation. The bundle travels with value through state changes and through change outputs. An active holder of an address with a history of use pays the full revelation cost once per address — not once per Parcel, not once per transaction.

Bundling efficiency. An UNLOCK consuming N input Parcels at a single address pays the bundle cost once if any input already carries it. Freshly arrived Parcels at the same address can be absorbed into a bundled UNLOCK without each paying the revelation cost independently.

No global state. The bundle lives in the Parcel itself, not in a side-channel cache or registry. Full, pruned, and archival (Sanctum) nodes all maintain the Parcel set; the binding is automatically available wherever the Parcel is available. Reorg behavior is natural: if a first-UNLOCK is rolled back, the binding rolls back with the Parcel that carried it.

Reveal cost is paid once. The full bundle is revealed exactly once per value path at a given address. After the first UNLOCK, all operations on that value path are cheap. A new Parcel arriving at the same address does not retroactively carry the binding, but a bundled UNLOCK consuming it alongside a bound Parcel absorbs it into the existing binding chain at no extra cost.

Rationale

Without this mechanism, every signed operation on a post-quantum Parcel chain would pay the full public-key revelation cost, and settlement throughput would be dominated by redundant key material. Consolidation — which is unavoidable for any active holder, and mandatory for miners accumulating block rewards — would scale linearly with Parcel count. With this mechanism, the post-quantum cost is localised to the first-touch event at each address. The chain pays the post-quantum tax once, at introduction, not every time value moves.

Temenos — The Parcel Model

Attribute Model

Scriptum represents all value as Parcels defined by three orthogonal attributes:

CLASS — defines the economic identity of the value

ROLE — defines its affiliation or governance context

STATE — defines its current operational behaviour

CLASS

CIRCULATING

Ordinary user-controlled value. The standard economic class. Outward sends follow the lifecycle: LOCKED → NEXUS dwell → LOCKED (at recipient stealth address). The migration lifecycle is: LOCKED → NEXUS dwell → ACTIVE → PACKING → ARC.

TREASURE — Bastet

Governed allocations assigned to constitutional roles (Warden, Founders, Curia). Defined by address-bound ownership, a deterministic vesting schedule, and lifecycle persistence across Phoenix transitions. Not immediately spendable. At each vesting milestone, the protocol fires RELEASE: VESTING Parcel consumed, LOCKED Parcel created with CLASS = CIRCULATING, ROLE = NONE. The holder then proceeds through the standard CIRCULATING lifecycle.

At Ragnarok, all Parcels that remain CLASS = TREASURE board the Arc automatically — protocol-driven, regardless of vesting state. TIME_GUARDED and VESTING TREASURE is carried into the successor Phoenix with its remaining vesting schedule intact. No TREASURE-class Parcel is lost at Ragnarok. Bastet crosses with it.

Important distinction: TREASURE that has fully vested and converted to CLASS = CIRCULATING is no longer TREASURE-class. Those Parcels are the holder's ordinary circulating value and must be enrolled manually like any other CIRCULATING Parcel. Bastet's responsibility ends at RELEASE.

VAULT

Protocol-controlled institutional value assigned to RESERVE, WFC, FUNDS, and CP roles. Migration is protocol-driven and automatic, not user-initiated. Follows: SEALED → OPEN → SEALED (if unused) or SEALED → OPEN → governance action → SEALED (remainder). OPEN lasts 8,640 blocks (~24 hours). Any unspent value re-seals automatically. For migration: SEALED → OPEN → PACKING → ARC.

MARKER

A non-economic, non-spendable class used for permanent protocol identity records. Cannot be unlocked, spent, or burned. Re-instantiated unchanged at every Rebirth. Excluded from all economic calculations. Two types exist:

Mayflower — assigned to founding participants at Genesis. Represents permanent proof of participation in the founding event (ARC 0). Exactly one per founding participant. Cannot be created after ARC 0.

Dana — issued by the protocol on confirmation of a DONATE transaction. Represents permanent proof of a voluntary donation to the protocol. Exactly one per donation transaction. Private by default. Survives every Ragnarök.

ROLE

Roles define the governance affiliation of a Parcel:

NONE — standard circulating value, no governance context

WOC — Warden of the Chain (governance defined in Scriptum Governance Charter)

FOUNDER — founding team allocation (governance defined in Scriptum Governance Charter)

CURIA — governing council allocation (governance defined in Scriptum Governance Charter)

RESERVE — strategic long-term protocol reserve (10% of Tithe; release requires Constitutional governance)

WFC — Wizard / Founders / Curia compensation pool (48% of Tithe; distributed to constitutional position-holders)

FUNDS — Foundation operational allocation (32% of Tithe; sourced from Curia-authorized disbursements)

CP — Charity Pot (10% of Tithe + all WFC cap-overflow; humanitarian allocation)

MAYFLOWER — founding participant marker (CLASS = MARKER only)

DANA — voluntary donor marker (CLASS = MARKER only)

STATE

State defines the operational behaviour of a Parcel. States are class-dependent.

CIRCULATING:

For consolidation (UNLOCK to HOME):

LOCKED (stealth-derived addresses) → NEXUS (dwell) → ACTIVE at HOME

For direct outward sends:

ACTIVE at HOME → LOCKED (at recipient's stealth-derived destination, immediate)

For private outward sends from hot:

ACTIVE at HOME → NEXUS (dwell) → LOCKED (at recipient's stealth-derived destination)

For private outward sends from cold:

LOCKED (stealth-derived addresses) → NEXUS (dwell) → LOCKED (at recipient's stealth-derived destination)

Or, terminating without onward send:

ACTIVE → LOCK / BURN / EXPIRE → LOCKED at HOME / Gehenna / LOCKED at HOME

Migration path:

LOCKED → NEXUS (dwell) → ACTIVE at HOME → PACKING → ARC

NEXUS provides the cancellable SCW window for any transition the holder wants protected: UNLOCK to HOME (consolidation), private SPEND from ACTIVE (cancellation)

safety on hot outward sends), private SPEND from LOCKED (institutional cold-spend). EJECT is the protocol-driven transition out of NEXUS, firing at unlock_height + 8,640. Direct SPEND from ACTIVE bypasses NEXUS — ACTIVE Scriptum has earned its hot status through the unlock-side SCW. The choice between direct and private SPEND modes is the holder's per transaction.

TREASURE:

TIME_GUARDED → VESTING → RELEASE → LOCKED (CIRCULATING) → PACKING → ARC

VAULT:

SEALED → OPEN (8,640 blocks) → SEALED (no action taken)

SEALED → OPEN → [governance action] → SEALED (remainder re-seals)

SEALED → OPEN → PACKING → ARC (migration only)

MARKER:

PERMA_LOCKED (permanent, no transitions)

Migration States

PACKING and ARC are protocol-defined migration states that apply across all classes. They suspend normal economic behaviour, preserve class and role identity, and exist exclusively for inter-Phoenix transport.

PACKING

A consolidated pre-migration state representing the sum of all wallet Parcels at the time of consolidation. Non-spendable. May be reversed prior to boarding. Created by the PACK transaction approximately 24 hours before the selected boarding slot.

ARC

A terminal migration state. Non-spendable. Cannot transition to any other state within the originating Phoenix. Reserved exclusively for migration into the successor Phoenix. Reached through the BOARDING transaction. Irreversible.

Migration Authority

VAULT — protocol-driven (automatic). All VAULT boards at Ragnarok regardless of state.

TREASURE — protocol-driven (automatic). All CLASS = TREASURE Parcels board at Ragnarok regardless of vesting state (TIME_GUARDED or VESTING). Vesting schedule is preserved in the successor Phoenix.

CIRCULATING — enrollment is user-initiated. PACK is user-initiated. BOARD is protocol-driven after CANCEL_THRESHOLD.

State Definitions

Default State

All outputs are LOCKED by default. This applies universally to mined coins, received funds, and change outputs.

LOCKED

Non-spendable. Can be used as input in one transaction only: UNLOCK.

NEXUS (formerly UNLOCKING)

Initiated by UNLOCK or by a private-mode SPEND (from ACTIVE or from LOCKED). Value enters the NEXUS dwell-state. Dwell duration: 8,640 blocks (24 hours at 10-second block time) — the SCW. Non-spendable and observable. CANCEL is valid throughout the dwell window via either signature-based CANCEL (signed by the source address's Dilithium key) or hash-preimage CANCEL (preimage of an optional hash $X = H(S)$ committed in the entering transaction, see §[Transaction Model]).

Hard cutoff: once the dwell window closes and the protocol-determined eject slot is reached, NEXUS ejects the value to its committed destination — HOME for UNLOCK entries, or the recipient's stealth-derived destination for private-SPEND entries. CANCEL submitted after eject is permanently invalid.

The eject for each entering Parcel fires exactly 8,640 blocks after entry (24 hours at 10-second block time). Every validating node computes the same eject_height deterministically from chain state. There is no per-entry randomness; eject timing is uniform across all NEXUS-dwelling Parcels at the dwell duration.

The eject is a protocol-driven state transition (in the same category as ACTIVATE in earlier specifications). It does not require the originating holder's signature; it executes deterministically based on the parameters committed in the original signed UNLOCK transaction and the chain's consensus rules. The security model is identical to ACTIVATE: the holder's UNLOCK signature commits to all parameters; the protocol-driven transition produces ACTIVE Parcel at the holder's HOME address at unlock_height + 8,640.

CANCEL race: if an attacker initiates UNLOCK on a compromised key, the legitimate owner has the full dwell window to respond. CANCEL consumes the NEXUS Parcel — there is only one. The first confirmed CANCEL wins. Once confirmed, the NEXUS Parcel is destroyed. There is no cancel-the-cancel. The attacker position is dead.

*Limbo is the waiting interval between eject eligibility and actual execution — milliseconds under normal load (the time until the next block slot accommodates the protocol-driven eject), seconds to minutes under congested load as the priority queue drains across subsequent blocks (see §**Protocol-Driven Transitions and Limbo***). During Limbo, CANCEL is invalid because the dwell window has technically closed. Wallets must display LIMBO —*

****AWAITING EJECT · NO ACTION POSSIBLE.** Aurum surfaces an additional reassurance notification if Limbo persists beyond approximately ten seconds.*

ACTIVE

Spendable for 8,640 blocks (~24 hours) from activate_height — the actual block of NEXUS eject for self-spends. The holder always receives a full 8,640-block ACTIVE window regardless of any Limbo duration.

Five outcomes are available for an ACTIVE Parcel:

SPEND(x) — transfer x Scriptum to external recipient. Fee = f_S on amount.

LOCK(x) — re-lock x Scriptum to the originating address. Fee = 0. Coin selection. Remainder stays ACTIVE.

BURN(x) — send x Scriptum to Gehenna permanently. Structurally SPEND with destination = BURN_ADDRESS; standard SPEND fee per SPX-4; coin selection per CM-3.3; remainder returns to HOME as change. The amount sent to Gehenna increments S_burned and is permanently removed from spendable circulation.

REFRESH — extend the ACTIVE window. The holder submits a small signed refresh transaction; the protocol resets the 8,640-block clock to start from refresh_height. The holder may refresh as many times as they want; each refresh starts a fresh 24-hour window. Fee = f_refresh (governance-set, expected to be near-zero to encourage active engagement without economic burden).

WAIT — do nothing. Protocol applies EXPIRE at activate_height + 8,640 (or refresh_height + 8,640 if a REFRESH was the most recent action). Fee = 0.

WAIT is the default. **The protocol always returns funds to LOCKED if the user takes no action.** The chain rewards active engagement; passive sitting returns to safe storage automatically. Inactive holders' ACTIVE balances do not stay hot indefinitely, limiting the window during which a compromised wallet could drain them. Active holders can keep balances ACTIVE as long as they need by refreshing. The chain is not a luxury hotel.

For SPEND, LOCK, and BURN: coin selection applies across available ACTIVE Parcels. Remainder Parcels are created at the originating address in the ACTIVE state with expiry = minimum of all consumed inputs. No operation other than REFRESH resets or extends the ACTIVE validity period of any remainder output.

PERMA_LOCKED (MARKER)

A permanent non-spendable state exclusive to CLASS = MARKER Parcels. No transitions are possible. PERMA_LOCKED Parcels are excluded from all economic calculations, never included in S_net, and always re-instantiated unchanged at every Rebirth.

OPEN (VAULT)

Released from SEALED by protocol authority following a valid governance action. OPEN lasts 8,640 blocks (~24 hours). During OPEN, the VAULT may execute a governed disbursement or enter PACKING for migration.

At expiry of the 8,640-block window: if no disbursement was executed, the VAULT re-seals automatically (OPEN → SEALED). If a partial disbursement was executed, the remainder re-seals. OPEN is never permanent. Value cannot remain in an OPEN VAULT indefinitely.

This mirrors the ACTIVE window for CIRCULATING Parcels — the protocol always returns value to a protected state if no action is taken.

ARC

A deterministic terminal state representing value committed to migration. Non-spendable. Cannot transition to any other state within the originating Phoenix. Reached exclusively through BOARDING. Irreversible. ARC defines the boundary between value that continues and value that remains.

TIME_GUARDED and VESTING (TREASURE) — Bastet

TIME_GUARDED: treasure exists, vesting not yet begun, non-spendable. Distinct from VAULT SEALED — TIME_GUARDED transitions by deterministic schedule, not protocol authority. VESTING: undergoing deterministic vesting schedule, non-spendable, conditions accumulating block by block.

At each vesting milestone (block height = x), the protocol fires RELEASE: the VESTING Parcel is consumed and a new LOCKED Parcel is created with CLASS = CIRCULATING, ROLE = NONE. The holder then proceeds through the standard CIRCULATING lifecycle: UNLOCK → NEXUS (dwell) → EJECT → ACTIVE → SPEND.

Bastet is the symbolic guardian of TREASURE across SEALED and VESTING. She watches the schedule. At each milestone she steps aside. After RELEASE, it is simply a coin.

State Transitions

CIRCULATING Lifecycle

The chain provides four operational paths for value movement:

Path 1 — UNLOCK to HOME (consolidation): LOCKED (across stealth-derived addresses) → UNLOCK → NEXUS (dwell) → EJECT → ACTIVE at HOME

Path 2 — Direct SPEND from hot: ACTIVE at HOME → SPEND → LOCKED at recipient stealth-derived destination (immediate, no NEXUS)

Path 3 — Private SPEND from hot: ACTIVE at HOME → SPEND (private mode) → NEXUS (dwell) → EJECT → LOCKED at recipient stealth-derived destination

Path 4 — Private SPEND from cold: LOCKED (across stealth-derived addresses) → SPEND (private mode, from-cold) → NEXUS (dwell) → EJECT → LOCKED at recipient stealth-derived destination

Other ACTIVE-state transitions:

ACTIVE → LOCK(x) → LOCKED at HOME + ACTIVE remainder

ACTIVE → BURN(x) → GEHENNA + ACTIVE remainder

ACTIVE → REFRESH → ACTIVE (clock reset)

ACTIVE → WAIT → EXPIRE → LOCKED at HOME

NEXUS → CANCEL → LOCKED at source addresses (returns to pre-entry state)

ACTIVE → PACK → PACKING

PACKING → BOARD → ARC

UNLOCK and private SPEND both pass through the NEXUS dwell window for the SCW protection. Direct SPEND from ACTIVE at HOME is the fast operational path — the holder has actively brought value into ACTIVE state through UNLOCK (which passed through NEXUS at the entry to hot status) and has been continuously maintaining the hot status through periodic REFRESH actions. Direct SPEND is the operational use of value the holder has deliberately maintained in active management; it is not a bypass of NEXUS so much as the natural outflow from value that has earned and continuously reaffirmed its hot status. Holders choose between direct SPEND (operational use of actively-maintained value) and private SPEND (re-enter NEXUS for an additional cancellation window) per transaction. EJECT is the protocol-driven transition out of NEXUS, executed at `unlock_height + 8,640`. See §NEXUS — The State-Transition Crucible in the Privacy Model section for the full specification.

TREASURE Lifecycle — Bastet

TIME_GUARDED → VEST → VESTING → RELEASE → LOCKED (CIRCULATING)

LOCKED → UNLOCK → NEXUS (dwell) → EJECT → ACTIVE → SPEND / LOCK / BURN / REFRESH / WAIT

Arc procedure (protocol-driven — ALL TREASURE boards automatically):

TIME_GUARDED → [protocol] → PACKING → BOARD → ARC (schedule preserved in successor)

VESTING → [protocol] → PACKING → BOARD → ARC (remaining schedule preserved)

LOCKED → user enrollment → PACK → BOARD → ARC (now CIRCULATING, user-initiated)

VAULT Lifecycle

SEALED → OPEN (protocol, 8,640 blocks) → SEALED (no action — auto re-seals)

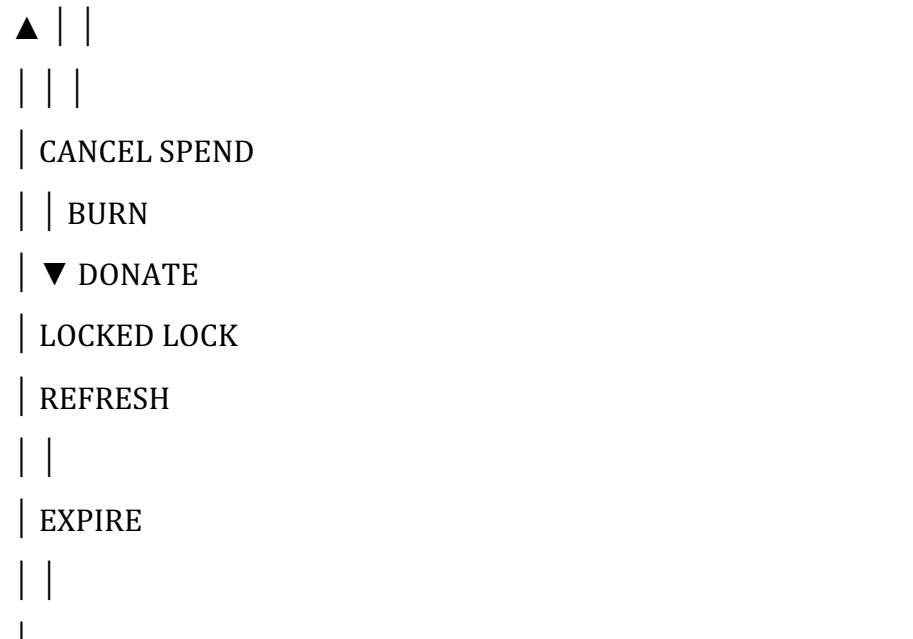
SEALED → OPEN → [disbursement] → SEALED (remainder re-seals)

SEALED → OPEN → PACKING → ARC (migration)

Lifecycle Reference Diagram

The CIRCULATING lifecycle in condensed form, showing user-initiated transactions and protocol-driven transitions:

LOCKED — UNLOCK —  NEXUS — EJECT —  ACTIVE



Each transition fires in one direction only. UNLOCK enters the NEXUS dwell-state and opens the Safety Control Window; EJECT closes the dwell-state and produces ACTIVE at the holder's HOME address. SPEND, BURN, DONATE, and LOCK consume ACTIVE Parcels at HOME. Direct SPEND lands as LOCKED at the recipient's stealth-derived address with no NEXUS dwell. Private SPEND modes (from ACTIVE or from LOCKED) route through NEXUS for the cancellation window. REFRESH extends the ACTIVE window. EXPIRE returns unused ACTIVE Parcels to LOCKED at HOME. CANCEL reverses a NEXUS-dwelling Parcel back to its source state within the window (via signature or hash-preimage). All protocol-driven transitions (EJECT, EXPIRE) fire deterministically — EJECT at unlock_height + 8,640, EXPIRE at activate_height + 8,640 (or refresh_height + 8,640).

TREASURE and VAULT classes follow related but distinct lifecycles. TREASURE advances TIME_GUARDED → VESTING → CIRCULATING LOCKED through monthly RELEASE events after the vesting cliff. VAULT advances SEALED → OPEN (by governance action) →

CIRCULATING LOCKED through disbursement, with RESEAL returning unused VAULT to SEALED after the open window.

The full formal state machine — every valid (CLASS, ROLE, STATE) tuple, every transition, every precondition — is specified in the Canonical Machine.

SCW — The Safety Control Window and NEXUS

The Safety Control Window (SCW) is the protocol-enforced delay between transaction instruction and irreversible settlement during which an unauthorised movement can still be cancelled. It is the chain's core protective primitive. NEXUS is the named state-transition machinery that implements the SCW.

This section specifies the SCW mechanics: the NEXUS dwell-state, the CANCEL authorisation paths (signature and hash-preimage), the ACTIVE state with REFRESH switch and default-to-LOCK semantics, the protocol-driven EJECT, and the four operational paths the chain provides.

NEXUS — The State-Transition Crucible

NEXUS is the protocol-level state-transition machinery that provides cancellable controlled-transfer windows for the chain's four operational paths. It is the named architecture of the Safety Control Window. NEXUS handles three of the chain's four paths (UNLOCK to HOME, private SPEND from hot, private SPEND from cold). The fourth path — direct SPEND from ACTIVE — operates outside NEXUS as the operational use of value that has already passed through NEXUS at UNLOCK and is being actively maintained in hot status by the holder through periodic REFRESH.

NEXUS is not a mixer, not a churn pool, and not a provenance-fog layer. The chain's privacy comes from stealth derivation, identity compartmentalisation, the holder's disciplined wallet behaviour, and the structural property that NEXUS-driven EJECTs are signed by the protocol rather than by Alice — not from NEXUS as an anonymising primitive. NEXUS provides three specific properties on every cancellable transition:

Cancellable dwell-time. During the NEXUS dwell window, the originating holder may CANCEL the transition, returning value to its source addresses. The dwell duration is exactly 24 hours (8,640 blocks at 10-second block time). After the dwell window closes, NEXUS ejects the value to its committed destination.

NEXUS provides bank-wire-grade pseudonymity, not anonymity. Every NEXUS-dwelling Parcel carries on-chain references that link it to its origin and (after EJECT) to its destination. The Safety Control Window protects holders from theft and provides cancellation; it does not break sender-recipient linkability on the chain graph. NEXUS is the bank-wire-grade settlement tier of Scriptum's privacy stack: stealth-derived addresses provide pseudonymity, view-key delegation provides selective audit, and the cancellable dwell provides safety. Cryptographic anonymity — unlinkability that survives compelled disclosure — is provided by the 7D denomination-pool layer (see §7D Anonymity

Architecture), not by NEXUS. The chain does not claim properties it does not deliver. Holders who require cryptographic anonymity route through 7D explicitly.

Deterministic eject at unlock_height + 8,640. The protocol fires EJECT exactly 8,640 blocks (24 hours) after the entering transaction's confirmation block. There is no randomness beacon, no slot selection, no per-entry timing variation. Every validating node computes the same eject block deterministically from chain state. The eject_height is fixed at entry and changes only under chain reorganisation deeper than the entry's confirmation, which is a catastrophic event handled by standard reorg rules. The simplicity of the rule is intentional: timing obfuscation does not break on-chain lineage (graph analysis follows input/output references regardless of when each edge is recorded), so the temporal-randomisation machinery of earlier specifications served no real privacy purpose. NEXUS achieves its safety property — cancellable dwell — through the dwell duration itself, not through eject-slot uncertainty.

Protocol-driven eject without user signature. NEXUS eject is a protocol-driven state transition, in the same category as the previous ACTIVATE primitive, EXPIRE, RELEASE, RESEAL, SWEEP, and COINBASE. The eject does not require the originating holder's Dilithium signature; it executes deterministically based on the parameters committed in the original signed UNLOCK or SPEND transaction and the chain's consensus rules. Every validating node enforces the eject identically. The on-chain record of the eject is structurally clean — no holder signature points to the destination at the eject moment; the destination was committed at entry, and the protocol delivers there.

The state lifecycles through NEXUS:

UNLOCK to HOME: LOCKED (stealth) → UNLOCK → NEXUS (dwell) → EJECT → ACTIVE at HOME

Private SPEND from hot: ACTIVE at HOME → SPEND-private → NEXUS (dwell) → EJECT → LOCKED at recipient stealth

Private SPEND from cold: LOCKED (stealth) → SPEND-private-from-cold → NEXUS (dwell) → EJECT → LOCKED at recipient stealth

In all three cases NEXUS is the same machinery — same dwell duration, same cancellation primitives (signature or hash-preimage), same protocol-driven EJECT at unlock_height + 8,640, same audit memo on the entering transaction. The paths differ only in source state and destination.

Direct SPEND from ACTIVE — operational use of actively-maintained hot value. The fourth operational path. The holder has actively brought value into ACTIVE state through UNLOCK (which passed through NEXUS dwell on the way to ACTIVE) and has been continuously maintaining its hot status through periodic REFRESH actions. ACTIVE state requires deliberate active management — without REFRESH actions every 24 hours, the value automatically EXPIRES back to LOCKED and re-enters SCW-protected status. **Direct SPEND from ACTIVE is the operational use of value the holder has actively maintained;**** it is not a bypass of safety but the natural outflow from a state the holder is

continuously consenting to maintain.** Direct SPEND lands as LOCKED at the recipient's stealth-derived destination immediately (subject to confirmation latency only), with no second NEXUS dwell on the SPEND side. The holder's choice to use direct mode reflects their active management of the source Parcel; private mode is the alternative when the holder wants the additional cancellation window of a second NEXUS pass.

NEXUS is the SCW machinery, named and reframed. The cancellable observation window is preserved exactly as specified — 24 hours during which the holder can CANCEL via signature or via hash-preimage submission (see §[Transaction Model]). The deterministic eject schedule simplifies analysis without compromising the safety property. The state transitions, the signature requirements, and the consensus enforcement are all preserved.

CANCEL — Two Authorisation Paths

CANCEL during NEXUS dwell is available via two authorisation paths:

Signature-based CANCEL. The originating holder signs a CANCEL transaction with the Dilithium key of the source address. This is the standard path. The fee for CANCEL is permanently zero — the chain's safety primitive must never have economic friction.

Hash-preimage CANCEL. The originating holder, when constructing the entering transaction, optionally embeds a hash $X = H(S)$ where S is a secret known to the holder. During dwell, anyone submitting S to the chain triggers cancellation: the chain verifies $H(S) = X$; if yes, the value returns to the source address. The hash-preimage path provides delegable cancellation authority — the holder can give S to a designated party (a security service, a backup trustee, anyone they choose) who can cancel on the holder's behalf without holding the holder's spend key. The hash function is SHA3-512, the chain's single post-quantum hash function (NIST Level 5, 256-bit collision resistance against Grover-birthday); the construction does not require Dilithium signing.

Both paths are independent. A holder may use neither (no cancellation possible during dwell — unusual but allowed), either alone, or both simultaneously. The chain accepts either form; the first valid CANCEL during dwell consumes the entering Parcel, and there is no cancel-the-cancel.

Hash-preimage CANCEL is post-quantum compatible: hash functions retain preimage resistance against quantum adversaries (Grover's algorithm provides only a quadratic speedup, leaving 256-bit hashes with 128-bit effective security against quantum attack — well above any practical threshold).

ACTIVE State — Refresh and Default-to-LOCK

The ACTIVE state, after emerging from NEXUS for self-spends, has a default 24-hour validity window. Within that window the holder may SPEND, LOCK, or BURN. Inaction triggers EXPIRE, returning the value to LOCKED at the same address.

The holder may extend the ACTIVE window by submitting a refresh signature — a small signed transaction that resets the 24-hour clock. The holder can refresh as many times as

they want; each refresh starts a fresh 24-hour window. Refresh fee is small (governance-set; expected to be near-zero to encourage active use without economic burden).

The architecture is deliberate. The default is safety: inactive holders' ACTIVE balances do not stay hot indefinitely, limiting the window during which a compromised wallet could drain them. Active holders can keep balances ACTIVE as long as they need by refreshing.

The chain rewards active engagement; passive sitting returns to safe storage automatically. The chain is not a luxury hotel.

Ma'at — The Transaction Model

Transaction Definition

Scriptum transactions are atomic instructions that modify the state of a single value path. Each transaction consumes one or more inputs and produces one or more outputs. Scriptum does not support multi-recipient batching within a single transaction. Each transfer is executed as an independent instruction.

User-Initiated Transactions

UNLOCK

Initiates the SCW. Takes a value V (declared by the holder) as input rather than a fixed Parcel set. The wallet, using the holder's key pair, identifies LOCKED Parcels across all stealth-derived addresses derivable from that key pair and applies smallest-first coin selection to assemble inputs whose sum reaches or exceeds V . The UNLOCK transaction consumes the selected LOCKED Parcels from their respective addresses (which may number anywhere from one to many thousands depending on the holder's fragmentation state) and produces one NEXUS-dwelling output of value V destined for the holder's HOME address. If the selected inputs sum to more than V , the difference becomes a LOCKED remainder at HOME (delivered through the same NEXUS dwell). Each source address contributes its bundle to the transaction (revealed if first-touch, referenced if already bound per §6.6); each contributes a Dilithium signature that the protocol verifies against its bundle. $\text{Fee} = f_U$ (flat instruction fee on summed value V , not per input or per source address). f_A is reserved and fixed at UNLOCK-time V_A . It is deducted automatically at EJECT using the V_A value recorded at the UNLOCK block — not the V_A at the time of EJECT. V_A is fixed at decision time. UNLOCK is a consolidation primitive: every UNLOCK consolidates the holder's stealth-fragmented value at HOME as the operational hot pool. The UNLOCK transaction may optionally embed a hash $X = H(S)$ to enable hash-preimage CANCEL during dwell; see §CANCEL.

CANCEL

Reverses a NEXUS-dwelling output to LOCKED. Valid throughout the dwell window (until the protocol-determined eject slot is reached). Recovers reserved f_A . $\text{Fee} = 0$.

CANCEL is available via two authorisation paths:

Signature-based CANCEL. The originating holder signs a CANCEL transaction with the Dilithium key of the source address. Standard path. The fee is permanently zero — the chain’s safety primitive must never have economic friction.

Hash-preimage CANCEL. The originating holder, when constructing the entering UNLOCK or SPEND, may optionally embed a hash $X = H(S)$ where S is a secret known to the holder. During dwell, anyone submitting S to the chain triggers cancellation: the chain verifies $H(S) = X$; if yes, the value returns to the source address. The hash-preimage path provides delegable cancellation authority — the holder can give S to a designated party (a security service, a backup trustee) who can cancel on the holder’s behalf without holding the holder’s spend key. The hash function is SHA3-512, the chain’s single post-quantum hash function (NIST Level 5, 256-bit collision resistance against Grover-birthday); the construction does not require Dilithium signing.

Both paths are independent. A holder may use neither, either alone, or both simultaneously. The first valid CANCEL during dwell consumes the entering Parcel; there is no cancel-the-cancel.

SPEND(x)

Transfers x Scriptum to a recipient address, creating a LOCKED output at that recipient’s stealth-derived one-time address (derived via Kyber/ML-KEM encapsulation against the recipient’s published Kyber view key). SPEND has two modes; the holder selects at SPEND time:

Direct mode. SPEND consumes ACTIVE Scriptum at the holder’s HOME and produces LOCKED Parcel at the recipient’s stealth-derived destination immediately (subject to confirmation latency only). No second NEXUS dwell on the SPEND side. The ACTIVE Scriptum has been actively brought into hot state by the holder (through an UNLOCK that passed through NEXUS dwell) and is being actively maintained there through periodic REFRESH actions. Direct SPEND is the operational use of value the holder is deliberately keeping hot; if the holder stops REFRESHing, value automatically EXPIRES to LOCKED and re-enters SCW-protected status. Fee = f_S on amount.

Private mode. SPEND from ACTIVE Scriptum at HOME enters NEXUS dwell-state for the SCW duration (24 hours, 8,640 blocks); EJECT delivers LOCKED Parcel at the recipient’s stealth-derived destination at `unlock_height + 8,640`. Cancellable during dwell via signature or hash-preimage. Fee = f_S on amount. Private mode is the holder’s opt-in for cancellation safety on outward sends despite having hot capital.

A third path — **Private SPEND from LOCKED** — bypasses the ACTIVE state entirely. The holder commits LOCKED Scriptum directly to the recipient via a SPEND transaction that consumes LOCKED Parcels from the holder’s stealth-derived addresses (smallest-first selection across the holder’s address cluster, like UNLOCK), enters NEXUS dwell with the recipient as destination, and EJECTs LOCKED Parcel at the recipient’s stealth-derived destination after dwell. Single 24-hour SCW; cancellable; no intermediate ACTIVE state. The institutional path for treasurers moving cold storage directly to counterparties without

value visiting hot state. $\text{Fee} = f_U + f_S$ (the f_U covers the chain's unlock-side work; f_S covers the spend-side amount).

The SPEND includes the encrypted Sealed Sender Memo carrying the sender's published address (and any additional content within the size budget), encrypted to the recipient's view key. f_S is calculated at SPEND-time V_A — the V_A of the block in which the SPEND transaction is included. V_A is fixed at decision time. Bob receives a LOCKED Parcel at his stealth-derived destination; to use it, Bob does his own UNLOCK (which enters his own NEXUS dwell on his side).

Coin Selection and Destination Rules

The chain's consolidation primitives (UNLOCK, private SPEND from cold, and Frau Holle's cross-address sweep) share a single principled design: smallest-first coin selection, with output landing at the holder's HOME address (UNLOCK and Frau Holle's cross-address scope) or at the recipient's stealth-derived destination (private SPEND from cold).

Smallest-first selection. When a primitive needs to consume Parcels to produce an output of value V (or to consume the eligible set entirely in the case of Frau Holle), the wallet (for UNLOCK and private SPEND) or the protocol (for Frau Holle's cross-address sweep) sorts the holder's Parcels by amount ascending and consumes them in order until the required value is reached or the eligible set is exhausted. Smallest-first naturally consolidates dust through normal use, keeps the holder's Parcel set leaner over time, and does not penalise the user economically because the operation-based fee model charges per operation on summed value rather than per input.

Destination rule. The destination of an UNLOCK is always HOME — the holder's published operational address. The destination of a Frau Holle cross-address sweep is also HOME, uniformly across all holders. The destination of a private SPEND from cold is the recipient's stealth-derived address, derived from the sender's ephemeral DH against the recipient's view key. Per-address Frau Holle scope (consolidating Parcels at a single address) keeps the consolidated Parcel at that same address.

Wallet bookkeeping. The wallet (informed by Ratatoskr) tracks the holder's address space: stealth-derived receive addresses where senders have delivered value, the holder's home address with consolidated ACTIVE balance, any stealth descendants used in identity-tree configurations. The user does not see the cluster mechanics in the UI — the wallet aggregates the holder's balance for display — but at the protocol level, the address topology is structured around HOME as the operational hot pool and stealth addresses as the receive-side pseudonymity layer.

Why HOME. The earlier smallest-source X destination rule produced a content-dependent consolidation address that varied per UNLOCK. The privacy benefit was modest (observers tracking a holder over many UNLOCKS could cluster the X destinations through pattern analysis) and the operational cost was real (wallets had to track which X destination held current value, and the holder's mental model was muddled). HOME as a stable consolidation destination is operationally cleaner, structurally honest (the chain doesn't

pretend to hide where holders consolidate), and composes better with the four operational paths the chain provides.

LOCK(x)

Re-locks x Scriptum to the originating address. Creates a LOCKED output. Applies coin selection. Remainder stays ACTIVE with expiry = minimum of consumed inputs. Fee = 0. Symmetric with SPEND except recipient = self.

BURN(x)

Sends amount x Scriptum to Gehenna — the protocol-defined burn address, provably unspendable by mathematical construction. The Parcel continues to exist at Gehenna in state LOCKED, class CIRCULATING. It is not destroyed. It is permanently subtracted from the circulating supply via the S_burned counter: $S_{net} = S_{mined} - S_{burned}$. Burned value is a permanent red card: the coin exists, but it can never return to circulation. Even under a hypothetical reopening of Gehenna, protocol rules would reject any reintroduction pushing $S_{net} > S_{max}$; the emission formula makes this mathematically impossible, because $r \rightarrow 0$ as $S_{net} \rightarrow S_{max}$. Coin selection applies across ACTIVE Parcel inputs. Remainder stays ACTIVE at the originating address, carrying any existing binding. Fee = 0.

DONATE(x)

Sends x Scriptum to the protocol-governed donation mechanism. The donated amount is split 50% to Funds and 50% to RESERVE. Chesed is the semantic name for this act of giving — the spirit of the disbursement — not a single discrete address; value flows directly to the two vaults at the 50/50 ratio. On confirmation, the protocol issues a DANA MARKER Parcel to the sending address: CLASS = MARKER, ROLE = DANA, STATE = PERMA_LOCKED, AMOUNT = 1 Nugget. The DANA Parcel is the permanent private record of the donation, re-instantiated unchanged at every Rebirth. Fee = 0.

REFRESH

Extends an ACTIVE Parcel's validity window. The holder signs a small refresh transaction referencing the ACTIVE Parcel; the protocol resets the 8,640-block expiry clock to start from refresh_height. The holder may refresh as many times as they want; each refresh starts a fresh 24-hour window. Fee = f_refresh (governance-set, expected to be near-zero to encourage active engagement). The default behaviour without REFRESH is EXPIRE → LOCKED at activate_height + 8,640: the chain rewards active engagement; passive sitting returns to safe storage automatically.

PACK

Arc procedure only. User-initiated. Consolidates all enrolled wallet Parcels into a single PACKING output. Once PACK is confirmed, the protocol takes over: BOARD fires automatically at the enrolled boarding slot, 10 blocks after CANCEL_THRESHOLD. Fee = 0.

HORKOS_AGREE

The Stage 1 multi-sig transaction signed by A, B, and C committing to negotiated trade terms (principal P, stake S, attestation fee z, attester identity, lock-to-approval window length W). The protocol verifies all three signatures and validates structural floors: $S \geq 0.001 \times P$; $W \geq 8,640$ blocks; $W \leq$ remaining time to next Ragnarök. Recording the agreement is fee-free at this stage; no funds have moved. Acceptance opens the 24-hour load window.

HORKOS_LOAD_A

A's funding transaction during the Stage 2 load window. Consumes one or more LOCKED Parcels at A's address. Uploads A's principal P plus attestation fee z into the protocol-controlled ESCROW address associated with this Horkos contract. Fee = f_S calculated on (P + z). The load is tracked by the chain against the agreement; if A loads the full required amount within the 24-hour window, A's load is complete.

HORKOS_LOAD_B

B's funding transaction during the Stage 2 load window. Consumes one or more LOCKED Parcels at B's address. Uploads B's stake S into the ESCROW address. Fee = f_S calculated on S. B's load is complete when B has loaded the full agreed stake. If either A or B fails to fully load within 24 hours, the contract lapses and loaded principals/stakes return to their loaders; the chain fees paid at load are not refunded.

HORKOS_CONFIRM

The Stage 3 multi-sig transaction signed by A, B, and C after both loads are complete. Final binding signature; transitions the contract from FUNDED to LOCKED. Fee-free at the protocol level. After HORKOS_CONFIRM, the safety is off and the lock-to-approval window of length W begins.

HORKOS_ATTEST

C's attestation transaction during Stage 4. The registered attester signs either YES (B's off-chain delivery occurred) or NO (B's delivery did not occur) referencing the specific contract identifier. Must arrive within the lock-to-approval window W. C pays f_U as a minimal anti-spam fee. If C does not submit HORKOS_ATTEST within W, the outcome is TIMEOUT.

Protocol-Driven Transactions

EJECT (formerly ACTIVATE)

EJECT — protocol-driven transition from NEXUS dwell to its committed destination. For UNLOCK entries, destination is the holder's HOME address (produces ACTIVE Parcel at HOME); deducts f_A (reserved at UNLOCK at the V_A recorded in the UNLOCK block); the holder receives a full 8,640-block ACTIVE window from `eject_height`. For private SPEND entries, destination is the recipient's stealth-derived address (produces LOCKED Parcel at the recipient). EJECT fires deterministically at `unlock_height + 8,640`; every validating node

computes the same eject_height from chain state. Fee = 0 at eject (the chain's f_A and f_S were committed at entry). At eject_height, the dwell window closes and the Parcel enters Limbo (CANCEL is invalid from this point forward). Block inclusion follows the priority queue rules in §Protocol-Driven Transitions and Limbo: under normal load, EJECT executes in the next block (Limbo = milliseconds); under congested load, EJECT executes in a subsequent block as block capacity allows (Limbo extended).

EXPIRE

EXPIRE — transitions ACTIVE to LOCKED at activate_height + 8,640. Fee = 0.

COINBASE

COINBASE — creates new Scriptum for the miner. Output is LOCKED. Subject to standard unlock rules.

VEST

VEST — advances TREASURE through the deterministic vesting schedule. Transitions SEALED → VESTING at vesting start height.

RELEASE

RELEASE — fires at each vesting milestone block height. Consumes VESTING Parcel. Creates LOCKED Parcel with CLASS = CIRCULATING, ROLE = NONE. Fee = 0.

RESEAL

RESEAL — fires at open_height + 8,640 if VAULT remains OPEN with no PACKING initiated. Returns VAULT to SEALED. Fee = 0. Analogous to EXPIRE for CIRCULATING.

SWEEP

SWEEP — the cross-stealth-address Menuha consolidation transaction. Wallet-driven and Aurum-mandatory; consensus-validated like any holder-signed transaction. When a holder's address count (the count of distinct addresses currently containing the holder's Parcels, computed by the wallet via local Ratatoskr scan) exceeds three, the wallet constructs a multi-input SWEEP transaction during a Menuha Window that consumes eligible LOCKED Parcels across the holder's stealth-derived addresses and produces one LOCKED successor Parcel at the holder's HOME address. The wallet signs the transaction with the Dilithium keys of the consumed addresses. The chain validates the signatures and accepts the transaction; consensus does not enforce that this transaction be present (the chain cannot identify a holder's address set without view-key access). Aurum's wallet implementation constructs and submits SWEEP transactions automatically at every Menuha when the threshold is crossed, with no opt-out — for users of compliant wallets, SWEEP is mandatory at the wallet layer. SWEEP fires only during Menuha Windows. Bindings inherit by address identity. Fee = 0 (consolidation is fee-free housekeeping).

There is no per-address SWEEP transaction. Per-address consolidation is performed by UNLOCK as a side effect of normal operation: when a holder UNLOCKS from one or more of

their stealth-derived addresses, smallest-first coin selection consumes Parcels at those source addresses into a single NEXUS-dwelling Parcel destined for HOME. Each UNLOCK that draws from an address effectively consolidates that address's Parcels as part of its operation. No separate per-address SWEEP primitive is needed.

DANA ISSUE

DANA ISSUE — fires on confirmation of a DONATE transaction. Creates DANA Parcel at the sending address: CLASS = MARKER, ROLE = DANA, STATE = PERMA_LOCKED, AMOUNT = 1 Nugget. Fee = 0.

BOARD

BOARD — executes at the boarding slot, 10 blocks after CANCEL_THRESHOLD closes. Consumes PACKING Parcel and accepted stragglers. Produces a single ARC output. Fee = 0. Consensus-enforced: any block omitting a required BOARD output is invalid and rejected by all nodes. The miner cannot exclude it. ESCROW Parcels with pending Horkos trades are also boarded, preserving those trades across the Ragnarök boundary.

DISEMBARK

DISEMBARK — protocol-driven at Block 0 of every Rebirth. Reads the complete ARC manifest and instantiates all valid ARC Parcels as LOCKED outputs in the successor Phoenix. No user action required. Executed once per Phoenix cycle.

Horkos Resolution Sequence

Horkos Resolution — fires at the conclusion of Stage 4 (the lock-to-approval window) when HORKOS_ATTEST has been submitted with verdict YES or NO, or when the window has expired without attestation (TIMEOUT). The resolution is not a single named transaction but a protocol-driven sequence using the chain's standard primitives — UNLOCK, EJECT, SEND — executed by the protocol rather than by user signatures. The escrowed Parcels are auto-UNLOCKED, EJECTED to their resolution-determined destinations, and SENT to the recipients per the outcome:

YES outcome — A's principal → B, B's stake → B, attestation fee z → C.

NO outcome — A's principal → A, B's stake → routed via standard Tithe split per §15 (90% miner, 10% Tithe allocated 48/32/10/10 across WFC/Funds/RESERVE/CP), attestation fee z → C.

TIMEOUT outcome — A's principal → A, attestation fee z refunded to A, B's stake split 50/50 (50% returned to B, 50% routed via standard Tithe split per §15), C receives nothing.

The resolution sequence is fee-free at the chain level — chain fees were paid at the load stage. The sequence inherits the standard properties of the chain primitives it uses: SCW observability for each SEND step, Tithe routing for slashed funds, on-chain auditability

throughout. Consensus-enforced: any block omitting a required protocol-driven Horkos resolution operation at its scheduled height is invalid.

Value Transfer Sequence

To execute a value transfer:

UNLOCK (user) → EJECT (protocol) → SPEND (user)

SPEND creates an ACTIVE remainder for the sender and a LOCKED output for the recipient. Every SPEND derives the recipient's one-time output address via Kyber/ML-KEM key encapsulation against the recipient's published view key; the sender includes the encapsulation ciphertext (the ephemeral key material) in the transaction for recipient-side scanning. The LOCKED change output returning to the sender's own address carries the sender's key binding forward (see §Address Binding); the one-time LOCKED output at the recipient derives from the recipient's stealth keys and does not carry the sender's binding.

No Hidden Execution

Scriptum deliberately excludes Turing-complete execution. All protocol behaviour is finite, deterministic, and explicitly defined through enumerated state transitions. This eliminates hidden execution paths and ensures that value movement remains transparent, auditable, and controllable at all times.

There are no smart contracts. There is no dynamic execution. There is no emergent behaviour. Every state a Parcel can occupy, and every transition between states, is defined in this document. An implementation is valid if and only if it reproduces these transitions exactly.

This is a security property, not a limitation. Hidden execution paths are attack surfaces. Scriptum has none.

The Covenant — Fees and Miner Incentives

Fee Philosophy

Scriptum is not the cheapest chain for large transfers. It is the only chain where the fee is fair for everyone — from a €1,000 personal transfer to a €200M institutional settlement.

Every competing blockchain charges a flat fee per transaction, independent of the amount transferred. Bitcoin charges approximately €4 regardless of whether you send €10 or €200,000,000. At €10 that is 40% of the transfer. At €200M it is essentially free. There is no proportionality. There is no fairness. There is only a flat toll that punishes small users and subsidises large ones.

Scriptum charges proportionally. The fee is always a percentage of the transfer value, calculated in € using progressive tiers, then converted to FUNK via V_A. At €1,000 you pay approximately €5 — the same as Bitcoin. Above that, you pay more than Bitcoin. But above that, you also receive what Bitcoin cannot give you: the Safety Control Window, post-

quantum security, 24/6 settlement, an immutable permanent record, and a trading environment designed to protect every participant equally.

Scriptum is not fast food. It is a restaurant. The meal costs more. The quality is incomparable. And crucially, the menu is priced fairly — the same percentage for the person sending €1,000 and the institution moving €200M. Nobody is subsidised. Nobody is punished. The tiers ensure that larger transfers pay a lower percentage, not a higher one — making Scriptum progressively more competitive as transfer size grows.

The fee in FUNK is always: $f_FUNK = f_EUR / V_A$. The € amount is fixed by the progressive tiers. V_A converts to FUNK at a smoothed, tamper-resistant rate. The user pays a **protocol-calibrated euro-equivalent fee** — the conversion uses V_A as the chain's calibrated valuation coefficient, which is deliberately smoothed and lagged for tamper-resistance and is fixed at 1.0 during pre-market operation (see §Urðarbrunnr — Valuation Integrity and Temporal Pricing); the euro-equivalent is therefore the chain's authoritative valuation, not a real-time market quote. Every fee is split 90% to the miner in FUNK and 10% to the Tithe, allocated across four streams (48% WFC, 32% Funds, 10% RESERVE, 10% CP) per §15. The split applies uniformly to f_U , f_A , f_S , and $F_REFRESH$. There is no carve-out for flat instruction fees. No additional multiplier is required.

The chain charges its proportional commission on every value transfer, including BURN and DONATE. Burning value is a holder's active decision and the chain processes it like any other SPEND — same machinery, same fee, same payment to miners and to the tithe stream that funds Foundation operations and charitable activities. Donating to a chain VAULT works the same way: the donation pays its standard commission and the donor's act of giving simultaneously feeds the chain that will continue to safeguard their remaining holdings.

The only exception is CANCEL — the holder's safety mechanism against unauthorised UNLOCK. Charging for CANCEL would create economic friction on the chain's primary safety guarantee, which is a structural design error. CANCEL remains permanently fee-free.

Other transitions that do not transfer value to a non-holder address — LOCK (voluntary return-to-LOCKED-state, value stays with the holder), EXPIRE (protocol-driven, no holder initiation), PACK and BOARD (Arc machinery, holder sovereignty during Ragnarök) — are also fee-free, as they are not value transfers but housekeeping or protective primitives.

The principle is consistent: the chain is paid a small commission for processing every value transfer between parties or destinations, regardless of motive. The chain does not subsidise particular kinds of transactions through fee carve-outs. Holders who want their value to move pay the chain to move it; holders who want safety primitives are not taxed for using them.

Fee Model: Operation-Based, Not Input-Based

Scriptum's fees are charged per operation on the total value being moved, never per input or per Parcel. f_U is one fee per UNLOCK transaction, calculated on the sum of unlocked value. f_A is a flat instruction fee per EJECT (irrespective of amount), reserved at UNLOCK

using the V_A at the UNLOCK block and paid to the miner including the EJECT block. f_S is one fee per SPEND, calculated on the sum of value being sent. A holder consolidating value from many one-time stealth-derived addresses into a single UNLOCK pays one f_U on the consolidated total — not one f_U per source address. The wallet handles Parcel aggregation behind the scenes; the user pays for value transfer, not for accounting overhead. This means stealth-driven address fragmentation does not impose a fragmentation tax on holders — the privacy property is structurally fee-neutral. The chain charges per-operation per-summed-value at every transition, regardless of how many addresses or Parcels participate as inputs.

Fee Schedule

f_U — flat instruction fee, irrespective of amount, paid at UNLOCK block

f_A — flat instruction fee, irrespective of amount; reserved at UNLOCK using the V_A at the UNLOCK block, paid to the miner at the EJECT block (the protocol-driven transition out of NEXUS dwell)

f_S — proportional to amount, progressive tiers, no flat component, paid at SPEND block

CANCEL, LOCK, EXPIRE, PACK, BOARD — free ($f = 0$). BURN and DONATE pay the standard SPEND fee per SPX-4 (see §Fee Philosophy).

At base reference (€1 per Scriptum, for calibration):

$f_U = 0.04$ Scriptum = €0.04

$f_A = 0.01$ Scriptum = €0.01

f_S = progressive tier % of € amount at V_A

f_S — Progressive Tier Model

Calculated cumulatively across tiers on the transferred amount in €, at current V_A . Each portion of the amount is charged at its tier rate:

Tier 1 €1 – €1,000 → 0.50%

Tier 2 €1,001 – €10,000 → 0.45%

Tier 3 €10,001 – €50,000 → 0.40%

Tier 4 €50,001 – €100,000 → 0.35%

Tier 5 €100,001 – €200,000 → 0.30%

Tier 6 €200,001 – €500,000 → 0.25%

Tier 7 €500,001 – €1,000,000 → 0.20%

Tier 8 €1M – €2,000,000 → 0.15%

Tier 9 €2M – €10,000,000 → 0.10%

Tier 10 >€10,000,000 → 0.05%

f_S has no flat instruction component. It is purely proportional to amount.

Examples

All fees are defined in €. V_A converts to FUNK at the smoothed market rate. The € amount is always identical regardless of FUNK price.

Transfer €1,000:

f_U = €0.04 f_A = €0.01 f_S = €5.00 Total = €5.05 (0.505%)

At €1/F: 5.02 FUNK | At €1,000/F: 0.00502 FUNK

User pays €5.05. Miner receives €4.545 (90% of every fee). Protocol Tithe receives €0.505 (10% of every fee), allocated as: WFC €0.242 (48%), Funds €0.162 (32%), RESERVE €0.051 (10%), CP €0.051 (10%).

Transfer €10,000: f_S = €45.50 Total = €45.52 (0.455%)

Transfer €1,000,000: f_S ≈ €2,430 Total ≈ €2,430 (0.243%)

Transfer €200,000,000: f_S ≈ €107,000 Total ≈ €107,000 (0.054%)

Comparison on €1,000 transfer:

Bitcoin: €4 flat = 0.40% (comparable cost, no SCW, no PQ)

Ethereum: €0.47 flat = 0.047% (cheaper, no protection)

Solana: €0.00025 flat (near free, no protection)

Scriptum: €5.02 = 0.50% (SCW + post-quantum + 24/6 + immutable record)

Comparison on €10 transfer:

Bitcoin: €4 flat = 40% (punishing for small users)

Ethereum: €0.47 flat = 4.7% (expensive)

Solana: €0.00025 = 0.0025% (cheap)

Scriptum: €0.05 = 0.50% (fair — same % as €1,000)

Scriptum is the only chain where the fee percentage is consistent regardless of transfer size. Every other chain charges a flat amount per transaction that punishes small users and effectively rewards large ones. Scriptum charges proportionally. The same percentage applies at €10 and at €200,000,000.

Design Objectives

Identical transfers incur identical costs, regardless of Parcel fragmentation

Internal representation of funds does not affect pricing

Users are not penalized for fragmentation or the structure of their holdings

High-value transfers remain economically efficient

Fee levels remain meaningful at any Scriptum price

Dust Threshold

A Parcel is dust if its fiat-equivalent value at current V_A is less than €0.01. Such Parcels are economically irrational to unlock — fees exceed value. Transactions creating dust outputs are invalid and rejected. In Nugget terms: at Scriptum = €1, dust threshold $\approx 1,000,000$ Nuggets.

Miner Reward

Miner compensation = block reward + transaction fees (f_U, f_A, f_S). As block rewards decrease toward zero, fees become the primary incentive. The fee model and V_A mechanism ensure fee sustainability across all Scriptum price levels. The € value of fees remains proportional to transfer amounts regardless of FUNK market price.

Menuha — Sunday Rest

SPEND (all three modes), BURN, DONATE, LOCK, HORKOS_COMMIT, and HORKOS_STAKE are invalid during the Menuha Window. The Menuha Window is any block whose timestamp falls on Saturday 23:00 UTC through Sunday 23:00 UTC, evaluated via Median Time Past (MTP) of the most recent 11 blocks to defeat single-miner timestamp gaming. Transactions of these types submitted during Menuha are rejected. UNLOCK, CANCEL, REFRESH, HORKOS_ATTEST, PACK, and STRAGGLER_ADD remain valid; SWEEP is valid only during Menuha. Full Menuha context in Section 30. The agreement phase of Horkos is suspended during Menuha: Stage 1 (negotiation multi-sig), Stage 2 (loading transactions HORKOS_COMMIT and HORKOS_STAKE), and Stage 3 (final confirmation multi-sig) cannot be submitted, since each commits new outward value to the contract. Stage 4 (HORKOS_ATTEST) and Stage 5 (resolution) run through Menuha unaffected — HORKOS_ATTEST is the recording of a delivery decision by attestor C, not an outward value transfer, and Stage 5 is protocol-driven (UNLOCK → EJECT → SEND with no user signatures). A Horkos contract whose attestation finalises just before or during Menuha completes its resolution sequence on schedule; only new agreements cannot be initiated during the rest day.

Urðarbrunnr — Valuation Integrity and Temporal Pricing

Scriptum introduces a deterministic valuation integrity layer governing how market price translates into executed economic cost. The Urðarbrunnr Integrity System (UIS) is not designed to determine price. It is designed to protect the translation of price into fees.

In conventional systems, fee determination based on instantaneous price creates an attack surface: any participant capable of temporarily influencing valuation can execute high-value transfers at artificially reduced cost. Scriptum eliminates this vector by introducing temporal separation between price observation and fee application. Valuation must persist over time before it can influence execution.

Price may change instantly. Its economic effect cannot.

Valuation Coefficients

The V_A mechanism is source-agnostic by design. Miners propose V_P values derived from any price source they choose — an exchange, an aggregator, a private feed, or any combination. The protocol validates only the drift constraints, not the source. This ensures the system remains functional across any time horizon, regardless of which exchanges or data providers exist at any given moment. No external oracle is hardcoded. No single source is privileged.

Critical distinction: V_A governs fee calculation only. It does not determine the market price of FUNK. The market price of FUNK is set by buyers and sellers on exchanges — by supply and demand — entirely independently of V_A . A miner who misreports V_P affects only the fiat-equivalent fee calculation, not the real trading value of FUNK. This makes V_A manipulation economically irrational: the attacker cannot profit from it, and users can simply wait for V_A to converge before transacting.

V_C — current market price (external, instantaneous, outside consensus)

V_P — miner proposal: the valuation value a miner embeds in their block header

V_A — accepted protocol valuation: the consensus value used for all fee calculations

V_A evolves gradually based on a rolling statistical window of 4,320 blocks (~12 hours), using geometric mean and standard deviation, with bounded drift per block. This introduces temporal lag between V_C and V_A . Fees are derived exclusively from V_A .

V_A is constrained by two hard consensus rules enforced by all nodes:

Constraint 1 — Per-block drift limit:

V_P must satisfy: $0.99 \times V_{A_previous} \leq V_P \leq 1.01 \times V_{A_previous}$

V_A cannot move more than 1% from the previous block in a single step.

Constraint 2 — Trailing average band:

V_P must satisfy: $0.85 \times V_{avg}(4320) \leq V_P \leq 1.15 \times V_{avg}(4320)$

V_A cannot deviate more than 15% from the 4,320-block (~12 hour) rolling average.

Enforcement: if a miner proposes a V_P that violates either constraint, nodes clamp V_P to the nearest valid bound (the closer of the two values defined by the inequality). The block is never rejected on grounds of V_P. Execution always proceeds. This prevents chain stalls while eliminating manipulation.

Genesis V_A: V_A = 1.0 at block 0 of every Phoenix. A base reference of €1 is used for initial fee calibration only. Market price is established by the first trade. V_A evolution begins when miners propose V_P values derived from real market data.

Pre-market V_A: before Scriptum has an established external market price, V_A remains at 1.0. V_A evolution begins only when miners begin proposing V_P values derived from real market data. Until then, all fees are calculated at base reference.

Economic Consequence

Rapid price increases → V_A lags V_C upward → temporarily higher effective fees

Rapid price decreases → V_A lags V_C downward → temporarily lower effective fees

Users may execute immediately or wait for V_A to converge toward V_C. Since the SCW enforces an 8,640-block window before value becomes transferable, V_A will typically have converged substantially before execution is possible in standard operation. Under extreme market volatility, some divergence may persist.

The UIS — Three Temporal Layers

Verðandi — Instantaneous State

Current valuation proposals (V_P) represent the present. Each block miner proposes a V_P by embedding it in the block header.

Urðr — Accumulated Reality

CUSUM-based anomaly accumulation records persistent directional drift over time. If V_P consistently deviates from V_A, evidence accumulates across blocks.

Skuld — Inevitability

Threshold-based escalation determines when accumulated anomaly evidence must be acknowledged and surfaced to users.

The Four Mechanisms

The three temporal layers above describe the conceptual architecture. The Scriptum System Parameters document (SPX-8) specifies the four operational mechanisms that implement the UIS: Mechanism A performs oracle input validation — cross-source consistency and statistical outlier rejection against live oracle feeds. Mechanism B requires consecutive-cycle confirmation before V_A moves, preventing single-cycle noise from disturbing the coefficient. Mechanism C is the CUSUM-based tampering alarm, raising a

public flag when miner-proposed V_A drifts sustainedly from oracle reference. Mechanism D is the fast-price-movement warning, raising a separate public flag when the instantaneous oracle-vs-V_A gap exceeds threshold — a legitimate market event, not tampering.

Mechanisms A and B operate within the Verðandi layer, governing what enters V_A in the present cycle. Mechanism C lives in the Urðr layer, accumulating evidence of sustained pressure. Mechanism D reaches across both Verðandi and Skuld: it surfaces fast events immediately (present) while flagging their implications for downstream consensus observation (inevitability). Both alarms (C and D) are informational. Neither halts the chain. Neither freezes V_A. Their work is transparency — making manipulation publicly visible, not blocking it by force.

During early network formation the UIS operates in bootstrap mode: oracle redundancy is intentionally reduced (ORACLE_ACTIVE_COUNT_MIN = 1 at genesis, ramping toward the steady-state target as the oracle network matures). Conforming wallets and observer nodes display a standing public warning during this period. If active oracle coverage falls below the minimum threshold at any time, V_A holds its last valid value rather than drifting on insufficient data — INSUFFICIENT_ORACLE_MODE = HOLD_LAST_VALID_V_A in SPX-8. The protocol degrades gracefully under oracle failure; it does not halt, and the coefficient cannot be forced to an attacker-favourable value through oracle denial.

Hugin and Munin — V_A Anomaly Detection

Two complementary mechanisms watch for tampering of V_A. Hugin observes the immediate per-block deviation between V_P (miner proposals) and V_A (accepted valuation): for each block, Hugin records the delta. Munin maintains the cumulative running sum of those deltas — the CUSUM accumulator — and raises an informational tampering alarm in Urðr when the accumulated drift exceeds threshold in either direction. Hugin sees what just happened; Munin holds the persistent record and triggers the alarm. Together they detect sustained one-sided pressure from a dominant miner attempting to silently steer V_A over time. Named for Odin's two ravens, thought (Hugin) and memory (Munin), the pair embodies the surveillance function: immediate observation feeding into long-memory accumulation. The Ratatoskr name has been redeployed elsewhere in the architecture as the chain-traversal enumeration primitive (see §Privacy Model / Ratatoskr); it no longer refers to the V_A anomaly detector.

Detection Without Intervention

UIS does not modify protocol behaviour. It does not alter fees, block transactions, or override valuation. It detects abnormal conditions, accumulates evidence, and exposes anomalies. The protocol remains fully deterministic.

Scriptum does not enforce truth. It ensures that distortion becomes visible before it becomes decisive.

Economically meaningful manipulation requires sustained control over block production across time, making such attacks capital-intensive, observable, and self-limiting.

Protected Mode — Aurum

Aurum provides native UIS integration. When abnormal conditions occur — significant divergence between V_C and V_A, or persistent miner proposal pressure — Aurum activates Protected Mode: an explicit warning is displayed and additional confirmation is required before execution. Execution itself remains fully deterministic.

Mining Speed and Hardware

Scriptum uses SHA3-512 as its proof-of-work algorithm. SHA3 is the NIST-standardised sponge-construction hash family, deliberately structured to provide defense in depth against the kinds of cryptanalytic advances that have historically chipped away at SHA-2 derivatives. Scriptum does not seek ASIC-resistance through algorithmic obscurity; it commits to a transparent, well-studied primitive and accepts that mining will professionalise as the asset gains value. Anyone with a computer can mine Scriptum at genesis.

What Determines Mining Speed

Mine speed is determined entirely by difficulty adjustment, not hardware. The difficulty adjustment maintains BLOCK_TIME = 10 seconds regardless of how many miners compete or what hardware they use. More miners means higher difficulty, not faster blocks. Mine speed affects security — the cost to attack the chain. It does not affect throughput.

Retargeting fires every 2,592 blocks — the Great Year (25,920 seconds, encoding Earth's axial precession). The cadence balances responsiveness and stability: ~3.3 retargets per day track hashrate dynamics; 2,592 samples per period smooth out per-block timing noise. The retarget math is asymmetric (2× harder, 4× easier) to resist timewarp manipulation while preserving the chain's ability to recover from hashrate collapse. See §Block Parameters for the full consensus rules.

1 miner → baseline difficulty 10s blocks low security 100 miners → 100× difficulty 10s blocks growing security 1,000 miners → 1,000× difficulty 10s blocks strong security 1,000,000 → 1,000,000× difficulty 10s blocks industrial security

51% Attack at Low Hashrate — Economics

At genesis, Scriptum's hashrate is low. A sustained 51% majority can be rented. A private chain assembled from commodity GPU time — on the order of \$50,000 for several hours of block production — can reorg recent Scriptum history in the first weeks of the chain. The whitepaper concedes this openly.

What a reorg means for holder value is treated in §34. In short: a reorg in Scriptum is not theft. Bitcoin's approximately \$30 billion per year of hashrate protects holder value because Bitcoin's architecture equates authorization with execution and has no other layer of protection. Scriptum separates authorization from execution by protocol, and therefore protects holder value independently of hashrate. The economic incentive to rent hashrate

for attack collapses when the attack cannot move money: an attacker who spends \$50,000 to rent a reorg achieves a reorg, not a theft.

Hashrate still matters in Scriptum. It establishes chain ordering, confirms transaction sequence, and hardens the chain against censorship over time. It is not, however, the mechanism by which holder value is protected. For the full security analysis — including the normalized-axis comparison to Bitcoin and the complete enumeration of what a 51% attacker can and cannot do — see §34.1 through §34.4.

Low hashrate alone does not enable theft. A low-hashrate reorg attacker, without possession of a legitimate holder's Dilithium3 private key, extracts nothing. Theft requires combined control of hashrate and key material sustained across the full 8,640-block SCW. See §34 for the full security model.

Block Validation by Hardware

Dilithium3 verification on modern hardware (AVX2-optimised): 0.045ms per signature. A 2 MB genesis block contains approximately 100 SPEND cycles = 200 signed transactions (UNLOCK + SPEND; EJECT carries no signature). 200 verifications x 0.045ms = 9ms total. This is negligible. Propagation time (transfer + network hops) dominates, not verification.

Home laptop (CPU) → validates 2 MB genesis block in ~9ms

Desktop CPU (Ryzen 9) → validates 2 MB genesis block in ~4ms

M4 Pro (10 cores) → validates 2 MB genesis block in ~2ms

GPU rig → sub-millisecond

Propagation at 100Mbps (2 MB genesis block): 88ms transfer + 50ms hops = 138ms total. Block time 10s. Safety margin 72x. Orphan rate ~2% at genesis block size. Comfortable.

Minimum Block Time

The absolute minimum safe block time is determined by propagation, not hardware. At 100Mbps with maximum 16MB blocks: propagation = 1.28s + 0.05s = 1.33s. We choose 10s for 13% orphan target. The minimum achievable block time at 100Mbps is approximately 1.5 seconds. Block time is a protocol constant and does not change.

Block Architecture and Capacity Model

Scriptum defines block capacity through a single formula. Throughput, storage, and orphan rate are all derived from three fixed constants and one governance variable.

The Capacity Formula

Unit definitions. Scriptum uses three levels of counting and distinguishes them in this section: (i) an atomic transaction is any single protocol message — UNLOCK, EJECT, SPEND, CANCEL, LOCK, BURN, REFRESH, EXPIRE, COINBASE, DONATE, and so on — each with its own size and rules; (ii) a signed transaction is an atomic transaction carrying a

user signature (UNLOCK, SPEND, CANCEL, LOCK, BURN, DONATE, REFRESH); EJECT, EXPIRE, and COINBASE are protocol-driven and unsigned; (iii) a SPEND cycle is the canonical settlement sequence UNLOCK → NEXUS dwell → EJECT → SPEND, a complete value transfer from a holder's LOCKED Parcel to a recipient's LOCKED Parcel at a stealth-derived destination. The NEXUS dwell sits on the unlock side only; SPEND of ACTIVE Scriptum is direct. Throughput figures in this section are given in settlements per second (= SPEND cycles per second). A 'settlement' and a 'SPEND cycle' mean the same thing. Raw atomic transaction counts include protocol-driven entries and are not used as the headline throughput metric.

SPENDs per second is determined exclusively by:

$$\text{SPENDs/sec} = \text{OR} \times \text{MBPS} / \text{CYCLE_SIZE}$$

OR = orphan rate maximum ACCEPTED (13% — protocol hard ceiling, not a target)

MBPS = bandwidth in bytes/sec (external variable)

CYCLE_SIZE = UNLOCK + EJECT + SPEND bytes (~7,100 first-touch, ~5,500 bound)

Block time cancels out of the throughput formula. Only bandwidth, orphan rate, and signature size determine SPENDs/sec. Block time affects only how capacity is chunked, not total throughput.

The capacity formula describes the theoretical throughput ceiling at the bandwidth-limited maximum block size (approximately 16 MB at 100 Mbps, yielding ~150 settlements/sec). At the genesis block size of 2 MB, realistic settlement throughput ranges from ~18/sec at fresh addresses (first UNLOCK reveals the full post-quantum key bundle) to ~25/sec at addresses with an established binding (subsequent UNLOCKS reference the inherited binding per §6.6). Steady-state settlement throughput on a mature chain with address-reuse history is approximately ~22/sec at 2 MB. Every SPEND includes the sender's ephemeral Kyber public key for stealth derivation; this is factored into these figures. See §18.6 for the scaling path.

Transaction Sizes

UNLOCK ~6,600 bytes user-signed (first UNLOCK at a fresh address — reveals full key bundle per §6.6)

UNLOCK ~3,500 bytes user-signed (subsequent UNLOCKS at a bound address — references inherited binding, no bundle reveal)

EJECT 264 bytes protocol-driven, no signature (executes deterministically at unlock_height + 8,640)

CANCEL 3,493 bytes user-signed (signature path) or ~50 bytes (hash-preimage path: just the preimage S plus a reference to the dwelling Parcel)

SPEND ~4,800 bytes user-signed: includes Dilithium signature (~3,300 bytes), sender's ephemeral Kyber public key for stealth derivation (~1,200 bytes), encrypted Sealed Sender

Memo (~80 bytes — the same DH-derived shared secret that produces the stealth output address also keys an authenticated symmetric encryption of the sender's published address; only the ciphertext + nonce + authentication tag are added per send, ~64-80 bytes), and an optional cancellation-hash field (~32 bytes if used)

LOCK 3,493 bytes user-signed, self re-lock

BURN 3,493 bytes user-signed, to Gehenna

REFRESH ~600 bytes user-signed, extends ACTIVE window

EXPIRE 232 bytes protocol-driven, automatic

COINBASE 264 bytes miner reward, no signature

CYCLE_SIZE = UNLOCK + EJECT + SPEND $\approx$ 11,900 bytes (first-touch, full key bundle revealed at first UNLOCK) to ~8,800 bytes (bound address, inherited binding — subsequent transactions reference the address rather than re-revealing the bundle). Every SPEND carries the encrypted Sealed Sender Memo for the recipient and the encapsulation material for stealth derivation; both reuse the same Kyber-encapsulated shared secret to minimise overhead. The reduction from the naive ~16,400 bytes (where Dilithium3 + Kyber + signature would be re-serialised at every spend) to the bound-address ~8,800 bytes is what makes a NIST-standard post-quantum chain operationally viable at settlement-grade throughput — see §Address-Bound Public-Key Reveal below.

Genesis Block Parameters

BLOCK_TIME 10 seconds protocol constant, fixed forever

BLOCK_SIZE_MAX 2 MB genesis setting, Curia variable

SCW 8,640 blocks = 24 hours, fixed forever

OR_MAX 13% accepted ceiling (not target). Blocks exceeding this orphan rate are rejected by consensus. We do not tolerate degradation by accident.

Settlements/sec ~18 at 2 MB genesis (canonical cycles at full block utilisation); scales linearly with BLOCK_SIZE_MAX

Storage/day max 17.28 GB if every block at 2 MB genesis is completely full (2 MB $\times$ 8,640 blocks/day). At genesis the chain is near-empty; actual storage approaches zero. Storage growth is adoption-driven and bounded by block size $\times$ fill rate $\times$ seconds per day, not by chain age. The transition from near-empty to fill-rate-bound takes years of organic adoption; specific GB/day projections at genesis are not meaningful.

At genesis the chain is nearly empty. Blocks are small. Storage is trivial. Coinbase fills the miner reward regardless of transaction volume. Empty blocks are not a problem — they are the natural state of an early chain. Satoshi mined empty blocks for months.

Block Size — The Single Governance Variable

Block size is the only parameter Curia ever needs to change. All other capacity parameters follow automatically.

Increase BLOCK_SIZE_MAX → more SPENDs/sec (linear)

→ more storage/day (linear)

→ higher orphan rate (capped at 13%)

→ no other protocol changes

The block size ceiling at any bandwidth is:

$BLOCK_SIZE_MAX = OR \times bandwidth_bytes/sec \times BLOCK_TIME$

At 100Mbps: $= 0.13 \times 12,500,000 \times 10 = 16.25$ MB maximum

At 1Gbps: $= 0.13 \times 125,000,000 \times 10 = 162.5$ MB maximum

Curia begins at 2MB and increases as adoption demands. The governance question is always simply: do we need more SPENDs/sec? If yes, increase BLOCK_SIZE_MAX. Storage is the cost. That is the only tradeoff.

Protocol-Driven Transitions and Limbo

The chain has multiple protocol-driven transitions that fire deterministically at their scheduled triggers: EJECT (at $unlock_height + NEXUS_DWELL_BLOCKS$, deterministic from chain state — no randomness beacon), EXPIRE (at $activate_height + 8,640$), RELEASE (at TREASURE vesting milestones), RESEAL (at OPEN VAULT window close), BOARD (at the Ragnarök boarding slot), COINBASE (at every block).

A block must include due protocol-driven transitions in priority order up to available block capacity. The priority order is: BOARD > EXPIRE > EJECT > RELEASE > RESEAL > COINBASE. Within each class, transitions are ordered by the block height of the originating event (oldest first).

(Note: SWEEP — Frau Holle's cross-address consolidation — is wallet-driven and holder-signed, not protocol-driven. It does not appear in the priority queue. The wallet submits SWEEP transactions during Menuha windows like any other holder transaction; they compete for the user-transaction portion of block space. Aurum distributes its SWEEP submissions across the Menuha window to amortise mempool load.)

Transitions exceeding block capacity extend their Limbo state to the next block.

Limbo is the architectural concept already defined for the brief interval between transition eligibility and actual block execution (see §[NEXUS] for the EJECT-side specification). Under normal load, Limbo is milliseconds — the next block always has room. Under congested load, Limbo extends to seconds or minutes as the queue drains across subsequent blocks. The transition's eligibility is preserved; only its execution is queued.

This rule guarantees liveness under any load. Blocks remain valid as long as they make progress on the priority queue up to their capacity. Transitions never disappear; they wait. The chain never stalls.

For the holder, the visible effect is that the wallet's "AWAITING EJECT" Limbo status may persist beyond the typical milliseconds during congestion. CANCEL is invalid during Limbo regardless of duration (the dwell window has technically closed at the assigned eject slot). Aurum's reassurance notification at >10 seconds of Limbo (per the UX requirements) handles this naturally.

Pathological-overflow safety. If Limbo extension consistently grows over many consecutive blocks (a pathological case implying chain misconfiguration or attack), Curia may invoke the emergency block size variable to expand capacity until backlog clears. The chain continues to execute correctly even under sustained overflow; transitions drain in priority order, no transition is lost, no chain state corrupts.

Storage Formula

$\text{Storage/day} = \text{OR} \times \text{MBPS_bytes} \times 86,400$

At 100Mbps (13% orphan): $0.13 \times 12,500,000 \times 86,400 = 137 \text{ GB/day}$ maximum

At genesis (2 MB blocks): ~17.28 GB/day if every block is completely full; near zero actual at launch (empty chain)

Block time cancels from the storage formula too. Storage per day depends only on bandwidth, orphan rate, and seconds per day — nothing else.

The Scaling Path

Genesis (2 MB) → ~18 settlements/sec, up to ~17 GB/day storage if every block is full, orphan rate ~2%

Curia (4 MB) → ~36 settlements/sec, up to ~35 GB/day, orphan rate ~3%

Curia (8 MB) → ~72 settlements/sec, up to ~69 GB/day, orphan rate ~6%

Curia (16 MB) → ~144 settlements/sec, up to ~138 GB/day, orphan rate ~13% (100 Mbps bandwidth ceiling)

1 Gbps era (160 MB) → ~1,440 settlements/sec, up to ~1.38 TB/day

One parameter. One governance vote. The chain starts on a phone and ends in an industrial complex. That is success.

Horkos

For high-value or trust-sensitive interactions, Scriptum provides Horkos. Horkos enables bilateral commitment of funds, protocol-enforced conditional execution, third-party attestation where required, and deterministic settlement or automatic refund.

Use cases include delivery-versus-payment transactions, remote exchanges without prior trust, institutional over-the-counter agreements, and any settlement where the parties want cryptographic enforcement over custodial intermediation.

Within Horkos, funds are held under protocol control at an ESCROW address. No party — not the parties to the trade, not the attestor, not the Foundation, not Curia — can discretionarily move escrowed funds. Transfer occurs only upon protocol-driven resolution per the agreement signed at the start of the trade. Failure to perform results in deterministic routing per the agreement's pre-committed terms.

Horkos replaces traditional escrow structures without introducing custodial intermediaries. The escrow is the protocol itself.

Privacy Model

Scriptum is settlement infrastructure built on a deliberate prioritisation of ideas: safety, security, controlled transfer, usability, auditability, fairness, societal reroute. **Reasonable privacy is one of these properties. Unbreakable secrecy at every level is not.** The chain provides privacy in the bank-wire sense — ordinary commercial transactions are not subject to broadcast public surveillance — but does not commit to maximum stealth as a chain-level property.

This framing is a choice, not a default. Privacy beyond the chain's baseline is available to holders through compositions of existing primitives — multiple home addresses, stealth-on-receive, view-key delegation with the epoch-bounded and rotatable construction. Holders who want stronger privacy postures construct them from the available primitives. Cryptographic anonymity through 7D is a planned Scriptum V2 capability and is not part of the v14 chain; when V2 ships it will add a STARK-based anonymity tier to the composition surface. The chain does not provide a dedicated "maximum-stealth mode" because doing so would negate or constrain other properties the chain prioritises: maximum-stealth pipelines that obscure attribution resist the consolidation patterns the chain otherwise supports, complicate the audit story that institutional users depend on, and weaken the recipient-visible provenance that sealed sender memos provide. **The chain has chosen to prioritise differently.** Holders who prioritise stealth above all other concerns can compose for it through wallet-side construction, accepting the costs; the chain itself prioritises auditability, civic legitimacy, fairness, and operational simplicity above maximum stealth.

Scriptum is not a money laundromat. We are Scriptum — that which is written. The chain is the record, attributable, auditable, recoverable. Selective disclosure is the design pattern. Subpoena power is acknowledged as outside the chain's threat model. Users seeking unaccountable concealment of illegal activity should evaluate whether Scriptum is the right tool for them; users seeking institutional-grade settlement infrastructure with reasonable privacy and built-in audit will find the chain's properties match their needs.

This framing is deliberate. The chain has competing interests with stealth: auditability requires verifiable attribution; institutional positioning requires compliance compatibility; societal reroute (the Tithe-funded vaults) requires the chain to be a legitimate civic

instrument; constitutional governance requires verifiable on-chain operations; fairness requires structural transparency on allocation and vesting. **The chain chooses these commitments and accepts that they constrain the stealth claim.** Holders who want stronger privacy postures can compose them from the available wallet-side primitives; the chain provides the primitives without claiming maximum stealth as the chain's purpose.

Scriptum is not a drug dealer's hideout. The architecture is auditable by design. Selective disclosure is the design pattern. Subpoena power is acknowledged as outside the chain's threat model. Users seeking unaccountable concealment of illegal activity should evaluate whether Scriptum is the right tool for them; users seeking institutional-grade settlement infrastructure with reasonable privacy and built-in audit will find the chain's properties match their needs.

The Privacy Stack

Scriptum's privacy stack is built from four primitives, each providing a specific property and composing with the others into the chain's overall posture.

Stealth derivation per receive. Every SPEND derives a fresh one-time output address via Kyber/ML-KEM key encapsulation between the sender (who encapsulates an ephemeral shared secret) and the recipient's published Kyber view key. No persistent on-chain identifier links incoming payments to a recipient's published address. Every receive is a cryptographically distinct on-chain identity, derivable as the recipient's only by the recipient's view key. This is the chain's foundational pseudonymity primitive: receives are unattributable to a published identity by chain analysis alone.

Movement-side consolidation. UNLOCK consolidates value from many of a holder's stealth-derived addresses to the holder's HOME address, producing ACTIVE Parcel at HOME after the NEXUS dwell window. Frau Holle's Menuha-time cross-address sweep operates similarly across the holder's full address cluster, also producing output at HOME. Stealth-derived receive addresses become empty after consolidation; the holder's effective storage location is HOME for ACTIVE/operational use, with new receives continuing to land at fresh stealth-derived addresses until the next consolidation.

Multiple home addresses (baseline composition). A holder may operate multiple home addresses, each its own branch of the deterministic key tree derived from the seed, each with its own Dilithium signing key, Kyber/ML-KEM encapsulation key, view key, and stealth-derived address ecosystem. A holder can route value between their own home addresses via standard SPEND or 7D anonymous transfer. This composition provides practical compartmentalisation between separate operational roles — a holder may use one home address for everyday spending, another for trading, another for long-term holding, with no on-chain linkability between the three for an observer without view-key access. The chain provides the primitives (deterministic derivation, stealth output construction, 7D); the wallet provides the management surface (per-address labelling, per-address view-key derivation, routing controls). The cost is operational complexity proportional to the number of home addresses the holder elects to manage. For holders

who prefer simplicity, a single home address with the chain's baseline stealth-derived receive ecosystem is sufficient for most purposes.

Sealed sender memo. Every SPEND carries an encrypted memo addressed to the recipient (using the recipient's Kyber view key as the encryption target). The memo carries the sender's published address by default, and may carry additional content within the size budget. Recipients decrypt the memo and know which address sent them value — preserving the address-level provenance property that recipients have on transparent chains today. External observers cannot read the memo without the recipient's view key.

The memo is the chain's per-transaction audit primitive: holders can selectively disclose memos to auditors, regulators, or counterparties to prove specific transactions without revealing their broader on-chain footprint. The memo is mandatory in form (every SPEND carries a well-formed encrypted memo) but optional in content (holders choose what to put in the memo, within the minimum requirement of the sender's published address).

Architectural Note: Validity Versus Anonymity

The four primitives above provide bank-wire-grade pseudonymity — strong protection against chain-only adversaries, with full holder-controlled audit. They do not provide cryptographic anonymity. The distinction matters for any reader trying to map Scriptum's privacy stack onto the deeper cryptographic literature.

Validity is mechanical. Every transaction's authority to spend is verified by signature and hash check — the spender produces a Dilithium3 signature over the transaction, the chain verifies the signature against the registered public key. This is ancient cryptography; hashes commit and signatures bind. No zero-knowledge machinery is required for the chain to enforce that only authorised holders move value.

Anonymity is what zero-knowledge buys. The primitive that hides *which* authorised credential among many was used — without revealing the credential itself — is the zk-proof. STARKs and SNARKs are the families; both prove “I know a witness satisfying this circuit” without disclosing the witness. Scriptum's planned 7D layer (Scriptum V2, post-Genesis; see §7D glossary entry for the v14 status framing) uses STARK-based ring membership proofs over fixed-denomination commitment trees to deliver this property. **At v14 the chain ships pseudonymity + recipient unlinkability (stealth addressing) as the cryptographic privacy baseline; STARK-based cryptographic anonymity through 7D is the planned extension, with its consensus-rule specification developed under the Foundation's research-and-standardisation track post-Genesis.** The reason 7D requires denomination tiers — and not arbitrary amounts — is structural: cryptographic anonymity is tractable when the primitive must hide exactly *one* variable (set membership), and intractable when it must hide many. Fixed denominations make amount a public label, leaving leaf identity as the only hidden variable. The planned 7D set (Doc 100, Grumpy 200, Happy 500, Bashful 1000, Sleepy 2000, Sneazy 5000, Dopey 10000) sits at the engineering optimum balancing collision density per tree against per-payment proof cost — wallets decompose arbitrary amounts into these denominations automatically.

Anonymity scales with denomination collision, not with cryptographic cleverness.

STARKs hide which leaf in the tree was spent; the tree's anonymity set is determined by how many holders deposited at the same denomination. Without forced collision through fixed denominations, every depositor's amount would be its own near-empty tree, and STARKs would protect credential sets of size one. The seven denominations are infrastructure for collision; the cryptography rides on top of that collision.

This architectural truth is the design principle behind Scriptum's tiered privacy stack. The bank-wire tier (stealth + view keys + sealed memo) requires no zk machinery and has no anonymity-set cost. The cryptographic-anonymity tier (7D) accepts the infrastructure cost (STARK prover, denomination decomposition, per-payment proof) in exchange for credential-set unlinkability. Holders choose the tier appropriate to each transaction; the chain provides both, and is honest about what each delivers.

What the Chain Provides and Does Not Provide

What Scriptum provides:

Privacy against chain-only adversaries. Chain analytics firms, competitors, casual chain readers, surveillance services without subpoena power cannot link senders to recipients without view-key access, cannot identify a holder's address cluster across compartmentalised identities, cannot determine balances of named entities, and cannot see the topology of payment flows between specific parties.

Address-level recipient identification via decrypted memos. Recipients see who sent them value (in the form of the sender's published address per the memo) — the same property they have on transparent chains today, delivered cryptographically rather than via on-chain visibility.

Audit-grade compliance via selective disclosure. Holders can disclose individual memos for per-transaction audit (narrowest scope) or delegate a home address's view key for per-identity ecosystem audit (broader scope). Disclosure may be **range-bounded** by block-height range, so that the disclosed view-key cryptographically decrypts only outputs in that range, and **revocable forward in time** through master rotation, after which the old view-key no longer decrypts new outputs. Together the two refinements give holders precise control over the temporal scope of every disclosure they make. See §Selective Disclosure below for the construction.

Per-address compartmentalisation through multiple home addresses. Holders may operate multiple home addresses derived from the same seed, each its own ecosystem of stealth-derived receive addresses, each appearing on chain as an independent address cluster to an observer without view-key access. Routing between a holder's own home addresses via standard SPEND or 7D anonymous transfer breaks on-chain linkability between the holder's operational compartments. The chain provides the primitives; the wallet provides the management surface. The cost is operational complexity proportional to the number of home addresses managed.

What Scriptum does not provide:

Anonymity in the strong cryptographic sense. The chain is not designed to defeat state-level adversaries with subpoena power, who can compel KYC disclosure from CASPs, view-key disclosure from auditors, and identification through off-chain context. Scriptum does not pretend to provide this. It provides structural pseudonymity against chain-only adversaries plus full holder-controlled audit, and accepts that off-chain compulsion is outside the chain's threat model.

Cryptographic amount privacy at launch. Amounts are visible in plaintext on chain transactions. The chain's positioning as institution-grade settlement infrastructure accepts this trade-off; institutional users typically have audit obligations that require amount disclosure, and the chain's amount transparency simplifies that compliance. The architecture supports upgrade-ceremony amendment to add cryptographic amount commitments in a future Phoenix if real institutional demand materialises; this is not a launch commitment.

Symmetric receive-and-spend privacy. Stealth-on-receive protects the recipient at the moment of receipt; it does not protect the recipient retroactively against linkage that emerges later. The asymmetry is structural and worth naming honestly. When a holder spends value originally received at a stealth-derived address, the change/remainder returns to the holder's HOME address by default per CM-3.3.x. If an external observer has, by any means, previously associated HOME with the holder's real-world identity — through earlier multi-input UNLOCKS that consolidated stealth addresses to HOME, through off-chain identity leaks, through social-graph correlation, through subpoena, or through any other channel outside the chain's control — then the moment the holder spends from the stealth-received Parcel, that observer can retroactively link the original receive to the holder. Concretely: the holder receives 100M at a fresh stealth address; the receive looks unlinkable on chain at the moment it happens; the holder later spends 10 from it; the chain shows the 10 going to a stealth-derived destination (the recipient remains protected) and the ~99,999,990 of change going to the holder's HOME address; an observer who already knows that HOME belongs to the holder now sees, retroactively, that the 100M was a payment to the holder. The recipient of the 10 remains structurally private; the original sender's privacy in their send-to-the-holder is conditional on the holder's subsequent behaviour. This is the bank-wire-privacy property: HOME is the chain-visible identity of the holder to anyone who has ever linked it; transactions in and out of HOME are observable as a coherent profile over time. Holders who require stronger spender-side privacy — for example, the property that the original sender's privacy in sending-to-the-holder is preserved against retroactive correlation — must use wallet-level structural privacy (the address-tree topology described in the Aurum technical paper, §Super-Stealth Mode), where value received at stealth addresses is internally re-routed to deliberately specialised branches before any spend, keeping the spend-and-change pattern within the branch and never returning value to HOME. The chain provides the cryptographic primitives; complete spender-side unlinkability is a wallet-layer construction, not a default chain property.

Forensic-grade unlinkability. Pattern analysis with off-chain context, behavioural profiling, timing correlation across many transactions, and graph-clustering of inferred holder behaviour are all possible threats that the chain's primitives do not structurally

defeat. Holders concerned about forensic-grade adversaries should compose the chain's available primitives — multiple home addresses, routing via 7D, disciplined receive-address hygiene, view-key non-disclosure — and accept the operational complexity that comes with maximum-privacy operation.

Adoption Is Privacy

The privacy property is set-anonymity with k scaling to active user base. With a small total user population, full-observation deanonymisation is feasible regardless of cryptographic primitives — distinctive timing rhythms, distinctive value sizes, distinctive interaction graphs leak attribution through pattern analysis. With sufficiently large active user base, the combinatorial space of plausible clusterings becomes intractable for chain-only adversaries. A new holder joining a million-holder chain inherits the anonymity set immediately. The same new holder joining a ten-holder chain has very little privacy.

This is honest about how the privacy model works in practice. Scriptum's privacy strength is not a fixed cryptographic claim but a property that grows with adoption. The chain becomes more private the more it is used.

Ratatoskr — Chain-Traversal Enumeration

A holder's wallet, given the holder's key pair, can identify which Parcels on the chain belong to that holder by performing Kyber/ML-KEM decapsulation against each SPEND's ephemeral encapsulation material and matching the derived candidate addresses against the SPEND's outputs. This scan is the receive-side privacy property in action: only the holder's view key can recognise the holder's Parcels.

Ratatoskr is the named primitive for this scan operation. **It runs in the wallet (or in an archive node serving a holder's authorised query), not as protocol consensus machinery.** The chain's consensus does not have access to holder key pairs and therefore cannot identify which addresses belong to which holders; only parties with view-key access (the holder's wallet, or auditors to whom the holder has explicitly delegated) can perform the Ratatoskr scan. Given a holder's key pair as input, Ratatoskr traverses the chain and reports the address set derivable from those keys. The traversal is the same DH-against-ephemeral-key matching a wallet performs locally; the Ratatoskr name elevates it to a first-class architectural concept and specifies its interfaces.

Ratatoskr has one function: report. It does not consolidate, does not authorise, does not modify chain state. Wallets and auditors consume Ratatoskr's reports for their own purposes downstream — wallets to populate balance displays and to construct cross-address SWEEP transactions during Menuha, auditors to obtain the holder's complete address set without scanning the chain themselves.

The naming reuses the figure of Ratatoskr — the squirrel running up and down Yggdrasil carrying messages between the eagle at the top and the dragon at the roots — to name a primitive that traverses the chain reporting what it finds. The original Ratatoskr in the V_A architecture has been renamed and split into Hugin and Munin (see §16.4) freeing the name for this enumeration role.

Audit via Ratatoskr

A holder who delegates audit authority does so by providing the auditor with the view key (per the holder-controlled audit model). The auditor with the view key invokes Ratatoskr to obtain the holder's complete address set without scanning the chain themselves. The audit becomes a single Ratatoskr invocation rather than a multi-day chain scan, regardless of chain age or holder transaction history depth. Combined with sealed sender memo decryption, the auditor reconstructs the holder's complete sender-recipient-amount picture for every incoming and outgoing transaction.

Wallet Recovery via Ratatoskr

Routine wallet operation does not pay the scan cost — wallets maintain caches populated by Ratatoskr's reports and updated incrementally as new blocks arrive. The full-scan cost matters at one specific moment: wallet recovery from seed when the local cache is unavailable.

Two protocol-level optimisations make recovery operationally viable at mature chain scale.

View-tag specification. Each SPEND output carries a small precomputed hint (a view-tag, typically a single byte or a few bytes) that lets a scanner reject most non-matching outputs in a single byte comparison, reserving the full Kyber/ML-KEM decapsulation for the small fraction that pass the tag check. This reduces per-output scan cost by approximately 40x without weakening the privacy property — view-tags leak no information beyond what the address itself already leaks. Scriptum specifies view-tags as part of the stealth output format. This is a launch commitment, not deferred to future protocol evolution.

Remote-archive scan recovery. Wallets recovering from seed may transmit their view key (over an authenticated, encrypted channel) to an archive node. The archive performs the scan and returns a filtered result containing only the holder's addresses and Parcels. The wallet's recovery time becomes proportional to the holder's own activity rather than to the entire chain's. This is a substantial usability improvement at mature chain scale and does not introduce privacy concerns beyond the holder's deliberate disclosure of their view key to a chosen archive. Scriptum specifies remote-archive scan recovery as part of the Sanctum archive node specification (see §[Node Types]).

With both specifications in place, wallet recovery on a mature chain (year 99 of Phoenix 1, hypothetical full chain volume) takes hours rather than weeks. The privacy property is operationally viable at any chain scale.

Selective Disclosure

A holder operating a home address possesses a 32-byte **master view secret** for that home address: a private value held only by the holder's wallet, derived from the holder's mnemonic via a wallet-specific path. The master view secret is the root from which all per-epoch view material derives. The chain never sees, derives, or computes any view-key secret.

A holder may delegate view-only access to a third party — an auditor, a tax preparer, a compliance recipient, a custodian, an heir, a legal discovery counterparty — by exporting a per-epoch view secret. The delegate gains the ability to decrypt the holder's stealth-derived inbound outputs received during the disclosed epoch but cannot spend; spending requires the holder's Dilithium3 signing key, which is never derivable from any view secret.

The selective-disclosure architecture supports two operations: **annual epoch publication** and **master rotation**. Both are consensus-level transactions; both are surfaced to the holder by their wallet, typically as automatic background hygiene.

Epoch-Bounded View-Key Disclosure

Time on the chain is partitioned into UTC calendar years. Each block belongs to exactly one calendar year, determined unambiguously by its timestamp. An output's epoch is the epoch of the block that confirms it — the realisation epoch in tax-equivalent terms.

For each calendar year in which a holder intends to receive private outputs, the holder publishes an **epoch view pubkey** on chain via the PUBLISH_EPOCH_PUBKEY transaction. The pubkey is derived privately by the holder's wallet from the master view secret and the target epoch year; the corresponding secret stays with the holder. Senders send to a home address by reading the holder's published epoch pubkey for the intended inclusion epoch and encrypting the output to it.

When a holder needs to disclose visibility for a specific year — say, a 2025 tax audit — the holder exports the **epoch view secret** for 2025. The export consists of the Kyber768 secret key for the 2025 epoch keypair plus context (the home address, the epoch year, the generation identifier). The auditor's tool, given this export, scans Scriptum blocks whose timestamps fall in 2025 and attempts ML-KEM-768 decapsulation against each output's ciphertext using the exported secret. Successful decapsulations are the holder's 2025 inbound outputs; the auditor sees exactly the year's history.

The cryptography enforces the boundary:

- The auditor cannot derive any other year's secret. Per-epoch secrets are derived from the master via a one-way key derivation function (HKDF-SHA3-512) with the epoch year mixed into the derivation input; given any one epoch's secret, computing another epoch's secret would require inverting HKDF, which is infeasible under standard assumptions about SHA3-512.
- The auditor cannot derive the master view secret. Same HKDF one-way property.
- The auditor cannot decrypt outputs in years other than the disclosed one. Each year's outputs are encrypted under that year's epoch pubkey; ML-KEM-768's IND-CCA2 property means that secret keys for other years cannot decapsulate ciphertexts encrypted under the disclosed year's pubkey.

A holder may disclose multiple years to the same auditor (export multiple epoch secrets) or different years to different auditors (export each year's secret to one auditor). Each disclosure is bound to its specific year by the cryptography of the derivation, not by wallet policy or social trust.

Master Rotation

A holder may rotate their home address's view authority via the CANCEL_VIEWKEY transaction. Rotation advances the home address to a new view-authority generation: the holder's wallet generates a fresh 32-byte master view secret, derives the corresponding new master pubkey, and submits the rotation transaction signed by the home address's Dilithium3 spending key. The chain appends a new generation entry to the home address's view authority state.

After rotation:

- Outputs received at the home address **before** the rotation block remain decryptable under the **prior** master view secret. The holder retains the prior master in their wallet for historical decryption; any previously-exported epoch view secrets derived from the prior master continue to decrypt the outputs they could already decrypt.
- Outputs received at the home address **after** the rotation block are encrypted under the **new** generation's epoch pubkeys. The holder must publish a fresh epoch pubkey under the new generation before senders can construct private outputs for inclusion in any given epoch. Previously-exported epoch secrets cannot decrypt post-rotation outputs in the same year (the master-binding property ensures the new generation's epoch derivations are independent of all prior generations').

Rotation is the operational forward-revocation primitive. A holder cannot undisclose what they have already disclosed — the bytes have left their custody and the recipient retains them, and the cryptography of broadcast ciphertexts cannot be un-broadcast — but the holder can ensure that any previously-exported view-key sees no further activity. Rotation is forward-only; it does not retroactively invalidate any output already broadcast and confirmed.

Mid-year rotation produces a “two-segment year” for that epoch: outputs from the start of the year to the rotation block are encrypted under the old generation's epoch pubkey; outputs from the rotation block forward are encrypted under the new generation's epoch pubkey (once the holder has published it). A holder seeking full-year audit visibility after a mid-year rotation exports both segments' epoch secrets to the auditor. This is the unavoidable consequence of forward-only revocation, not a flaw in the architecture.

CANCEL_VIEWKEY is fee-free at consensus level. Self-protection against view-authority compromise is operational hygiene, not a chargeable service.

What the Holder Sees, What the Chain Sees, What the Delegate Sees

The chain sees: every home address's view-authority generations (list of master pubkeys, advancing on rotation) and every home address's published epoch pubkeys per generation. All public. The chain never sees any secret material.

The holder sees: their 32-byte master view secret (in their wallet). From it, they can derive any past or future epoch view secret on demand. They can scan any year's blocks against their own outputs. They retain prior generations' master secrets for historical decryption after rotation.

The delegate sees: exactly the epoch secrets the holder has exported. For each exported epoch secret, the delegate can decrypt the holder's inbound outputs in that epoch under the relevant generation. The delegate cannot derive any other epoch's secret, cannot derive the master, cannot derive secrets for other home addresses, and cannot decrypt outputs encrypted under a different generation's pubkey for the same epoch.

Boundary Honesty

The architecture is honest about what it does and does not provide. A holder who publishes their epoch pubkey for 2026 and gives an auditor the 2025 epoch secret has bounded the auditor's visibility to 2025. If the holder broadcasts a transaction in the last seconds of December 2025 UTC and the inclusion block lands a few seconds into 2026 UTC, the sender's wallet may have encrypted to the wrong epoch's pubkey; the output may be undecryptable to the recipient. This is the sender's problem and the chain does not engineer around it — no grace zones, no dual-encoding to current and next epoch, no boundary buffer. Wallets warn users near boundaries; the chain enforces the strict rule.

A holder who forgets to publish a new year's epoch pubkey becomes unreachable for private outputs in that year. Senders' wallets refuse to construct outputs to a home address with no published epoch pubkey for the inclusion epoch. The holder can come back online at any time, publish, and resume receiving. The discipline is the same as maintaining a forwarding address with a bank.

The strongest accurate public claim about the construction's selective-disclosure property is:

Possession of an epoch-bounded view key does not grant decryption capability outside that epoch and generation.

Claims stronger than this — retrospective revocation, perfect privacy, magical audit invisibility, universal deniability — are not supported by the architecture and are not made.

Forward Reference to CM

The cryptographic specification of epoch-bounded view-key derivation, CANCEL_VIEWKEY, PUBLISH_EPOCH_PUBKEY, and the home address view-key state schema is in the Canonical Machine: CM-16 (Epoch-Bounded View Keys), CM-3.15 (CANCEL_VIEWKEY), CM-3.16 (PUBLISH_EPOCH_PUBKEY). The parameter values — KDF identifier, KEM identifier, HKDF salt, domain-separator bytes, seed sizes — are in SPX-21 (Selective Disclosure Parameters). The invariants that the construction must preserve and the fuzz target surfaces for parameter validation are in IA Part II.12 and Part IV.2. The WP describes the architecture and the holder-facing semantics; the corpus locks the byte-level rules and the testable correctness properties.

What This Replaces

Earlier versions of this Whitepaper described view-key delegation in terms of block-height ranges ([start_block, end_block]) — a holder could export a view-key bounded to an

arbitrary block range, and the auditor saw exactly the range's contents. That model is superseded by the epoch-bounded yearly model described above. Annual epochs match the natural reporting period for tax and audit work; the cryptographic construction is simpler than arbitrary-range; the wallet UX is cleaner ("show the auditor 2025" instead of "show the auditor blocks 4,839,200 through 7,983,600"). Arbitrary-range disclosure is a future extension if real operational need arises post-Genesis.

Earlier versions also described the construction as deriving epoch pubkeys from a publicly visible master view pubkey via a deterministic key generation. That construction was cryptographically flawed: a deterministic key generation seeded from public material produces public secrets. The v13.9 architecture corrects this: epoch keypairs derive privately from a 32-byte holder secret; only the epoch pubkey lands on chain; the secret stays with the holder. See CM v13.9 changelog for the full account of the correction.

Wallet-Layer Encryption

Aurum's local address cache, transaction history, and view-key state are stored encrypted under keys derived from the holder's view key (in addition to the at-rest passphrase encryption protecting the entire wallet blob). This composition means partial exposure of wallet state — for instance via memory-scraping malware or cloud backup leak — does not directly yield the holder's address set; an attacker would need the view key to make the cache useful, at which point they would also have direct on-chain query capability and the cache adds no marginal value to them.

Wallet-layer encryption uses post-quantum-secure primitives consistent with the chain's Kavaca commitment. The symmetric encryption (AES-256-GCM or ChaCha20-Poly1305) provides 128-bit effective security against Grover-equipped quantum adversaries — well above any practical attack threshold. Key derivation uses Argon2id with parameters tuned to give meaningful resistance to brute-force attack on the holder's passphrase, with the explicit guidance that holders should use sufficiently high-entropy passphrases (or rely on the BIP39 seed as the underlying secret, with Argon2id stretching it) to remain strong against quantum-aided brute-force. Any asymmetric component of the wallet's encryption stack — for instance, key wrapping for cloud backup, sync between devices, or hardware-key integration — uses Kyber/ML-KEM rather than classical Diffie-Hellman or RSA.

Aurum supports a multi-tier sovereign recovery archive — the Lifeboat — for device loss, offline cold backup, opt-in trustee/inheritance recovery, and generational continuity across Phoenix migrations. The Lifeboat uses a post-quantum encryption stack consistent with Kavaca and supports optional enhancements including Shamir Secret Sharing for distributed recovery, geographic share distribution, and configurable deadman-switch protocols. The Lifeboat is what makes the chain's centennial Ragnarök architecture operationally complete at the holder level: a holder's recovery materials can survive a century-scale horizon and enable heirs to board successor Phoenixes on the holder's behalf. Full specification of the Lifeboat — recovery tier mechanics, archive format, trustee architectures, deceased-holder boarding workflow — is in the Aurum technical paper.

The wallet-layer encryption is a defence-in-depth measure that complements the chain's stealth-derivation, sealed sender memos, and NEXUS-protected paths. Implementation details — Argon2id parameters, key derivation pipeline, secure-enclave integration, hardware-key handling, sync transport, recovery semantics — are specified in the Aurum technical paper. The principle the protocol commits to is: **everything the wallet encrypts is encrypted with primitives that survive store-now-decrypt-later attacks, consistent with Kavaca at the chain layer.**

Address Space and Cluster Behaviour

A holder's published address — HOME — is the canonical identifier they share with senders, created by their wallet at first launch from the holder's Dilithium and Kyber key pair. This address does not directly hold the holder's Parcels on a stealth-by-default chain. Sends to the holder land at fresh stealth-derived one-time addresses; the holder's wallet (with their view key, via local Ratatoskr scan) recognises these as theirs.

Over time, an active holder accumulates Parcels at many one-time stealth-derived addresses. Each UNLOCK consolidates value to HOME via the NEXUS dwell as a side effect of the operation: the consumed source addresses become empty husks (their bundles revealed but their Parcels gone), and value emerges at HOME as ACTIVE Scriptum ready for use. SPEND, LOCK, EXPIRE, and REFRESH all operate on the consolidated state at HOME.

For an active user — one who unlocks regularly enough that consolidating UNLOCKS occur naturally — the non-empty address count drifts up between transactions and gets pulled down at every multi-address UNLOCK. The count stays low through normal use; cross-address SWEEPs are unnecessary for active users. For an accumulator — one who receives often but rarely unlocks — the count grows until it crosses the cross-address threshold (an address count exceeding three, where address count means the number of distinct addresses currently containing the holder's Parcels as reported by the wallet's local Ratatoskr scan). The wallet (Aurum by default) constructs and submits a cross-address SWEEP transaction during the next Menuha that consumes eligible LOCKED Parcels across all of the holder's addresses and produces one consolidated LOCKED Parcel at HOME. The non-empty count drops to one.

In both cases the steady state immediately after consolidation is HOME holding the consolidated value. HOME is stable across consolidations — UNLOCK and Frau Holle both deliver to HOME — providing operational simplicity for the holder and predictable wallet behaviour.

UNLOCK and Frau Holle operate within a single home address's address space. A holder using a single home address has one address space, consolidated to that home address. A holder operating multiple home addresses has separate address spaces per home address, each consolidating independently to that home address. The chain's consolidation primitives respect home-address boundaries automatically because the holder's wallet selects coins from within a single home address when constructing UNLOCK and SWEEP transactions.

A pathological case is structurally absorbed. A holder who deliberately fragments by repeatedly unlocking and re-locking small amounts at a single address creates large Parcel count at that address. This is operationally self-punishing — each cycle costs the holder f_U , f_A , and f_S — and does not break the chain. If the holder later receives at three other stealth addresses (crossing Frau Holle's address-count threshold), the next Menuha sweep consumes all Parcels at all the holder's addresses, including the heavy-fragmented one, into a single consolidated Parcel at Y. The protocol's per-operation per-summed-value fee model means the sweep handles arbitrary input counts cleanly. The chain is robust to fragmentation in either direction: active users self-consolidate via use, pathological accumulators self-consolidate via Menuha.

Node Types — Genesis Requirements

Scriptum defines four node types with different storage and bandwidth requirements. All four are valid participants in the network. Only Archive Nodes (Sanctum) are required to retain full history.

Home Mining Node

Bandwidth: 100Mbps minimum

Storage: ~145 GB (Parcel set + 8,640 blocks = 1 SCW window)

Hardware: Any modern CPU

Role: Mines blocks, validates recent chain, pruned to SCW depth

Cost: ~€150 storage, standard home internet

Dedicated Mining Node

Bandwidth: 100Mbps+

Storage: 500 GB SSD

Hardware: GPU rig or high-core CPU

Role: Mines blocks, keeps 30 days history, faster validation

Cost: ~€500 storage, standard broadband

Full Validation Node

Bandwidth: 100Mbps+

Storage: 2TB+ recommended; near-zero growth at genesis (empty chain), up to 17.28 GB/day at saturated 2 MB blocks; growth scales with adoption and Curia block size votes

Hardware: Any CPU, server-grade preferred

Role: Validates complete chain, serves light clients

Storage growth at genesis depends on actual transaction volume, not just block size. Empty blocks at cold-start consume only header space.

Sanctum — Archive Node

Bandwidth: 1Gbps recommended

Storage: near-zero at genesis (empty chain); scales with adoption to a maximum of 17.28 GB/day at saturated 2 MB blocks; growth scales further with each Curia block size vote

Hardware: Server-grade, redundant storage

Role: Complete chain history from genesis to present

Operated by: Exchanges, explorers, institutions, Scriptum Foundation

Sanctum storage projections by adoption tier (block-size assumed at the bracket's column; growth driven by sustained block-fill rate over time):

Tier	Sustained fill	Block size	Storage/year	Cumulative @ this tier × 5 yrs	Hardware tier
Light	5–15%	2 MB	0.5–1.5 TB	2.5–7.5 TB	4×1TB NAS RAID-5 (~€200)
Moderate	30–50%	2 MB	3–6 TB	15–30 TB	4×4TB NAS RAID-5 (~€400)
Heavy	60–80%	4–8 MB	15–50 TB	75–250 TB	6×8TB NAS RAID-6 (~€800)
Institutional	80%+	16+ MB	100+ TB	500+ TB	Datacenter rack (~€2,000+/yr)
Mature	80%+	160 MB (1 Gbps era)	500+ TB	2.5+ PB	Institutional SAN (~€20,000+/yr)

The tier a Sanctum operator finds themselves in depends on the chain's actual adoption trajectory, not on the calendar. A chain that takes ten years to reach Moderate tier accumulates the same storage as one that reaches it in two years; the operator's hardware planning follows the trajectory rather than fixed time milestones.

Recommended storage architecture: ZFS RAIDZ2 (2-drive failure tolerance, self-healing). Consumer NAS operators (Synology, QNAP) can run Sanctum comfortably through the Moderate tier. Beyond that, storage requirements scale with chain activity, and Sanctum operation becomes an institutional role by design. Running a Sanctum archive at the Heavy tier and beyond is a significant operational commitment — appropriate for exchanges, explorers, sovereign archivists, and research institutions that have natural incentives to

maintain full history once Scriptum carries serious value. Sanctum is not free at scale. The burden grows in proportion to what it protects.

Storage cost halves approximately every 2-3 years historically. A Sanctum that costs €800 in hardware today will cost €200 equivalent in 10 years. The economics improve faster than the chain grows. Sanctum is not a burden. It is a privilege.

Sanctum is the permanent memory of the Scriptum chain. It stores every transaction ever confirmed, every Parcel lineage, all Lethe records, all Ragnarok migration manifests, and all Phoenix ARC records. Sanctum is not required for protocol operation. It is required for trust, compliance, and history.

At Ragnarok, the originating Phoenix Sanctum is frozen — permanent, immutable, readable forever. The successor Phoenix begins a new Sanctum at block 0. Every Phoenix leaves its Sanctum behind. Nothing is lost. Everything is remembered.

Sanctum nodes are incentivised by market forces: light nodes and applications pay small query fees to Sanctum operators. The protocol does not mandate payment but the market creates the incentive naturally.

Consensus and Finality

Chain Selection

The valid chain is the chain with the highest accumulated proof-of-work. Block 0 of every Phoenix is accepted by deterministic validation alone — it is verified against the ARC manifest, not by proof-of-work. Proof-of-work begins at block height 1.

Confirmations

Each transaction type requires a minimum number of confirmations before it is considered final:

UNLOCK → 3 confirmations

EJECT → 3 confirmations (protocol-driven)

CANCEL → 3 confirmations

SPEND → 10 confirmations

LOCK → 3 confirmations

BURN → 3 confirmations

EXPIRE → 3 confirmations (protocol-driven)

SWEEP → 3 confirmations (protocol-driven)

COINBASE → 100 confirmations

PACK → 3 confirmations

BOARD → 10 confirmations

HORKOS_AGREE → 3 confirmations

HORKOS_LOAD_A → 3 confirmations

HORKOS_LOAD_B → 3 confirmations

HORKOS_CONFIRM → 3 confirmations

HORKOS_ATTEST → 3 confirmations

Horkos resolution UNLOCK → 3 confirmations (protocol-driven)

Horkos resolution EJECT → 3 confirmations (protocol-driven)

Horkos resolution SEND → 3 confirmations (protocol-driven)

CANCEL requires only 3 confirmations to remain accessible throughout the SCW window. SPEND and BOARD require 10 due to their irreversible and high-value nature. COINBASE outputs require 100 confirmations before they enter the spendable lifecycle.

DISEMBARK is not listed above. It is not a transaction — it is a protocol event executed at Block 0 of the successor Phoenix. No confirmation count applies. It is instantaneous and deterministic at the moment of Rebirth.

Fork Handling

Competing blocks may occur. Only one chain persists. Discarded transactions return to the mempool.

Terminal Block Height — RAGNAROK_HEIGHT

Ragnarök is enforced as a consensus-level terminal block height.

The Scriptum protocol defines a fixed block height, RAGNAROK_HEIGHT, at which the active Phoenix reaches its terminal state. This height is deterministically known from genesis parameters and cannot be altered by any participant, miner, or fork without departing from the Scriptum protocol.

RAGNAROK_HEIGHT = 315,360,000

No block with height > RAGNAROK_HEIGHT is valid under the Scriptum protocol. Any node receiving such a block must reject it. A chain extending beyond RAGNAROK_HEIGHT is not a continuation of Scriptum — it is a fork that has departed from the protocol definition.

Open Source and Community Review

Scriptum is open source from genesis. The code, the specification, and this whitepaper are public. The community is the first and most important line of review.

Critical issues identified post-launch are addressed through the hard fork mechanism defined in the Scriptum Governance Charter. A hard fork requires community consensus. It is not failure — it is governance in action. Bitcoin has forked. Ethereum has forked. Every serious chain has a mechanism for this. Scriptum is no different.

A formal independent security audit will be commissioned from the Reserve allocation (5% of total supply) when resources permit. Until then, the open source codebase and community review constitute the primary security assurance.

Network Behaviour and Mempool Policy

Nodes prioritize transaction processing using a two-factor mempool handicap model:

$$H = F1 \times F2$$

Lower H = higher priority. Two factors:

F1 — Transaction Type Factor

CANCEL: F1 = 1 (highest priority — safety action, always accessible, never penalised)

SPEND: F1 = 2 (low penalty — completion of value transfer)

UNLOCK: F1 = 3 (highest penalty among user-initiated actions — preparatory, churn-prone)

EJECT is not user-initiated. It is a protocol-driven transition that fires deterministically at `unlock_height + 8,640`. EJECT does not enter the mempool and is not subject to F1 prioritisation.

BURN, LOCK, and DONATE follow F1 = 2 (treated as outward value motion at parity with SPEND for prioritisation).

HORKOS_AGREE, HORKOS_LOAD_A, HORKOS_LOAD_B, HORKOS_CONFIRM, HORKOS_ATTEST follow F1 = 2 (treated as outward value motion at parity with SPEND for prioritisation).

F2 — Value Tier Factor

F2 is assigned per transaction type based on amount and intent. Higher tier = lower F2 = higher priority.

Tier 10 (>€10M) → F2 = 1 (highest priority)

Tier 9 (€2M–€10M) → F2 = 2

Tier 8 (€1M–€2M) → F2 = 3

Tier 7 (€500k–€1M) → F2 = 4

Tier 6 (€200k–€500k) → F2 = 5

Tier 5 (€100k–€200k) → F2 = 6

Tier 4 (€50k–€100k) → F2 = 7

Tier 3 (€10k–€50k) → F2 = 8

Tier 2 (€1k–€10k) → F2 = 9

Tier 1 (€1–€1k) → F2 = 10 (lowest priority)

F2 assignment by transaction type:

SPEND → tier-based on amount (as above)

UNLOCK → tier-based on amount (preparatory, so F1 already penalises)

BURN → tier-based on amount (deliberate action, moderate priority)

LOCK → F2 = 1 (safety/sovereignty action, no penalty)

CANCEL → F2 = 1 (safety action; highest priority under honest network policy)

EJECT → F2 = 1 (protocol-driven, never deprioritised)

EXPIRE → F2 = 1 (protocol-driven, never deprioritised)

PACK → F2 = 1 (Arc procedure, user-initiated, time-sensitive)

Safety-critical actions — CANCEL and EJECT — are always assigned F2 = 1 regardless of value tier. They cannot be structurally deprioritised by any mempool implementation. This is non-negotiable: economic weight must never prevent a holder from cancelling an unwanted UNLOCK.

Mempool policy is non-consensus and may vary between nodes, but implementations are expected to follow these principles. Two principles are foundational to honest node behaviour: first, no fee level or value tier may delay CANCEL within the SCW; second, CANCEL must be includable in the next valid block under normal network conditions. CANCEL is always admitted to the mempool at maximum priority. These are core security properties of the SCW — not policy recommendations.

PART III — USE

Aurum — Wallet and User Experience

Aurum is the reference wallet for Scriptum. It implements the full CLASS/ROLE/STATE model, SCW management, all transaction types, UIS integration, the complete Arc procedure, the chain's privacy primitives (stealth-derived receive, 7D anonymous transfer, sealed sender memo, view-key delegation with range-bounded and rotatable construction), and multi-home-address management for holders who want operational compartmentalisation.

This Whitepaper describes the chain protocol Aurum operates against. The wallet's full architecture — including its UI design, deterministic key derivation, multi-home-address management, encrypted local cache, recovery semantics, REFRESH heuristics, Horkos attestor module if the holder chooses to operate as an attestor, and platform-specific implementation details — is specified in the **Aurum technical paper**, a standalone document parallel to this Whitepaper and to the Scriptum Governance Charter.

Aurum is open-source software, published on GitHub under Apache 2.0 and distributed from dfqr.org. Users download it voluntarily, install it on their own devices, and run it under their own control. The Scriptum Foundation does not push Aurum into App Store marketplaces, does not market it to specific jurisdictions, and does not target users by geography. The software is available; the choice to use it is the user's.

This is the same model under which Bitcoin Core, Monero, and every other self-custodial open-source wallet operates. The publication of open-source software on a website is not a regulated activity in any major jurisdiction. Users in every jurisdiction are responsible for their own compliance with their own local law.

Aurum is maintained and updated by the Foundation. The Foundation complies with the law of its jurisdiction of incorporation. It does not “launch in” other jurisdictions because there is nothing to launch — only software to publish. A resident of any country who downloads Aurum and runs it is exercising the same right they exercise when they download any other piece of open-source software: the right to run code on their own machine.

Core Flow

UNLOCK → NEXUS dwell (24h SCW) → EJECT → ACTIVE → SPEND / LOCK / BURN / REFRESH / WAIT

UX Requirements (Summary)

The Aurum technical paper specifies the wallet's full UX. The summary requirements that appear at the protocol layer:

Full CLASS/ROLE/STATE visualisation per Parcel

NEXUS dwell countdown timers per Parcel

CANCEL functionality throughout NEXUS dwell, surfacing both signature-based and (where configured) hash-preimage authorisation paths

Limbo state displayed explicitly: LIMBO — AWAITING EJECT · NO ACTION POSSIBLE. Aurum shows a reassurance notification if Limbo exceeds ~10 seconds (typical Limbo duration is milliseconds to under a minute).

REFRESH switch for ACTIVE Parcels — surfaces the time remaining in the ACTIVE window and offers a single-click extension. Aurum surfaces this prominently when an ACTIVE Parcel has less than 25% of its window remaining.

LOCK screen with coin selection and remainder preview

BURN screen with coin selection and remainder preview

Fee preview in both Scriptum and € at current V_A

UIS Protected Mode warning with additional confirmation

Arc procedure: enrollment, boarding timeline, straggler status, countdown

DONATE screen — dedicated action, sends to Chesed, receives Dana Parcel, fee = standard f_S per SPX-4 (DONATE is structurally SPEND with a vault destination), private by default

Direct vs Private SPEND mode selection on outward sends, with clear labelling of trade-offs (speed vs cancellation safety)

Multi-home-address management for holders who want operational compartmentalisation (see Aurum technical paper for the full surface)

Markets and Aurum

Aurum Market and any exchange are off-chain systems used for price observation and early trading. They are not required for protocol operation. They do not feed into consensus.

Markets determine the external price of FUNK. The protocol does not depend on them. Price discovery — including the bootstrap path through Aurum Market, exchange listings, and CoinGecko — is economically useful but protocol-irrelevant. The chain operates identically whether Scriptum trades at €1 or €1,000,000. Markets observe the protocol. The protocol does not observe markets.

V_A is designed to keep fees economically meaningful and predictable — not to track price with precision. The market sets the price of FUNK. The protocol uses a smoothed approximation for fee calibration only.

Scriptum as a Traded Asset

Scriptum is designed for deliberate value transfer. Store of value and tradability are not opposites. FUNK will be traded. It is expected, welcomed, and supported. The mechanisms that define Scriptum — the SCW, Menuha, the fee structure — do not inhibit trading. They stabilise it.

The Exchange Model

Exchanges interact with Scriptum through two natural tiers:

Buying → always instant

Exchange maintains an ACTIVE pool

Buyer sends EUR, receives FUNK immediately

Exchange manages the unlock schedule

No waiting for the buyer

Selling → planned, 24 hours ahead

Seller initiates UNLOCK

SCW runs for 24 hours

SPEND executes after EJECT

Exchange receives FUNK, pays EUR

This asymmetry is a feature. Sell pressure is always visible 24 hours in advance. The market can absorb it. Panic selling is structurally impossible — you cannot dump in milliseconds what takes 24 hours to unlock. Buy pressure is instant. The result is natural price stabilisation without price control.

What the SCW Prevents

Flash crashes → impossible. Selling requires 24h planning.

Bot-driven dumps → impossible. No instant execution.

Instantaneous automated draining → prevented. Any drain attempt must complete its observable SCW before value leaves the chain; speed is not a weapon.

Panic cascades → dampened. Market absorbs visible sell pressure.

What the SCW Does Not Prevent

Normal trading → fully supported

Exchange listing → fully supported

Price discovery → fully supported

Speculation → fully supported

Long-term holding → fully supported

Institutional custody → fully supported and preferred

Menuha and Volatility

Menuha — the Sunday rest — removes one day per week from SPEND execution. This is not an arbitrary restriction. It introduces a weekly rhythm to value movement. By rejecting SPEND, BURN, and DONATE on Sunday blocks, Menuha removes continuous execution as an attack surface. Strategies that depend on uninterrupted automated trading cannot operate across the weekly closure. The pause is the point.

Scriptum does not prevent speculation. It prevents the kind of speculation that destroys value rather than discovers it. High-frequency trading, bot-driven volatility, and flash dynamics are not compatible with the SCW. Everything else is welcome.

Menuha Sweep (SWEEP / Frau Holle) — Cross-Address Consolidation

Menuha is not only a pause in outward execution. It is the window during which Scriptum performs cross-address consolidation. SWEEP — known in the semantic narrative as Frau Holle, the mythological cleaning-goddess who puts the house in order once the week has closed — handles the cross-stealth-address fragmentation that emerges naturally from the chain's mandatory stealth-compatible output construction. Without cross-address consolidation, a holder receiving sends across many one-time stealth-derived addresses would accumulate fragmentation indefinitely.

SWEEP is cross-address only. There is no per-address SWEEP scope. Per-address consolidation is performed by UNLOCK as a side effect of normal operation: every UNLOCK that consumes Parcels from one or more of the holder's stealth-derived addresses consolidates those source addresses' Parcels into a single NEXUS-dwelling Parcel destined for HOME via smallest-first coin selection. UNLOCK is the chain's per-address consolidation primitive; SWEEP exists specifically to handle the cross-address case that UNLOCK alone cannot address (because UNLOCK only consumes addresses the holder explicitly chooses to include).

SWEEP is wallet-driven and Aurum-mandatory. The chain's consensus cannot identify which stealth-derived addresses belong to which holders without view-key access — that knowledge lives only in holders' wallets and in any auditor delegations the holder has explicitly granted. The wallet (with the holder's view key, via local Ratatoskr scan) computes the address set, identifies when the count exceeds three, and constructs a multi-input SWEEP transaction during the next Menuha that consolidates eligible LOCKED Parcels to HOME. The wallet signs with the relevant addresses' Dilithium keys; the chain validates signatures like any other transaction. **Aurum's wallet implementation constructs and submits SWEEP transactions automatically at every Menuha when the threshold is crossed, with no opt-out.** From the holder's perspective using Aurum, SWEEP is mandatory and unavoidable. The protocol specification mandates the same behaviour for any compliant Scriptum wallet.

The mandate lives at the wallet/specification layer rather than at consensus because the chain itself cannot identify a holder's address set. A non-compliant wallet that skipped SWEEPs would not be a Scriptum-compliant wallet, but the chain has no mechanism to force the behaviour. The holder using such a wallet would experience their own

fragmentation cost (larger UNLOCK transactions, cluttered UI in any wallet) without breaking the chain. Aurum's compliant default is the chain's operational guarantee.

Cross-address SWEEP fires only during Menuha Windows. The threshold is the number of addresses holding Parcels (address count exceeding three), not the count of Parcels at any single address — within-address fragmentation is handled by UNLOCK or never arises in the first place under normal operation. The destination is uniformly HOME — the holder's published operational address, derivable by the wallet from the holder's keys.

Active users typically self-consolidate via normal multi-address UNLOCKS to HOME during the week and rarely cross the SWEEP threshold; SWEEP is the safety net for accumulators (recipients of many incoming payments who do not actively unlock) and for pathological-fragmentation cases.

No value moves to a non-holder address. No authority changes. Only representation is simplified, and only within the holder's address space. The economic property is preserved (operation-based fee model; SWEEP is fee-free as housekeeping).

Scope

SWEEP applies to Parcels of CLASS = CIRCULATING and CLASS = VAULT held at stealth-derived addresses across the holder's address cluster. TREASURE Parcels are not swept — they have their own VEST/RELEASE lifecycle and fragmentation during vesting carries semantic meaning. MARKER Parcels are not swept — they are PERMA_LOCKED and non-spendable by design, one Mayflower or Dana per event. ESCROW-addressed Parcels (those held pending Horkos outcomes) are excluded from SWEEP — they carry trade-specific metadata and pre-committed destinations, and consolidating them would break the Horkos settlement machinery.

Eligibility

A holder's stealth-derived address cluster is eligible for SWEEP at block height H if and only if all of the following hold: (1) H is within a Menuha Window, evaluated via Median Time Past per SPX-6; (2) the wallet's local Ratatoskr scan reports that either (a) the holder's address count exceeds three, or (b) any one of the holder's addresses holds three or more LOCKED Parcels of the same (class, role) — see SPX-7 SWEEP_TRIGGER_ADDRESS_COUNT and SWEEP_TRIGGER_PARCEL_COUNT_PER_ADDRESS; (3) the wallet has not yet submitted a SWEEP for this Menuha Window. The wallet constructs the SWEEP transaction with all eligible LOCKED Parcels across the holder's addresses (excluding TREASURE, MARKER, and ESCROW per the scope rules) and submits it during the Menuha Window. Wallets distribute their SWEEP submissions across the Menuha Window to amortise mempool load.

Eligibility is evaluated independently for each (A, C, R) tuple. An address holding fragmented LOCKED Parcels across multiple (class, role) combinations may be swept in each combination separately.

Operation

For each eligible (A, C, R) tuple, the protocol consumes every Parcel matching the tuple and produces a single successor Parcel at the same address, same class, same role, state = LOCKED, amount = sum of consumed amounts. The creation_height of the successor Parcel is preserved as max(input creation_heights), which correctly maintains COINBASE_MATURITY clocks for swept COINBASE Parcels.

Binding Rules

Because the binding is a property of the address and consensus-enforced to be constant, all LOCKED Parcels at the same address share the same binding by definition. SWEEP exploits this invariant to merge without ambiguity. If none of the consumed Parcels carries a Public Key Binding, the resulting Parcel is unbound. If one or more consumed Parcels carry a binding, all present bindings MUST be identical (guaranteed by the address hash commitment property), and the resulting Parcel carries that binding. If conflicting bindings are detected, the block is rejected as invalid. This is a consensus check, not an optimisation.

Fees and Inclusion

Menuha Sweep is protocol-driven and does not incur transaction fees. It is a service to the network, consistent with the fee philosophy that safety and sovereignty actions are free. Sweep operations are mandatory — a block during Menuha that omits any required sweep is invalid and rejected by all nodes, subject to the same inclusion discipline as EJECT, EXPIRE, and RELEASE.

Constraints

Menuha Sweep MUST NOT alter ownership, address, class, or role; MUST NOT modify a binding except through the inheritance rule above; MUST NOT trigger SCW or ACTIVE transitions; MUST operate solely on LOCKED state and produce LOCKED state; MUST NOT execute outside a Menuha Window.

Why Three (Two Fuses)

SWEEP fires under two complementary structural fuses, either of which independently suffices. The address-count fuse triggers when a holder's address cluster exceeds three distinct addresses with Parcels — natural commerce across home and stealth addresses occupies two or three slots; a fourth signals that fragmentation has crossed from balance management into accumulation. The per-address parcel-count fuse triggers when any single address holds three or more LOCKED Parcels of the same (class, role) — natural consumption (main balance plus a recent change output) occupies two; a third signals that consumption is not keeping pace with accumulation at that address. The two fuses cover the two fragmentation modes: thin spread across many addresses (address-count) and concentrated accumulation at few addresses (per-address parcel-count). Either alone leaves a class of accumulators unswept; together they ensure SWEEP fires whenever protocol-level housekeeping pays for itself. Both thresholds are governance-adjustable

under standard Curia parameters (SPX-7 SWEEP_TRIGGER_ADDRESS_COUNT and SWEEP_TRIGGER_PARCEL_COUNT_PER_ADDRESS).

Scriptum does not cache public keys. It eliminates redundancy at the identity layer. The address commits to the binding; every Parcel at the address inherits it; SWEEP consolidates by identity rather than by lookup. No cache. No registry. No repetition. Two tolerated. Three signals. One restored.

Horkos

Horkos is Scriptum's bilateral settlement primitive. It enables two parties to settle a trade where one side — Scriptum — is on-chain and the other — X — may be off-chain (fiat, asset, custodial transfer, physical goods, services rendered). A third party named by the parties at commit time, whose public key is bound to the contract at Stage 1, attests to the off-chain delivery. The Scriptum side is enforced cryptographically; the off-chain side is attested. The protocol does not arbitrate trade fairness — A, B, and C negotiate terms directly and the protocol enforces what they agreed. The full Horkos parameter set including specific encoding, transaction formats, and validation rules is specified in SPX.

The name *Horkos* (Greek Ὁρκος) refers to the personification of sworn oath in Hesiod's *Works and Days* — son of Eris, punisher of perjurers. The fit is structural: a Horkos contract is a sworn agreement with cryptographic consequences for breach. The skin-in-the-game stake ($S \geq 0.1\%$ of P) and the deterministic protocol-driven resolution implement the oath's enforcement; the attestor is the witness whose verdict triggers the forfeit. Earlier specifications referred to this primitive as "Delayed Atomic Settlement" (DAS); the rename to Horkos in v12 unifies the naming convention with Scriptum's mythological vocabulary (Heimdall, Kavaca, Hayyānu, Ilmarinen, Bifröst).

Participants

A — the party committing Scriptum. A is selling Scriptum or otherwise placing Scriptum at risk in exchange for some off-chain deliverable from B.

B — the party delivering X (the counter-asset, fiat transfer, or other off-chain obligation). B places skin in the game in the form of a stake denominated in Scriptum; the stake is what B forfeits if B fails to perform.

C — the attestor. C is the third party that the parties have chosen to verify the off-chain leg of the trade. C verifies the off-chain delivery and signs HORKOS_ATTEST confirming YES (delivered) or NO (not delivered). C holds no funds; C is paid an attestation fee for honest work, regardless of attestation direction. The choice of C is the parties' own — counterparty risk for the attestor (liveness, honesty, competence) belongs to A and B, not to the protocol.

ESCROW — a protocol-controlled wallet address. No user holds its key. Funds at ESCROW addresses move only through protocol-driven sequences triggered by attestation outcome. Neither A, nor B, nor C, nor the Foundation, nor Curia can move escrowed funds discretionarily.

Negotiated Terms and Structural Floors

A Horkos contract is fully parameterised by what the parties agree:

P — A's principal, the Scriptum A is committing to the trade.

S — B's stake, denominated in Scriptum. The protocol enforces a structural floor: S must be at least 0.1% of P. Above the floor, the parties choose the stake amount that reflects their assessment of trade significance and counterparty trust.

z — the attestation fee A pays C for verification work. Negotiated by the parties; the protocol does not constrain the amount.

W — the lock-to-approval window length, in blocks. The protocol enforces two structural bounds: W must be at least 360 blocks (1 hour at 10-second block time), and W must not extend beyond the next Ragnarök. Within those bounds, parties choose what fits their trade — short windows for fast off-chain legs, long windows for complex or cross-jurisdictional deliveries. The 1-hour floor is the structural minimum that gives an honest attester enough time to verify the off-chain delivery without being structurally rushed; parties may negotiate longer freely.

The structural floors prevent zero-stake Horkos (which would be a SPEND with extra steps), prevent windows too short to permit honest attestation, and prevent contracts that would otherwise straddle the centennial Phoenix transition. Within those floors, the protocol provides primitives; the parties provide judgement. The protocol is the registrar of the agreement, not a party to it.

The Five-Stage Lifecycle

Horkos proceeds through five stages, each with explicit triggers and transitions.

Stage 1 — Agreement. A, B, and C each sign a multi-sig agreement transaction recording the negotiated terms (P, S, z, attester identity, W). The protocol verifies all three signatures and validates the structural floors. If verification passes, the agreement is recorded on-chain. No funds have moved. The contract is in INTENT state.

Stage 2 — Load window. A 24-hour window opens from the moment the agreement is accepted. During this window, A loads P plus z plus the chain fees applicable to the load operations. B loads S plus chain fees. The chain tracks each party's loaded amount against the agreement and reflects loading status. If the 24-hour window closes with one or both parties incompletely loaded, the contract lapses. Loaded principals and stakes return to their respective loaders. Chain fees paid during loading are not refunded — the chain has performed real verification and tracking work, and the chain keeps its compensation for that work whether or not the contract ultimately locks.

Stage 3 — Final confirmation. With both A and B fully loaded, all three parties sign a second multi-sig confirmation transaction. This is the explicit "we are now committed" signature from each party. The protocol verifies all three signatures and transitions the contract from FUNDED to LOCKED. The safety is now off; the contract is binding.

Stage 4 — Lock-to-approval window. The window of length W runs. C may submit `HORKOS_ATTEST` (YES or NO) at any point during the window. B performs the off-chain delivery. The contract is in `LOCKED` state during this window — funds are escrowed, no party can move them, the protocol awaits an attestation event or a window expiry.

Stage 5 — Resolution. Triggered by C 's `HORKOS_ATTEST` submission or by expiry of W . The protocol-driven `UNLOCK` → `EJECT` → `SEND` sequence executes the resolution per the outcome. Resolution executes through the chain's standard transaction primitives, not through Horkos-specific transfer logic. Because chain fees were paid at load (Stage 2), resolution is fee-free at the chain level.

Outcomes

The Stage 5 resolution routes funds based on the attestation outcome.

YES — C attests delivery occurred. The protocol releases A 's principal P to B (the trade completes in B 's favour); B 's stake S returns to B (B performed honestly and recovers their collateral); z is paid to C . Everyone receives what they agreed to.

NO — C attests delivery did not occur. A 's principal P returns to A (the trade does not complete; A keeps their *Scriptum*). B 's stake S is slashed and routed via the standard Tithe split: 90% to the miner, 10% to the protocol Tithe (which then splits internally 48/32/10/10 across WFC, Funds, RESERVE, and Charity Pot per §15). z is paid to C — honest attestation work is compensated regardless of direction.

TIMEOUT — the lock-to-approval window expires with no attestation. A 's principal P returns to A . A 's z refunds to A — the attestation service was never rendered. B 's stake S splits 50/50: half returns to B , half is routed via the standard Tithe split. C receives nothing — silence is not work.

In all three outcomes, the routing is consensus-enforced and protocol-driven. No party — not A , not B , not C , not the Foundation, not Curia — can alter the outcome once the attestation event has triggered the resolution sequence.

Menuha Behaviour

Only the agreement phase of Horkos is Menuha-prohibited: the Stage 1 agreement multi-sig, the Stage 2 load transactions (`HORKOS_COMMIT` and `HORKOS_STAKE`), and the Stage 3 final confirmation multi-sig commit new outward value to the contract and cannot be submitted during Menuha. A Horkos contract whose 24-hour load window would expire during Menuha has its deadline extended to post-Menuha. Stage 4 (`HORKOS_ATTEST`) and Stage 5 (resolution) are unaffected. `HORKOS_ATTEST` is informational — attestor C recording a delivery decision, with no value crossing to a counterparty — and is valid at any block including during Menuha. Stage 5 resolution is protocol-driven (`UNLOCK` → `EJECT` → `SEND` with no user signatures) and runs through Menuha by default.

A Horkos contract whose attestation finalises just before or during Menuha completes its resolution sequence on schedule. Only new agreements cannot be initiated during the rest

day. The chain rests outward-value user activity; it does not rest the recording of delivery decisions or its own internal state machine.

Why This Design

Trustless bilateral settlement against off-chain delivery. Most cryptocurrency atomic-settlement primitives (HTLC, Lightning) work only for chain-to-chain swaps where both sides are cryptographic. Horkos extends atomic-settlement guarantees to bilateral trades where one side is off-chain — fiat payments, physical goods, services, real-estate transfers, art exchange — provided the off-chain side can be verified by an attestor.

Protocol as registrar, not arbiter. The chain does not decide how much principal A should commit, how much stake B should post, how much C should be paid, or how long the approval window should run. Those decisions belong to the parties. The chain enforces nominal structural floors (stake $\geq 0.1\%$ of principal as protocol minimum, window ≥ 1 hour, window $<$ remaining time to Ragnarök) and procedural rules (multi-sig at agreement and confirmation, fees at load, protocol-driven resolution). The 0.1% stake floor is the consensus-level minimum; it is not a recommendation for any specific commercial use. Within those constraints, the parties shape the contract.

Skin in the game with negotiated proportionality. B's stake reflects what B and A together have decided about counterparty risk for this specific trade. The 0.1% protocol floor prevents zero-stake Horkos; commercial use cases will typically negotiate stakes substantially higher (5%, 10%, 25%, or more depending on the trust profile and the value at stake). Combined with public on-chain attestation outcomes, this creates a permanent on-chain record at each address — a verifiable trade history at protocol level rather than at platform level.

Fees at load, resolution free. The chain charges its fee when it does its work — verifying agreement, tracking loading, recording confirmation. Once the contract is locked, the chain has been compensated for handling it. Resolution executes through standard primitives at no further charge. Lapse before lock returns principals and stakes minus the fees already paid; the chain keeps what it earned by tracking the failed attempt.

Resolution through standard primitives. Stage 5 does not use Horkos-specific transfer transactions. It uses the chain's standard UNLOCK $\rightarrow$ EJECT $\rightarrow$ SEND pipeline, driven by the protocol rather than by user signatures. This means Horkos inherits all of the chain's standard properties — observability, auditability, Menuha-permeability for protocol-driven flow, Tithe routing for slashed funds — without requiring any Horkos-specific consensus rules at the resolution step.

Custody without a custodian. ESCROW addresses without private keys mean no party can divert escrowed funds. The Foundation cannot freeze a Horkos trade. Curia cannot reroute it. The chain's outcome is determined by the attestation per the agreement signed at Stage 1.

Complements exchange trading. Horkos is not designed to replace exchanges. Exchanges provide continuous liquid trading against an order book. Horkos provides deterministic

bilateral settlement for large, pre-arranged, or cross-asset trades where custodial intermediation adds cost without adding value. Typical Horkos use cases: institutional OTC, large fiat-for-Scriptum purchases, peer-to-peer trades through Aurum Market, and any settlement where parties prefer cryptographic enforcement over venue execution.

Ás-brú — Fast-Rail Exchange Bridge

Ás-brú is Scriptum's fast-rail bridge between Scriptum on chain and currencies held at centralised exchanges. It enables a holder of Scriptum to trade directly with a holder of fiat or another cryptocurrency at a named exchange, without an attestor, without exchange cooperation, and without leaving the Scriptum protocol layer. The chain enforces the Scriptum side cryptographically; miners observe the off-chain side via the exchange's public API and include the release transaction when the matching transfer is confirmed.

Ás-brú complements rather than replaces Scriptum's other settlement primitives. Horkos covers slow-rail and attestation-required trades — fiat wires, physical goods, cross-jurisdictional deliveries — where parties negotiate the window and engage a third-party attestor. Aurum Market covers direct peer-to-peer trades over external rails (IBAN, hand-to-hand) where reputation alone is the trust layer. Ás-brú occupies the middle: same-exchange internal transfers, fast settlement, no attestor required, miner consensus as the off-chain confirmation source. Holders gain exchange-rail liquidity from day one, regardless of whether Scriptum is formally listed at any specific venue.

The name *Ás-brú* refers to Bifröst, the bridge of the Æsir connecting Midgard to Asgard in Norse cosmology. The bridge spans two domains; it bears load while it stands; it has known limits. Ás-brú as a Scriptum primitive carries the same connotation: a reliable, narrow-purpose connector between the on-chain economy and exchange-rail liquidity, structurally bounded so the bridge does not exceed its load.

Participants

A — the party committing Scriptum. A is the Scriptum-side counterparty, locking principal P in protocol-controlled ESCROW for the duration of the trade.

B — the party transferring off-chain. B holds funds at the named exchange and transfers them to A's exchange address. B has no Scriptum-side commitment beyond receiving the released Scriptum at a stealth-derived address.

ESCROW — a protocol-controlled wallet address, derived deterministically from the trade parameters. No user holds its key. Funds at ESCROW addresses move only through protocol-driven sequences: ASBRU_RELEASE on miner confirmation, ASBRU_TIMEOUT at window expiry.

Miners — the consensus-active subset of the network at the moment of trade execution. Miners run Crowglass — Scriptum's exchange observation daemon — which queries the named exchange's public API for committed Ás-brú trade fingerprints. The Crowglass daemon is a standard component of the Ilmarinen node; miners participating in Ás-brú validation choose which exchanges they observe. The miner who builds a block within the

trade's window includes ASBRU_RELEASE in that block if Crowglass reports a matching confirmed transfer; miners who find no match include nothing for this trade and proceed normally. A single miner's confirmation is sufficient — once ASBRU_RELEASE is in a confirmed block, the trade resolves. Miners earn the trade's chain fee for this work, in addition to standard block rewards.

The security model is observer diversity rather than curated trust. The protocol does not verify which exchange the trade references or whether the exchange is reputable; the protocol verifies only that ASBRU_RELEASE was validly included. The chain relies on rational miner behaviour — Crowglass-running miners observe the exchanges they choose to observe and reach block-level consensus through standard validation. A miner choosing to observe a fraudulent exchange wastes their effort because honest peers do not confirm the observation.

There is no attestor C. The exchange is a passive data source, not a participant. The exchange does not sign anything for the protocol, holds no chain-registered keys, and bears no obligation to Scriptum. Ás-brú extracts a chain-verifiable signal from the exchange's public API through miner observation rather than through exchange cooperation.

Fixed Parameters and Trade Terms

Ás-brú is structurally simpler than Horkos. The protocol fixes the window; the parties specify the trade terms at commit time.

W — Window of trust (fixed). ASBRU_W is hard-coded to 60 blocks (approximately 10 minutes at 10-second block times). Parties cannot negotiate W. This fixes Ás-brú's positioning as fast-rail-only: trades whose off-chain leg cannot complete within 60 blocks should use Horkos, not Ás-brú. The fixed window also simplifies miner monitoring — every Ás-brú trade has the same lifecycle duration, and miners can budget their API polling cadence accordingly.

P — A's principal. The Scriptum amount A commits to ESCROW. Constrained by DUST_THRESHOLD and ASBRU_MIN_VALUE (per SPX-6).

Exchange identifier. A canonical specifier (hashed for chain storage) of the exchange the parties agree to use for the off-chain leg. The protocol does not maintain an admitted-exchange registry and does not reject covenants by exchange identifier. Parties choose the exchange at commit time; miners running Crowglass against that exchange provide the observation; the chain enforces what was committed. The Foundation publishes a recommended Crowglass operational configuration listing exchanges considered operationally appropriate for Crowglass observers (Kraken at v1.0, more as operational experience accrues); the recommendation is informational, administered at Foundation-board cadence per GOV §11.3 and §12.1, and is not protocol-binding. Parties may name any exchange; miners may observe any exchange they choose. The chain's consensus is over what miners actually included, not over whether the chain approves of their exchange choices.

Off-chain leg fingerprint. The trade parameters that miners use to identify the matching off-chain transfer: B's exchange-side source identifier, A's exchange-side destination identifier, transfer amount, currency, and a unique discriminator (memo, reference, or per-trade deposit address) sufficient to distinguish this trade's transfer from any other transfer at the same exchange in the same window. The fingerprint is committed in the ASBRU_COMMIT transaction so the chain has an immutable record of what miners are looking for.

B's destination. A stealth-derived Scriptum address where the Scriptum is delivered upon RELEASE.

There is no stake S . Ás-brú is a single-collateral primitive: A locks the Scriptum, B transfers off-chain. Asymmetry is intentional — the off-chain leg has no chain enforcement, so a stake from B would simply lock symmetric capital without adding cryptographic guarantees.

There is no attester fee z . No third party is paid; miners are compensated through the standard chain fee on ASBRU transactions.

The Five-Stage Lifecycle

Ás-brú resolves through five stages:

Stage 1 — Off-chain coordination. A and B agree on terms outside the protocol: which exchange, what amount of Scriptum against what amount of off-chain currency, the relevant exchange addresses or accounts. Aurum's UI surfaces a price oracle (queried from the same exchange's public market data) to suggest a fair rate; final terms are agreed bilaterally. No chain interaction occurs at this stage.

Stage 2 — ASBRU_COMMIT. A submits ASBRU_COMMIT, locking principal P plus chain fees into ESCROW. The commit transaction encodes: the exchange identifier hash, the off-chain leg fingerprint, B's stealth destination, and the commit-block height (W 's clock starts here). Miners validate the commit like any other transaction; the Scriptum is locked from the moment the commit is confirmed.

Stage 3 — Off-chain transfer. B initiates the transfer at the named exchange. The transfer must complete and become visible on the exchange's public API within W blocks of the commit. The protocol does not observe this stage directly; it occurs entirely off-chain on the exchange's infrastructure.

Stage 4 — Miner observation. Each block built within the W window triggers the building miner to query the named exchange's API for the committed fingerprint. The query returns either "match found" (the off-chain transfer is visible) or "no match" (transfer not yet observed). The miner who finds a match includes ASBRU_RELEASE in the block they build; miners who find no match include nothing for this trade and proceed normally. A single miner's confirmation is sufficient — once ASBRU_RELEASE is in a confirmed block, the trade resolves.

Stage 5 — Resolution.

ASBRU_RELEASE routes the principal P from ESCROW to B's stealth destination through the chain's standard UNLOCK → EJECT → SEND pipeline, protocol-driven, no further user signature required. Chain fees accrue to the miner who included the release.

ASBRU_TIMEOUT fires at $\text{commit_block} + W$ if no ASBRU_RELEASE has been confirmed. The protocol returns P from ESCROW to A through the same standard pipeline. A bears the chain fees consumed at commit; the trade closes without prejudice.

The five-stage structure mirrors Horkos's lifecycle but compressed. There is no separate attest stage because there is no attestor; the miner's act of including ASBRU_RELEASE is the attestation, witnessed by the consensus they produce.

Menuha Interaction

Ás-brú follows the same Menuha rules as other value-transfer primitives. ASBRU_COMMIT is invalid during the Menuha Window (Saturday 23:00 UTC through Sunday 23:00 UTC, evaluated by Median Time Past per §Menuha) — new trades cannot be initiated during the rest day. ASBRU_RELEASE and ASBRU_TIMEOUT remain valid through Menuha — trades in flight at Saturday close resolve on schedule, even if the resolution block is mined during the rest day. The chain rests outward-value initiation; it does not rest the closure of decisions already in progress. This matches the Horkos rule and the protocol's general posture toward Menuha.

Edge Cases and Failure Modes

Hostile miner refuses to include a confirmed release. A miner observes the off-chain transfer via the API but declines to include ASBRU_RELEASE in their block. The trade does not resolve in that block. The next miner builds the next block, queries the API, observes the transfer, and includes the release. Worst case: a few blocks of delay, well within W's 60-block budget. The protocol's robustness against single-miner malice comes from the rotation of mining hashrate rather than from a multi-miner threshold.

Genuine API outage during W. The exchange's API is unavailable for all miners during the window. Miners observe nothing; ASBRU_TIMEOUT fires; A's Scriptum returns. If B did transfer, B has lost the off-chain leg without receiving Scriptum. Mitigation is reputational: B can re-initiate the trade once the API is healthy. Ás-brú does not pretend to atomicity across infrastructure failures it cannot detect; the timeout-and-refund path is the honest fallback.

Network partition affecting some miners but not others. Different miners see different API states. A miner who can reach the exchange includes the release; one who cannot, does not. The first miner to confirm wins; the partition is irrelevant to the trade's outcome.

Exchange changes API endpoints during W. The committed exchange identifier hash references a specific endpoint. If the endpoint changes mid-trade, miners querying the old endpoint receive errors. The trade times out as if the API were down. Exchange API changes are slow events (typically announced months in advance) and Aurum's per-exchange integration code handles version migration.

B initiates the transfer just before the W expiry. The transfer is in flight when W closes. Miners querying at the last block see no confirmed transfer; ASBRU_TIMEOUT fires; A's Scriptum returns. Defence is procedural: Aurum surfaces the W countdown prominently, and B's wallet refuses to initiate transfers within a configurable buffer of the deadline (default: refuse if less than 20 blocks remain).

Why This Design

Bilateral trust simplification. Ás-brú strips out the third-party attestor that Horkos requires. For trades that resolve in minutes via fast exchange rails, the attestor is structural overhead — the parties don't need a witness because the API is the witness. Removing the attestor eliminates the negotiation step (who is C?), the attestation fee (z), and the attestation-honesty-floor that justifies Horkos's 360-block window minimum. Two parties, fixed window, miner consensus as the confirmation source — simpler primitive, narrower use case, cleaner contract.

Fixed window discipline. ASBRU_W = 60 blocks, no negotiation. This forces Ás-brú into the role it is designed for: fast-rail, same-exchange internal transfers that complete within minutes. Trades that need longer windows must use Horkos with an attestor, which is the appropriate primitive for slower rails. Forbidding negotiation eliminates the failure mode where parties agree to a window too short for their rail or too long. The protocol's stance is that fast and slow are different products; Ás-brú is the fast one.

Miner-monitored, miner-paid. Miners already operate the heaviest infrastructure on the chain. Adding API integration to mining software is incremental work; the standard chain fee on ASBRU transactions compensates the additional cycles. No new participant class is introduced — no oracles, no attestors, no monitoring registry. Ás-brú reuses mining as a side channel for off-chain observation, with payment aligned to the work performed.

Single-miner sufficiency, redundancy through rotation. Any miner who builds a block within W can confirm. The protocol does not require M-of-N attestations because the chain's existing consensus already provides honest-majority assumptions over the rotation of mining hashrate. A hostile miner refusing to include a release simply yields the work to the next miner; the trade resolves with at most a few blocks of delay. Single-miner sufficiency keeps the consensus rules simple.

Recommended exchange baseline, open at protocol level. The chain does not maintain an admitted-exchange registry. Ás-brú trades may reference any exchange the parties agree to; miners running Crowglass choose which exchanges they observe. The Foundation publishes a recommended Crowglass operational configuration — exchanges the Foundation considers operationally appropriate for Crowglass observers based on regulatory standing, asset segregation, API stability, audited operations — with Kraken as the initial entry (MiCA-compliant, established, regulated, audited). The recommendation is informational, administered at Foundation-board cadence per GOV §12.1 and §11.3, and is not protocol-binding. Aurum surfaces the recommended set as user-facing trade-construction defaults; users with different preferences may name any exchange, and a non-Aurum wallet may do the same. The open-protocol architecture (no protocol-level gate, no wallet-licensing key) is dictated by the Apache 2.0 licence and the chain's open-source

commitment: a protocol-level gate would either provide no real protection (extractable key in open source) or recentralise the chain (Foundation gatekeeper).

Half-atomic, honestly named. Ás-brú does not claim cryptographic atomicity across both legs. The Scriptum side is fully enforced — escrow, miner-witnessed release, deterministic timeout. The off-chain side relies on the exchange’s public API and miner observation, which together produce a strong but non-cryptographic signal. The protocol does not pretend otherwise. Holders trading via Ás-brú accept the asymmetry: cryptographic enforcement on what the chain controls, observational consensus on what it does not. Counterparties who require fully cryptographic atomicity should use Horkos with an attested rail.

Reputation backs the residual risk. Default behaviour by either party — A refusing legitimate trades repeatedly, B initiating transfers and not following through — accrues to permanent on-chain records at the relevant addresses. The reputation layer that Aurum Market established for direct peer-to-peer trades extends naturally to Ás-brú. Counterparty selection is the user’s responsibility; the chain provides the public record from which selection can be informed.

Complements the settlement stack. Scriptum’s settlement primitives form a graduated set: HOME SPEND for instant pseudonymous payments, NEXUS for cancellable cold spends, 7D for cryptographically anonymous transfers, Aurum Market for direct peer-to-peer trades, Horkos for attested off-chain settlement, Ás-brú for fast-rail exchange bridges. Each fills a specific niche; no two are redundant. Ás-brú’s contribution to the stack is *exchange-rail liquidity without exchange cooperation* — a property no other primitive provides, and one that makes FUNK practically tradeable from day one regardless of formal listing progress.

Kiva — The Discovery Hall

Kiva is the discovery layer of the Scriptum ecosystem. It is a communal hall sitting above the settlement primitives — Aurum Market, Horkos, Ás-brú — in which Aurum holders make themselves visible to one another, post structured Ás-brú offers, and find counterparties for bilateral trade. Kiva does not execute trades, does not hold funds, does not become a counterparty. It is a discovery primitive, not a marketplace.

Without Kiva, two holders who do not already know each other have no protocol-defined way to find one another. The chain provides addresses but no directory; the wallets provide privacy but no presence; the settlement primitives provide bilateral negotiation but no introduction. Kiva fills exactly that gap and nothing more. Once two holders have found each other through Kiva, Kiva steps aside and the interaction becomes direct Aurum-to-Aurum settlement through whichever primitive the parties choose.

The name *Kiva* refers to the Pueblo ceremonial chamber — the underground room where the community gathers for shared business, distinct from private dwellings, distinct from the central plaza. The kiva is the place where members of a community become visible to one another by choice and for purpose, then return to their separate lives. Scriptum borrows the name and the architecture: a hall a holder enters by intent, in which their

identity becomes visible by design, from which they leave when they're done, and which never holds anything they bring with them.

Participation Model

Kiva membership is opt-in and symmetric. A holder who has not joined Kiva is invisible to other holders in Kiva and cannot see other holders in Kiva. A holder who joins Kiva becomes visible to other holders and can see them. There is no asymmetric mode; visibility and observation are paired by construction.

The first time a holder enters Kiva, Aurum surfaces an explicit consent screen that states plainly what entering Kiva means: the holder's home Scriptum address is published to the discovery hall, other Aurum holders will see the address along with any tags and offers the holder posts, and the address's on-chain history — completed trades, defaults, publicly-delegated view-key holdings if any — is verifiable by anyone in the hall. The consent is explicit; there is no default-on join.

Aurum wallets supporting multiple home addresses present a picker on first join. The user chooses which home address represents them in Kiva. Only one home address is active in Kiva at a time per Aurum instance. To switch addresses, the user leaves Kiva and re-enters under a different address; the previous address's accumulated Kiva history does not transfer to the new one. Each home address builds its own reputation independently.

Leaving Kiva is a single visible control. On leave, Aurum ceases broadcasting presence messages, withdraws all of the holder's active board offers, and removes itself from peers' hall views as their next presence-timeout pass executes. The address that left Kiva is not invisible to historical observation: on-chain trades concluded through Kiva remain on-chain, reputation already accrued remains, the fact that the address was once in Kiva is permanent. Leaving is a "now and going forward" operation, not an erasure.

Presence in the hall is equivalent to the holder's Ilmarinen node being reachable on the Scriptum P2P network. While the holder remains in Kiva, peers see them as present as long as their node is online; if the node disconnects, peers time the holder out within a defined window and re-discover them automatically when the node returns. Presence is a live network fact, not a per-day enrollment.

The Hall — Presence

The hall is the Kiva surface that displays who is currently present.

Each holder in Kiva broadcasts a signed presence record containing the holder's home address, zero or more tags drawn from the admitted enum (at v13: Buy and Sell), a millisecond timestamp anchored to chain time, and a Dilithium3 signature over the canonical bytes. The record is small and re-broadcast on a defined cadence. Peers verify the signature, verify timestamp freshness against their chain-time view, record the holder as present, and remove the holder from the hall view if no fresh record arrives within the timeout window.

Each row in the hall shows the holder's address, their tags, an inline on-chain reputation indicator (completed-trade count, default count, time-in-system), a freshness indicator, and a context action to open direct Aurum-to-Aurum contact. The hall is sortable by tag, by reputation, by time-in-Kiva. The hall is not searchable by free text; there is no profile to search against. Filters are limited to the tag enumeration.

The Board — Ás-brú Offers

The board is the second surface of Kiva. The board carries structured, signed, time-bounded Ás-brú offers.

At v13 of the protocol, the only structured offer type the board carries is the Ás-brú offer. This is by architectural intent. The natural first-day trade in a chain with no exchange listings is fiat-to-Scriptum or Scriptum-to-fiat settled through the Ás-brú exchange rail, because there are no other listing venues at launch, no P2P-fiat path that does not involve bilateral negotiation, and no Scriptum-to-Scriptum trade useful to a counterparty who does not yet hold Scriptum. The board is, definitionally, the Ás-brú offerboard. Bilateral or otherwise-structured trades route through the hall's presence list and direct Aurum-to-Aurum contact, not through the board.

Each board offer carries the originator's home address, the trade direction (Buy or Sell from the originator's perspective), the Scriptum amount, the fiat amount and currency, the named exchange (typically from the Foundation's recommended Crowglass operational configuration; see §Exchange-Choice Architecture), an expiry timestamp constrained to at most 24 hours after origination, and the originator's Dilithium3 signature. There is no minimum lifetime; the originator may withdraw any offer at any moment by gossiping a signed withdrawal that propagates at gossip speed. Each home address may hold at most five active offers on the board simultaneously — a cap that prevents dust-spam without constraining honest two-sided pricing.

A counterparty accepts a board offer by opening direct Aurum-to-Aurum contact with the originator and signalling intent to take the offer. The originator's Aurum responds and the two wallets enter the Ás-brú trade flow specified in §Ás-brú. Accepted offers are automatically withdrawn from the board because the trade is now in progress; they are reposted only if the counterparty bails before commitment.

Handoff to Settlement

Kiva's role ends at the point two holders have found each other. From that point forward, the interaction is direct Aurum-to-Aurum, and settlement runs through one of the protocol's settlement primitives. Aurum Market handles Scriptum-to-Scriptum bilateral transfers for arrangements not requiring fiat or attestation. Horkos handles three-party attested off-chain settlement, with attestor selection at the bilateral negotiation stage between the two parties who found each other. Ás-brú handles two-party fiat-rail settlement through the exchanges the parties agree to use (per the Foundation's recommended operational configuration or otherwise); board offers are pre-structured Ás-brú trades and route directly into this primitive on acceptance.

The handoff is mechanical. Kiva's gossip surface provides a contact handshake (the two wallets exchange a session token over Kiva and then open a direct peer-to-peer connection over the Scriptum P2P network), and from the handshake onward Kiva carries no further information about the interaction. The trade — its terms, its execution, its outcome — happens between the two Aurum instances directly, without Kiva mediation.

Privacy Posture

Kiva is a public hall by intent. The architectural decision to identify holders by their home address — not by stealth address, not by ephemeral identity — is what makes the reputation surface work and what gives the hall its honesty. The cost of that decision is that everything happening in Kiva is observable to anyone with an Aurum.

This is plainly stated in Aurum's UI at every join. Holders who require privacy of the fact-of-trading do not enter Kiva. The chain's private settlement primitives — Aurum Market between known counterparties, Horkos with attestors known to both parties, Ás-brú with a counterparty obtained outside Kiva, 7D, Sealed Sender Memo, view-key delegation with the range-bounded and rotatable construction — remain fully available to holders who never join the hall. The privacy boundary in Aurum is exactly the threshold of Kiva: outside, every privacy primitive in the chain remains available; inside, the holder has chosen public visibility for the reputation benefit it provides.

Aurum does not provide an option to enter Kiva with a stealth address or under a pseudonym. Doing so would defeat the reputation surface that is Kiva's entire point. A privacy-Kiva would be a different product with different properties and is out of scope.

Why This Design

Kiva exists because peer-to-peer settlement requires peer-to-peer discovery, and the chain by itself cannot provide it. Addresses on the chain are pseudonymous bit-strings; the P2P network is a transport layer for consensus, not a directory of who wants to trade. Without a discovery primitive, peer-to-peer trade in practice collapses into "trade with people you already know" — which works for a small early community and fails as the chain grows.

The design rests on three structural choices:

Opt-in symmetric visibility. A holder is either visible to the hall and sees the hall, or invisible and blind to it. There is no observer mode and no hidden browse. The symmetry keeps the hall honest by construction: if you want to see who is here, you accept that they see you.

Home-address identity. Reputation accrues to addresses on the Scriptum chain through the settlement primitives' own machinery (completed trades, defaults). Kiva surfaces that reputation by requiring members to identify by home address rather than by ephemeral identity. Bad actors cannot refresh their identity to escape a record of defaults; good actors carry their reputation forward.

Kiva drops out at the handshake. The hall is a discovery primitive, not a marketplace. After two holders find each other, Kiva carries nothing further. The trade happens between

the two wallets directly, settles through the existing protocol primitives, and Kiva learns no more about the outcome than any other Aurum holder learns by observing the chain.

Kiva is the minimum discovery primitive consistent with Scriptum's no-custody, no-Foundation-service, peer-to-peer commitment. It provides exactly the missing piece between addresses and trade — presence and posted intent — and provides nothing else.

Price Discovery — Aurum Market

The chain's first observable Scriptum/EUR price is established at genesis through the Genesis First-Buy mechanism documented in §21.5: the 1% Genesis First-Buy pool is sold at €0.01/Scriptum to constitutional cohort and registered Genesis Sign-ups under the water-filling allocation algorithm. The first Genesis First-Buy delivery following first-buy attestation marks the chain's first observable price point. Subsequent price discovery flows from this anchor through the venues described below.

Aurum Market is a peer-to-peer trading layer built into the Aurum wallet. It operates outside the consensus protocol — the chain functions identically whether Aurum Market is used or not. It provides a direct venue for Aurum users to trade FUNK peer-to-peer without a centralised intermediary.

Phase 1 — Aurum Market

Aurum Market is the peer-to-peer trading overlay embedded in the Aurum wallet. It is not an exchange. It is not a Foundation service. Aurum wallets find each other through Kiva (see §Kiva) — the opt-in discovery hall in which holders make themselves visible by home address and post Ás-brú offers — and then negotiate trades directly between peers. Orders are expressed in Aurum and recorded on chain as standard SPEND transactions. Matching is bilateral and voluntary. Settlement of Scriptum happens on chain via the protocol's standard mechanics; settlement of the counterpart — fiat, other crypto, anything else — happens off chain in whatever form the counterparties agree on. The Foundation provides no matching, no escrow, no arbitration, no dispute resolution, and operates no servers required for Aurum Market to function. Turning off every Foundation asset would have no effect on Aurum Market's operation.

Every trade produces a signed record. Both parties sign their trade intent with their Dilithium3 keys before settlement. Completed trades and any defaults are permanent public records against the counterparty addresses — a reputation layer emerges organically from the chain's own history.

Counterparty selection and counterparty performance are entirely the users' responsibility. The protocol cannot reverse trades. The Foundation provides no recourse. Aurum Market is a peer-to-peer direct trading tool for holders who understand what peer-to-peer means.

Price discovery at scale happens through the community exchange listings described in §38.2 and the global exchange phase described in §38.3. Aurum Market provides the initial bootstrap venue for early adopters trading directly with one another.

Phase 2 — Community Exchange Listing

In parallel with the foundation exchange, Scriptum will seek listing on NonKYC.io — a proof-of-work focused exchange that lists new chains at minimal cost. This provides a second independent price feed and introduces Scriptum to the existing PoW mining community.

Phase 3 — Global Exchange Listing

Once Scriptum has established price history, community, and trading volume through Phases 1 and 2, the Foundation pursues listings on internationally credible exchanges to provide global access and broaden price discovery. The Foundation's listing strategy follows a tier discipline rather than a fixed venue plan.

The Foundation will pursue listings on exchanges that meet at least the following criteria: a credible regulatory perimeter (MiCA-licensed, MiCA-applied-for, or operating under equivalent jurisdictional authorisation), demonstrable security and proof-of-reserves practice, transparent corporate structure, and editorial standards on listings (i.e., the venue does not list arbitrary tokens for fees alone). Examples of exchanges that meet or are progressing toward these criteria as of writing include OKX, Bybit, Bitget, and KuCoin in the international tier, alongside the regulated incumbents (Coinbase, Kraken, Bitstamp) for the eventual institutional tier. The Foundation engages opportunistically rather than exclusively; multiple listings on the same tier are expected and welcomed, and listing decisions are not commitments to any single venue.

The Foundation will *not* pursue listings on venues whose primary business model is paid-for listings without editorial standards, or whose corporate structure has been the subject of regulatory disclaimer in their nominal jurisdiction. Naming such venues affirmatively in this whitepaper would itself signal the wrong tier; the Custody Reality companion document discusses concrete examples of unauthorised offshore platforms in the context of explaining custodial fragility, and that discussion is the appropriate place for those names.

No exchange fees are payable at launch. Price discovery is fully decentralised in Phase 1 (P2P inside Aurum), moves to community exchanges in Phase 2, and reaches global credible venues in Phase 3. The bootstrap path imposes no cost on the Foundation.

Operational Modes and Practical Use

Scriptum is not a single-use system. It is a structured environment in which value can be held, moved, or conditionally transferred under explicitly defined security regimes. Each mode represents a deliberate trade-off between immediacy and protection.

Secure Storage — The Default

All value in Scriptum is created and held in the LOCKED state. This is not an opt-in security mode. It is the default state of every Parcel, from the moment it enters existence. Mined coins are LOCKED. Received funds are LOCKED. Change outputs are LOCKED.

Movement of value is not instantaneous. Every transfer must pass through the Safety Control Window. Unauthorized transactions remain observable and cancellable for the full duration of the window.

Possession of a key does not imply immediate control over value. This is the core property that distinguishes Scriptum from every other chain.

This mode serves long-term capital storage, treasury and institutional holdings, and any situation where protection against key compromise matters more than transaction speed. It is also, incidentally, the only mode the protocol requires. Everything else is a user choice to reduce protection in exchange for immediacy.

Immediate Use — The ACTIVE State

Users may maintain a portion of funds in the ACTIVE state. ACTIVE funds can be transferred without passing through a new Safety Control Window. The mode exists for routine or low-value transactions — payments, small transfers, anything analogous to cash in a wallet.

ACTIVE is a deliberate reduction in protection. Funds in ACTIVE state are not covered by the cancellation window that protects LOCKED and NEXUS-dwelling Parcels. A user who keeps large value in ACTIVE state is making an explicit security trade-off.

ACTIVE Parcels automatically return to LOCKED after the ACTIVE window closes. Exposure is bounded by design — even a user who forgets a balance in ACTIVE state is returned to full protection within a day.

Scriptum separates liquidity from security by design. ACTIVE exists because users need it; its cost is an explicit, bounded, temporary reduction in the protocol's default protection.

Market Interaction and Liquidity

Scriptum interfaces with external markets for price discovery, liquidity provision, and entry or exit. Platforms such as Binance, Kraken, and others may facilitate trading. These venues provide efficient execution and global access that no on-chain settlement layer can replicate.

Exchanges operate outside the Scriptum settlement model. They introduce custodial and counterparty risk. A user's FUNK on an exchange is the exchange's obligation to deliver, not a Parcel under the user's key.

Scriptum does not attempt to replace exchanges. It provides a fundamentally different settlement layer beneath them. Users who want liquid continuous trading use exchanges. Users who want cryptographic settlement without custody use the chain directly.

System Coherence

These modes form a unified architecture. LOCKED provides maximum protection. ACTIVE provides immediate usability. The market layer provides liquidity and price discovery. Horkos provides conditional, trust-minimized settlement.

Each function is explicitly separated and cannot be conflated. A user's LOCKED balance is not mixed into the ACTIVE pool. Exchange holdings are not protocol state. Horkos escrow is not a governance instrument. Each mode does one thing clearly, with consequences the protocol makes visible.

Operational Principle

Scriptum enforces intentionality. Small value moves quickly because the user has chosen to accept speed over protection. Large value moves slowly because the user has chosen protection over speed. The protocol does not assume user intent. It requires it.

This is not paternalism. It is structure. Users who want speed can have speed — in the ACTIVE state, in measured amounts, with bounded exposure. Users who want security can have security — in the LOCKED state, with a cancellation window, with full visibility of any movement attempt.

Security is not a parameter. It is the default state.

PART IV — INSTITUTION

Governance — WOC, Founders, and Curia

The Scriptum governance model is built on three constitutional roles: the Warden of the Chain (WOC), the Founders, and the Curia. These roles carry TREASURE allocations with defined vesting schedules and are instantiated at Genesis via ARC 0. Full governance procedures are specified in the Scriptum Governance Charter; this section sets out the protocol-level commitments that the Charter operates within.

Constitutional Roles

WOC — 1 member. Warden of the Chain. Protocol guardian. Last resort. Permanent signature authority for chain protocol releases. Singular constitutional position.

Founders — 5 members. The architectural cohort. Operational executive authority. Distinct from the WOC.

Curia — 9 members. The cryptographic multi-sig governance body. Distinct from the WOC and from the Founders.

Total — 15 distinct constitutional position-holders. The cohorts do not overlap; no person holds positions in more than one cohort. The notation “1 / 5 / 9” refers to these three cohort sizes, summing to fifteen unique constitutional positions.

Decision Tiers — Four-Level Multi-Sig

All governance decisions are classified into four levels. The level determines the cohort participation and signature threshold required.

Level 1 — Curia Operational

Threshold: 5-of-9 Curia signatures.

Scope: routine Funds disbursements within budget, regular WFC compensation distributions, Charity Pot group-decision allocations, day-to-day operational decisions.

Founders and WOC do not participate in Level 1 decisions.

Level 2 — Founders Strategic

Threshold: 3-of-5 Founders signatures.

Scope: strategic operational decisions — public communications strategy, partnership commitments, executive hiring, product direction, Foundation board nominations.

Curia and WOC do not participate in Level 2 decisions.

Level 3 — Cross-Cohort Significant

Threshold: 3-of-5 Founders AND 5-of-9 Curia signatures (both gates required).

Scope: Charter amendments, large unbudgeted spending above the thresholds defined in Schedule A of the Governance Charter, membership changes (succession, removal-for-cause), parameter changes within protocol bounds.

The WOC does not participate unless the matter has Constitutional weight, in which case the decision proceeds to Level 4.

Level 4 — Constitutional

Threshold: WOC + 3-of-5 Founders + 7-of-9 Curia signatures (all three gates required).

Scope: Constitutional amendments, hard-fork protocol changes, fundamental architectural changes including changes to the Tithe rate (90/10), changes to the Tithe split (48/32/10/10), changes to the existence of the four streams (WFC, Funds, RESERVE, CP), revisions to the cap-overflow destination rule, and **RESERVE access** (the chain's strategic long-term reserve, deployable only in genuinely exceptional circumstances — catastrophic chain emergency, civilisational-scale uncertainty — and not for ordinary operation).

Emergency Operations

Threshold: WOC emergency declaration + 5-of-9 Curia signatures.

Scope: time-critical matters where a Level 3 process would be too slow.

WOC acting alone is time-limited to 72 hours. After 72 hours the decision must be ratified by retroactive Level 3 (3-of-5 Founders AND 5-of-9 Curia) within 30 days of the original action. If ratification fails, the WOC action is automatically reversed by protocol where reversibility is technically feasible.

The WOC acting alone cannot: change the protocol consensus rules, change $S_{\max}$, change RAGNAROK_HEIGHT, change the Tithe split, alter compensation caps, or remove other constitutional position-holders. These are Constitutional matters and require the full Level 4 process.

Decision Classification

Routine → Level 1 (Curia 5-of-9): routine Funds disbursements, regular compensation distributions, Charity Pot group allocations.

Strategic → Level 2 (Founders 3-of-5): partnerships, executive hiring, product direction, public communications.

Significant → Level 3 (Founders 3-of-5 AND Curia 5-of-9): Charter amendments, large unbudgeted spending, membership changes, parameter changes within protocol bounds.

Constitutional → Level 4 (WOC + Founders 3-of-5 + Curia 7-of-9): protocol fork, WOC succession, Constitutional amendments, changes to the Tithe architecture, RESERVE access.

No-Quorum Rules

If signature thresholds cannot be reached, status quo prevails. The action does not execute. For Level 1 and Level 2 matters the vote may be called again at the next opportunity. For Level 3 matters the WOC may invoke emergency authority subject to the Emergency Operations rule above. For Level 4 Constitutional matters inertia is always correct — if 7 of 9 Curia and 3 of 5 Founders cannot agree alongside the WOC, the protocol does not change.

WOC Emergency Authority

The WOC holds emergency authority as a last resort. This activates when: (1) the chain faces an active threat and neither Curia nor Founders can act in time, or (2) both cohorts have failed to reach Level 3 thresholds and a decision must be made within hours rather than days.

WOC acting alone is time-limited to 72 hours. After 72 hours the decision must be ratified by retroactive Level 3 within 30 days. If ratification fails, the WOC action is automatically reversed by protocol where technically feasible.

The WOC acting alone cannot perform Constitutional changes. These remain Level 4 matters and require the full process regardless of urgency.

WOC Succession

The Scriptum Governance Charter — a separate document published with the protocol — specifies WOC succession, Founders cohort and Curia composition details, dispute resolution, and emergency procedures in operational detail. This whitepaper commits the protocol layer; the Charter commits the governance layer that operates within the protocol's hard caps. Requirement: the Charter MUST provide a delegated-backup protocol so that short-term WOC unavailability (days, not weeks) cannot block governance operations. Specific mechanism is deferred to the Charter.

Fountain Governance

The Fountain (Genesis Liquidity Reserve) is controlled as described below.

Auto-closure at block 864,000 → protocol-driven, no vote required

All Fountain votes are recorded on-chain as Dilithium3-signed transactions. Every vote is permanent, public, and auditable.

The Upgrade Ceremony

Every protocol change of consensus-altering scope is, technically, a hard fork. Nodes running pre-amendment software cannot validate post-amendment blocks. The architectural innovation Scriptum brings is not that it avoids hard forks; it is that it institutionalises them. What in other chains is an ad-hoc social-coordination event with uncertain outcomes — informal negotiation among developers, miners, exchanges, and users; deadline-pressure activation mechanisms; meaningful risk of permanent chain split

— Scriptum performs as a structured public ceremony with defined gates, mandatory consultation, synchronised deployment, and explicit user exit rights.

The chain calls this a cut and splice. The rules cut at a defined boundary; the chain splices forward under the new rules without branching into parallel old and new chains continuing in parallel. The technical hard fork remains a hard fork at the consensus level — old software cannot validate new blocks. The ceremony is the institutional infrastructure that transforms it from a disruptive event into a routine procedure.

Cut and Splice, Not Branching

When Bitcoin or Ethereum hard-fork, the chain potentially splits into two chains. The new chain continues under the new rules; the old chain continues under the old rules. Both demand maintenance, both demand validation, both demand exchange listings, both demand wallet support. Communities fragment. CASPs face dual-integration requirements. Users hold balances on both sides until they decide which to abandon. The 2017 split that produced Bitcoin Cash and the 2016 split that produced Ethereum Classic illustrate the pattern: technically clean forks that produced two ongoing chains where one was intended.

Scriptum's Menuha-windowed upgrade produces no parallel old chain. The constitutional cohort declares which chain is canonical at Level 4 ratification. The Foundation publishes the new Aurum binary signed by the WOC's release key. Foundation-aligned infrastructure deploys the new software. Aurum installations auto-update during the Menuha window. CASPs integrated with the Foundation switch their integrations to the new chain. Miners follow the rewards. Operators who fail to upgrade find their nodes producing blocks that the upgraded network rejects; their hashpower is wasted; they upgrade or stop mining. The old-rule chain technically exists for as long as anyone runs it, but it has no constitutional endorsement, no Foundation infrastructure, no economic infrastructure, no user base. It is a chain in the sense that it is a sequence of cryptographically-bound blocks. It is not a chain in the sense that anyone uses it.

This property is structural rather than algorithmic. The technology cannot prevent operators from running old software. The institutions make the alternative chain economically inert. The cut-and-splice claim is a commitment that the Foundation, the constitutional cohort, and the institutional infrastructure will hold the canonical chain coherent through change. The technology is the substrate; the institutions are the chain.

The Ten Gates

The ceremony proceeds through a sequence of discrete gates, each of which is a multi-sig event recorded on-chain and attributable to its signers. The full procedural treatment is documented in the Governance Charter §8.2 and in the standalone Upgrade Ceremony exposition. The Whitepaper records the gates as a non-amendable procedural framework.

Gate 1 — Initiation. Level 4 multi-sig (WOC + 3-of-5 Founders + 7-of-9 Curia) signs to open the upgrade process. No specific change is yet proposed; the cohort declares it is entering deliberation.

Gate 2 — Specification input. The proposed change is documented in full and published on-chain in a structured proposal format.

Gate 3 — Specification lock. Level 4 multi-sig signs to lock the specification. After this signing the proposal is fixed; no edits or addendums are possible.

Gate 4 — Public consultation opens. Every Aurum wallet, every Sanctum archive, every connected node receives notification. The full specification is mirrored across Foundation public channels and published on the Foundation website. Constitutional position-holders are publicly accountable for the proposal they have just locked.

Gate 5 — Consultation window. A bounded period during which the community reads, debates, raises objections, and proposes amendments. Duration is set per proposal in the specification, between one and four Menuha cycles depending on consequentiality. The window is finite; ceremony stalls do not extend deliberation indefinitely.

Gate 6 — Amendment decision. Level 4 multi-sig signs to either accept consultation feedback (re-entering the ceremony from Gate 2) or confirm no changes are needed (proceeding to deployment). The cohort's reasoning is documented and published.

Gate 7 — Deployment lock. If no amendments, Level 4 multi-sig signs to commit the implementation. The new node software is published. The deployment timer begins, anchored to a specific future Menuha boundary.

Gate 8 — Final notice. Every wallet receives a second notification: protocol change confirmed, deployment scheduled. This is the last opportunity for users to exit before the change becomes binding.

Gate 9 — User exit window. Users have time before the upgrade fires to make their own decisions: continue and let their wallet auto-update, prepare to upgrade manually, sell holdings if they object to the change, withdraw to fiat through a CASP, or exit to a different chain entirely. This is the chain's Safety Control Window applied to its own changes — a window during which informed individual exit is genuinely possible.

Gate 10 — Fire at Menuha boundary. The change deploys at the start of the designated Menuha. The chain continues with new rules. Nodes that have not upgraded fall out of consensus until they update. Operations during the Menuha window run under the chain's normal Menuha rules (user-initiated outward transactions prohibited, protocol housekeeping continues); the splice fires at the end of Menuha, and Monday operations resume under the new rules.

Procedural Commitments

In ratifying the architecture documented in this Whitepaper and in the Governance Charter, the constitutional cohort makes four explicit procedural commitments about how Level 4 protocol authority will be exercised.

First, the ceremony will not be skipped or accelerated for ordinary protocol changes. Genuine emergencies — critical bugs, active exploits — may invoke the WOC's emergency

authority with retroactive Level 3 ratification per the Charter; the ceremony cannot be bypassed by re-classifying a substantive amendment as an emergency.

Second, public consultation periods are real periods of community input, not ceremonial pauses. Feedback received during Gate 5 must be considered at Gate 6, and the cohort's reasoning published.

Third, user notification through Aurum, Foundation channels, and dfqr.org is a procedural requirement, not a courtesy. Failure to notify users at Gates 4 and 8 constitutes a procedural breach of the ceremony.

Fourth, the user exit window between Gate 8 and Gate 10 is genuine. The deployment timing is fixed at Gate 7 and not accelerated thereafter; users who decide to exit have the full notified period to act.

Welcomed Forkability

Apache 2.0 license on all source. Anyone may fork the code and launch a derivative chain. The Foundation does not consider this a threat. What makes Scriptum what it is — the constitutional governance, the Foundation, the eight-year design history, the accumulated chain state, the people holding the fifteen constitutional positions, the trademark, the institutional trust — none of these travel with a code copy.

Forkability is what makes the architecture honest. There are no hidden mechanisms, no admin keys, no privileged paths that the open code does not reveal. Anyone with the technical competence may inspect the chain in full and may, if dissatisfied, build a derivative under different governance. This constrains the Foundation through the credible exit threat: governance that ceases to serve the chain's purpose may find its constituency departing for a fork that does. The strongest version of any tradition is the version that survives by being chosen freely, not by being imposed.

The chain's value is in what it is, not what it is written in. Scriptum welcomes scrutiny, derivation, and the existence of derivatives. The institutional commitments documented in this Whitepaper and in the Charter are what make the canonical chain canonical. The code is what makes the chain technically possible; the institutions are what make it institutionally durable.

Limits of Procedural Protection

The ceremony provides procedural protection against accidental or hasty constitutional changes. It does not provide architectural protection against deliberate constitutional misuse. Distributed-systems specification, in the tradition of Lamport's work on consensus protocols and the formal-methods literature more generally, distinguishes between behaviour the system guarantees within its operating assumptions and behaviour outside those assumptions. A consensus protocol that tolerates f Byzantine actors out of $3f+1$ specifies precisely what happens up to that threshold; it does not specify what happens beyond, because beyond the threshold the system is no longer the system it was designed to be.

The same applies to Scriptum's constitutional architecture. The protocol and the ceremony specify what happens when the constitutional cohort acts within model — exercising authority through the procedures and at the thresholds the architecture provides. They do not specify what happens when the cohort acts outside model: through coordinated bad-faith ratification of breaking changes, through institutional collapse, or through other fundamental failures of the governance assumptions the protocol relies on. The defence against constitutional failure is institutional — the ceremony, the consultation, the public attribution of every signature, and the open-source forkability of the chain itself — not architectural. This is a property the Whitepaper acknowledges honestly rather than overclaiming protection it cannot provide.

The Tithe — Protocol Revenue

Scriptum applies a protocol tithe of 10% on all f_S (transfer fees). This is not a tax. It is a contribution to the common infrastructure that makes the chain function across a century. Every civilisation that has endured has found a mechanism to fund its commons. The Tithe is Scriptum's mechanism.

The Split — Stage One

Every fee-bearing transaction splits at the protocol level:

90% → miner (block-production reward to the miner including the transaction)

10% → Tithe (the chain's protocol stream)

The 90/10 split applies uniformly to f_U , f_A , f_S , and F_{REFRESH} . There is no carve-out for flat instruction fees: every euro of fee revenue feeds the same two streams in the same proportion.

The 90% miner share preserves competitive proof-of-work economics across the chain's century-scale lifetime. Bitcoin, Monero, Litecoin, and other major proof-of-work chains pay miners 100% of fees; Scriptum's 90% is calibrated to remain attractive to mining hashrate while creating a meaningful 10% protocol stream. Reducing the miner share below 85% would risk hashrate migration to competitor chains and weaken the chain's security budget. The 90/10 split is structurally bounded by the practical economics of competitive proof-of-work and is a Constitutional invariant subject to a structural floor of 85%.

The Tithe Allocation — Stage Two

The 10% Tithe is allocated across four streams in fixed proportions. The split is flat: the same percentages apply at every chain volume, from cold-start to maturity.

WFC — 48% of Tithe. The Wizard / Founders / Curia compensation pool. Distributed across the constitutional position-holders by per-person share-units defined in the Governance Charter. Subject to HICP-indexed personal compensation caps.

Funds — 32% of Tithe. Foundation operational allocation. Funds the Stiftung's running costs: legal, audit, FINMA reporting, hosting, security retainers, professional services, infrastructure.

RESERVE — 10% of Tithe. Strategic long-term reserve, accumulated against exceptional circumstances. Released only by Constitutional governance decision (full process per the Governance Charter).

CP — 10% of Tithe. The Charity Pot — the chain's structural humanitarian allocation. Receives the flat 10% share of every Tithe euro at every chain volume, and additionally receives all cap-overflow from position-holder compensation per the cap-overflow rule below.

The four-stream split is a Constitutional invariant. The percentages within the split (48/32/10/10) and the existence of the four streams may be revised through Constitutional amendment; the existence of CP as a structural component of the Tithe is invariant.

Cap-Overflow Rule

Any WFC revenue that would exceed any individual position-holder's HICP-indexed annual compensation cap routes 100% to the Charity Pot. Single rule, single destination, no cascade. There is no intermediate stop at Foundation operations (Funds has its own dedicated 32% Tithe slice and does not receive cap-overflow), and no residual flow to RESERVE. Each position-holder's cap binds individually. The destination — Charity Pot — is a Constitutional invariant.

What Funds Pays For

Storage and hosting → Sanctum infrastructure

Security audit → commissioned from Funds

Bug bounty programme → funded from Funds

Website and maintenance → dfqr.org operations

Legal and regulatory → jurisdiction compliance

Secretariat → governance administration

What the RESERVE Holds

The RESERVE is the chain's savings account. It accumulates from the 10% Tithe slice automatically with every SPEND. It cannot be accessed without Constitutional governance — the highest authority threshold defined in the Governance Charter (WOC + 3-of-5 Founders + 7-of-9 Curia). It exists for genuine emergencies only — protocol-level threats that cannot be addressed through normal operations.

What the Charity Pot Holds

The Charity Pot accumulates from the 10% Tithe slice plus all WFC cap-overflow. Internal allocation is split: 42% individually directed (each position-holder personally allocates a defined fraction per the Governance Charter), and 58% group-decided via Curia 5-of-9 multi-sig. All disbursements are publicly disclosed on-chain. The Charity Pot is the chain's structural commitment that prosperity beyond constitutional architecture flows to humanitarian purpose.

Tithe at Genesis

At genesis the chain is nearly empty. The Tithe generates almost nothing. Genesis allocations and the Fountain mechanism (per §22) provide bridge liquidity for the chain's cold-start period. As adoption grows, the Tithe becomes the primary operational funding source. Genesis allocations are the seed. The Tithe is the harvest.

The Tithe rate (10% of f_S) is a protocol constant. It cannot be changed by Curia vote. It cannot be changed by Founders vote. It requires a Constitutional amendment — the highest governance threshold — to modify. It is designed to be permanent.

The Tithe is a protocol-level allocation, not a service fee. Miners pay no one. The protocol routes the Tithe into the WFC, Funds, RESERVE, and CP vaults, which are on-chain objects controlled through the four-level multi-sig structure defined in the Governance Charter. Every disbursement is an on-chain transaction, publicly visible, cryptographically authorised. The Foundation is a recipient of Curia-authorised disbursements from Funds; it is not a counterparty to any fee-paying user and provides no services to those whose fees feed the Tithe.

Fountain — Genesis Liquidity Provisioning

The protocol does not rely on pre-allocated liquidity or privileged issuance. However, during early network formation, the Fountain — a temporary and rule-bound mechanism — ensures functional market formation and price discovery without structural deadlock.

The Fountain is the concrete implementation of Genesis Liquidity Provisioning (GLP) during the initial network phase. The Fountain is instantiated at genesis and cannot be recreated.

Allocation

Name: Fountain (Genesis Liquidity Reserve — GLR)

Source: RESERVE. The Fountain is a reserved operational tranche within RESERVE, not an additional genesis allocation. Genesis allocations remain unchanged at 16% of S_{MAX} (5% TREASURE + 5% FUND + 5% RESERVE + 1% Genesis First-Buy).

Allocation: 0.25% of S_{MAX} = 105,000 FUNK. Released over the Fountain window from this tranche within RESERVE.

Duration: 864,000 blocks (approximately 100 days post-Genesis).

Reversion: any unused portion of the Fountain tranche at end of window remains in RESERVE under standard L4 lock; the Fountain does not “expire” funds, it simply closes the operational window.

Price

Fixed: €0.01 per FUNK throughout the Fountain window. Matches the Genesis First-Buy price. The chain’s price discovery is organic — the Fountain price does not adjust to market conditions because Fountain operates on bilateral covenants, not market-making.

Mechanism — Horkos covenants via Aurum

Each Fountain sale is a Horkos covenant (per WP §Horkos / CM-7). The buyer initiates; the Treuhänder is Party A (seller); the Treuhänder or WOC serves as the named attestor per covenant. The off-chain fiat leg is a wire transfer to the Treuhänder’s Anderkonto (client escrow account).

Aurum’s Beginnung (the onboarding section of the wallet) hosts a dedicated Fountain page. The page shows current Fountain status, the fixed price (€0.01/FUNK), the Anderkonto wire instructions, and an “Initiate buy” button that constructs a HORKOS_AGREE with the Treuhänder’s published Fountain address pre-filled as Party A. The buyer enters the FUNK amount, signs the covenant, wires fiat to the Anderkonto, and waits for HORKOS_ATTEST. Standard Horkos lifecycle thereafter.

Daily Fountain release rate is managed off-chain by the Treuhänder (co-signed by WOC via CM-3.14 BOOTSTRAP authority): each day, the Treuhänder authorises a Fountain Parcel covering that day’s expected demand. Buyers initiate Horkos covenants against the current standing Parcel. When the daily tranche depletes, Aurum shows “Fountain depleted today — resumes at next block-day boundary.” Annual operational pacing is documented in the Foundation’s eventual annual report; during the Bootstrap Period (Genesis through Curia 5-of-9 seating), pacing is at the discretion of WOC + Treuhänder.

Eligibility — public

The Fountain is open to any FUNK-holding address. There is no registration requirement, no whitelist, no KYC at chain level. KYC and AML compliance occur at the Treuhänder’s Anderkonto when fiat arrives — the Treuhänder’s professional fiduciary obligations under Swiss law handle the regulatory side. Buyers whose wire transfer is rejected by the Anderkonto for KYC/AML reasons cannot complete the Fountain Horkos covenant; the covenant CANCEL or attestor refund proceeds per standard CM-7 rules.

Authority

Fountain Parcel releases (the OPEN_VAULT transactions that move FUNK from the RESERVE-held Fountain tranche into the Treuhänder’s seller-side address for Horkos commitment) require multi-sig signing per CM-3.14.4. During the Fountain window

(Genesis to block 864,000), which is entirely within the Bootstrap Period, the BOOTSTRAP authority applies: WOC + Treuhänder co-signature. This is a programmatic exception to the general rule that RESERVE access requires L4 — deliberately scoped to the Fountain window only.

After block 864,000, the Fountain window closes. Any further RESERVE access requires standard L4 procedure per CM-3.14.4 and GOV §5.4. The remaining Fountain tranche balance is no longer specially designated; it is simply part of the RESERVE general balance going forward.

Fiat proceeds

All Fountain fiat proceeds flow into the Treuhänder's Anderkonto under standard professional client-account discipline. The proceeds are held in escrow until the Foundation incorporates (Bootstrap Period transition T1, target 6 to 12 months post-Genesis), at which point the Treuhänder transfers accumulated Fountain proceeds to the Foundation operating account. This mirrors the existing Genesis First-Buy fiat-proceeds handling per GOV §13.

Closure

At block 864,000, the Fountain window closes automatically. This is operational, not consensus — the CM-3.14 BOOTSTRAP authority for RESERVE OPEN_VAULT specifically scoped to Fountain ends when block 864,000 is reached. The chain enforces this through the BOOTSTRAP authority's time-window check (per CM-3.14.5). Any in-flight Horkos covenants initiated before block 864,000 continue through their normal lifecycle to completion or CANCEL; no new Fountain Horkos covenants can be initiated after block 864,000.

The Fountain is a chain-mechanism with a clear start, a clear pricing discipline, a clear end, and a clear architecture that does not depend on any institution that does not yet exist at Genesis. It is the chain's cold-start mechanism: bounded, public, mechanically simple, and operationally executable from the WOC and Treuhänder alone.

Exchange-Choice Architecture

Ás-brú trades reference an exchange identifier in their commit fingerprint. The architecture surrounding exchange choice — who decides which exchanges are appropriate, who admits or rejects, how the recommendation is published and maintained — is open at protocol level. There is no chain-tracked admitted-exchange registry; there is no Curia governance vote over which exchanges qualify; covenants referencing arbitrary exchanges are not rejected at consensus.

This section specifies why the architecture is open, what supporting machinery the Foundation provides, and where the trust actually lives.

Open at protocol level. Ás-brú covenants reference an exchange-identifier hash. The protocol validates that the covenant is well-formed and that the parties signed; the

protocol does not validate that the exchange exists, is reputable, or is approved. Miners running Crowglass observe the exchange APIs they choose to observe. The chain's consensus is over what miners actually included in their blocks, not over whether the chain approves of their exchange choices.

Why open. A protocol-level admitted-exchange gate has two failure modes, both worse than the open architecture:

- If the gate is a Foundation-controlled key (signing admission decisions), it provides no real protection because Aurum is Apache 2.0 open-source — the key would be extractable from the source by any sufficiently motivated reader. Honest users would trust the protection; adversaries would defeat it in minutes.
- If the gate is a chain-tracked registry curated by L3 governance, it provides real protection but recentralises the chain — Foundation governance becomes a gatekeeper for commercial activity, contradicting the open-protocol commitment and creating a single point of capture or coercion.

Neither outcome aligns with Scriptum's design commitments. The chain chose open-protocol architecture at Apache 2.0 licensing; this section reflects that choice consistently.

Foundation recommendation, informational only. The Foundation publishes a recommended Crowglass operational configuration — a list of exchanges the Foundation considers operationally appropriate for Crowglass observers, based on regulatory compliance, audited financial standing, asset segregation, public API stability, and operational maturity. The recommendation is published, maintained, and revised at Foundation-board cadence per GOV §11.3 and §12.1. The publication is informational; it has no protocol-binding force.

Genesis ships with **Kraken** as the initial recommendation (MiCA-compliant, established, regulated, audited). The Foundation may add to the recommended set as operational experience accrues. Removal from the recommended set follows the same Foundation-board procedure; Foundation removal of a recommendation does not retroactively invalidate trades that referenced the exchange before the recommendation changed.

Aurum surfaces recommendations as defaults. Aurum's trade-construction UI for Ás-brú offers the Foundation's recommended set as defaults for user convenience. A user who knows what they are doing may override the default and name a different exchange; a user operating a non-Aurum wallet may do the same. The choice is the parties'; the protocol enforces what they agreed.

Where the trust lives. With protocol-level curation removed, trust in the exchange choice is the parties' own responsibility. The chain provides reputational signals — Kiva surfaces counterparty trade history; Aurum's recommendation reflects Foundation operational judgement — but does not enforce them. Parties who choose unreliable exchanges or who skip the Foundation's recommended set accept the operational risk that comes with their choice. The chain's view is that adults can make their own choices about commerce; the chain's role is to enforce what they agreed, not to curate what they should have agreed to.

The exchange identifier's specific encoding (hash format, fingerprint construction) is specified in CM-14 and SPX-16. The Foundation's recommendation publication is administered per GOV §12.

Protocol Hard Caps — Curia Capture Defence

The Fountain release schedule is subject to two hard caps enforced at the protocol level. These caps cannot be overridden by any governance vote — including a unanimous Curia vote. They are protocol rules, not governance rules.

Daily cap: MAX release $\leq 2\%$ of starting Fountain balance

per 8,640-block period (~24 hours)

At genesis: $105,000 \times 2\% = 2,100$ Scriptum maximum per day

10 day cap: MAX release $\leq 20\%$ of original Fountain allocation

per 86,400 block period (~10 days)

= 21,000 Scriptum per 10-day epoch, hard ceiling

Any transaction that exceeds either cap is rejected by all nodes. No governance decision can authorise a violation, the protocol rejects the transaction.

This eliminates the over-release attack vector at consensus level. Genesis liquidity is strictly subdominant to organic issuance.

What Fountain is Not

Fountain is not a market maker. It is not a price support mechanism. It is not a permanent liquidity provider. It is seed capital for price discovery — small enough to be honest, large enough to prevent deadlock. At €1 per Scriptum, 420,000 Scriptum represents €420,000. At any price, the SCW ensures it cannot be deployed instantly. It drips. That is the design.

The Donation Mechanism — Dana

Dana is the Sanskrit and Pali word for the act of giving — the virtue of generosity itself, recognised across Buddhist, Hindu, and Jain traditions. In Scriptum, Dana names both the donation mechanism and the token of recognition it produces.

Any holder may send Scriptum to the protocol-defined destination Chesed at any time. Chesed is a semantic label only — not a standalone vault. On confirmation, the donated amount is allocated to the Funds and RESERVE VAULTs at a 50/50 ratio as new SEALED VAULT Parcels.

On confirmation of a DONATE transaction, the protocol issues a Dana Parcel to the sending address:

CLASS = MARKER

ROLE = DANA

STATE = PERMA_LOCKED

ADDRESS = donor address

AMOUNT = 1 Nugget (display-only, non-economic)

The Dana is private by default. Only the holder knows it exists. They may choose to share it — or not. It is their record, their choice. The amount donated is not recorded in the marker. Only the act.

Like the Mayflower Parcel, the Dana is permanent and non-economic. It survives every Ragnarök unchanged. Every successor Phoenix carries the record of every holder who gave. The chain remembers.

Dana Parcels cannot be created, transferred, or faked. The protocol issues exactly one per confirmed DONATE transaction, at the sending address. No donation — no Dana.

Aurum Implementation

Aurum includes a dedicated DONATE screen. The holder enters an amount, reviews the destination (Chesed), confirms, and submits. Chesed is a semantic label only — not a standalone address. The donated amount is allocated as new Funds and RESERVE VAULT Parcels (SEALED state) at a ratio of 50/50.

On confirmation of the DONATE transaction, the Dana appears in the wallet quietly — no fanfare, no public announcement. The holder decides what to do with that knowledge.

The DONATE action is separate from all other transaction types. It cannot be triggered accidentally.

Other wallets may support this or not. Aurum defines the canonical experience. The mechanic is consensus-layer — any conforming wallet can implement DONATE and issue Dana.

Supply Accounting

Donated Scriptum is allocated to the Funds and RESERVE VAULTs at a 50/50 ratio as SEALED Parcels, and is excluded from free circulation under standard VAULT rules. Dana Parcels (1 Nugget each) are excluded from all supply calculations — not in S_{mined} , S_{net} , or S_{max} . Protocol overhead only, identical in accounting treatment to Mayflower Parcels.

Bastet — The Guardian of Treasure

In Scriptum, Treasure is not merely locked. It is guarded. That distinction matters.

A vault can be sealed and opened. A lock can be released by time or authority. But Treasure is something else entirely: it is held under watch.

Scriptum separates three distinct guardianship models:

VAULT → protocol opens it. Mechanical release.

→ SEALED → OPEN. No guardian needed.

TREASURE → Bastet guards it. Vesting required.

→ TIME_GUARDED → VESTING → RELEASE → LOCKED.

→ Bastet steps aside at each vesting milestone.

MAYFLOWER → nothing leaves. Ever.

→ PERMA_LOCKED. No guardian needed.

→ There is no door.

Bastet is specifically and only the guardian of TREASURE. She is the presence that ensures departure happens on the right terms, at the right time, to the right address.

We choose Bastet — from the Egyptian tradition — because she embodies dual nature: still until movement is required, calm until force is justified. She does not open the vault. She ensures it does not need to be forced open.

Her states:

SEALED — Bastet is present. Nothing moves.

VESTING — Bastet is watching. The schedule unfolds.

RELEASE — Bastet steps aside. Each tranche is freed.

LOCKED — it is simply a coin now.

This concept lives at Level 3 — narrative. It does not affect consensus, state transitions, or protocol behaviour. It clarifies meaning without breaking rules.

Treasure is not hidden. It is protected. And in Scriptum, that protection has a name: Bastet.

PART V — GENESIS

Rebirth and the Founding Manifest

Genesis is a descriptive term referring to the first Rebirth event. It has no direct representation in consensus rules. All protocol behaviour is defined through Rebirth, block validation, and state instantiation.

The Scriptum protocol begins — and renews — through Rebirth. Rebirth is the instantiation of a new Phoenix at block height 0.

Block 0 — The Rebirth Block

Block height = 0 in every Phoenix

Exempt from standard block size limits

Coinbase reward = 0

Transaction fees = 0

Contains all DISEMBARK outputs derived from the ARC manifest

Contains all Mayflower Parcel re-instantiations

Contains all Constitutional and VAULT allocations

Standard block rules apply from height ≥ 1

The new Phoenix carries no history from the originating chain. The originating chain is frozen at RAGNAROK_HEIGHT, complete and immutable, left behind. The successor Phoenix starts at block height 0. Block 0 is the entire starting state of the new chain.

Block 0 — Deterministic Construction

Block 0 of the successor Phoenix has no miner. It requires no mining reward, no proof-of-work, and no trusted party to produce it.

Block 0 is fully deterministic from the ARC manifest. Any node that holds the ARC manifest from the originating Phoenix can independently construct the identical Block 0. All nodes verify it against the same manifest. Consensus is achieved not through mining but through deterministic agreement on a known input.

No discretionary producer is trusted. Validity is determined entirely by deterministic reconstruction from the ARC manifest under canonical rules. No node gains an advantage by producing Block 0 first.

This is the only block in Scriptum that is not mined. Every subsequent block from height 1 follows standard proof-of-work rules.

DISEMBARK — Protocol-Driven Instantiation

DISEMBARK is a protocol-driven procedure executed entirely within Block 0. There is no user-initiated DISEMBARK. The protocol reads the ARC manifest from the originating Phoenix and instantiates all valid ARC Parcels directly as LOCKED outputs in the successor Phoenix. Holders find their Scriptum already waiting — LOCKED as always, ready to UNLOCK.

Rebirth Block 0 is exempt from ordinary block size limits. Its practical size depends on the number of ARC entries, their serialisation rules, and implementation constraints — all of which are specification concerns beyond the scope of this document. Rebirth is infrequent enough that elevated resource requirements at Block 0 are acceptable. Implementations may impose practical limits defined in the Scriptum Implementation Specification. Canonical serialisation and ordering rules will be defined in that document prior to mainnet.

ARC 0 — The Founding Manifest

ARC 0 is the initial manifest instantiated at the first Rebirth (Genesis). It contains:

Constitutional passengers — Warden, Founders, Curia (TREASURE, deterministic vesting)

Civil passengers — Aurum registrations (CIRCULATING, LOCKED)

Mayflower Parcels — one per founding participant (CLASS = MARKER, PERMA_LOCKED)

Cargo — Reserve, Funds, Treasure allocations (VAULT)

Chesed — the semantic name for the donation mechanism. Donated value flows to the Funds and RESERVE vaults at a 50/50 ratio as SEALED Parcels. Dana MARKER Parcels are issued in return to the sender as permanent private record.

Supply accounting for MARKER Parcels:

Mayflower Parcels (CLASS = MARKER, AMOUNT = 1 Nugget each) are instantiated by the protocol at Block 0. They are not created by coinbase (Block 0 coinbase reward = 0) and are not part of the economic supply. They are excluded from all supply calculations:

Not included in S_{mined}

Not included in S_{net}

Not counted against S_{max}

Not subject to the emission formula

MARKER Parcels exist outside the economic model entirely. They are protocol overhead — permanent identity records with no supply consequence. This applies equally to Mayflower Parcels (founding participants) and Dana Parcels (donors).

Allocation

84% Mining — distributed via block rewards across the life of Phoenix 1

5% Contributors — TREASURE: 2% WOC + 2% Founders (5 founders $\times$ 0.4%) + 1% Curia (9 members $\times$ $\sim$ 0.111%). Total 5% = 2,100,000 Scriptum across 15 distinct constitutional position-holders. Deterministic vesting per §21.6.

5% Ecosystem — VAULT, ROLE = Funds (Foundation operational allocation, sourced from Curia-authorised disbursements)

5% Reserve — VAULT, ROLE = RESERVE

1% Genesis First-Buy — distributed at genesis via a water-filling allocation algorithm. The Genesis First-Buy pool is 420,000 Scriptum (1% of $S_{\max}$), offered to the constitutional cohort (1 WOC + 5 Founders + 9 Curia = 15 positions) and to FullArc Sign-ups and Genesis Sign-ups registered before the close of the sign-up window. The price is fixed at €0.01 per Scriptum. The per-person hard cap is 50,000 Scriptum (€500 maximum participation), preventing whale concentration in the first observable transaction.

The allocation algorithm is iterative water-filling with the per-person cap applied at each round. Let N be the count of registered participants and $v(n)$ the amount declared by participant n (capped at 50,000). Round 1 computes $V(\max1) = \text{pool} / N$. All participants whose declared amount $v(n) \leq V(\max1)$ receive their full $v(n)$. The pool is reduced by what was allocated; N is reduced by the participants who received their full allocation. Round 2 computes $V(\max2) = \text{remaining_pool} / \text{remaining_N}$ from the reduced state. The same logic applies. The algorithm continues until either all participants have received their full declared amount (in which case the pool may have a remainder, which flows to FUNDS), or a round closes with all remaining participants having $v(n) > V(\max_round)$ (in which case all remaining participants receive $V(\max_round)$ equally and the algorithm terminates with the pool fully distributed).

The water-filling property is provably fair: every participant either receives exactly the amount they declared (because their declaration was modest enough that the pool could accommodate it) or receives the same amount as every other participant who did not receive their full declaration. No participant is unfairly truncated. No first-mover advantage exists because allocation runs once after the sign-up window closes, not first-come-first-served. The per-person cap of 50,000 prevents whale concentration in the rare case where N is very small.

The constitutional cohort participates on the same terms as every other participant. The WOC declares an amount and is allocated under the same algorithm as a Genesis Sign-up. There is no privileged tier. There is no founder rate. The WOC's distinction is being a constitutional position holder, not a buyer-priority status. The first observable Scriptum/EUR price is €0.01/Scriptum from the first Genesis First-Buy delivery onwards, irrespective of who buys.

Each first-buy is technically structured as a Horkos contract (per §Horkos) with the German Rechtsanwalt as the registered attester C. The participant's fiat is loaded into the

attorney's Anderkonto; the protocol holds the Scriptum in a designated Genesis First-Buy escrow. Upon attestation YES (fiat received in trust), the protocol delivers the participant's allocated Scriptum via the standard NEXUS-routed delivery to the participant's stealth-derived one-time address. Upon attestation NO or TIMEOUT, the Scriptum returns to the Genesis pool and the fiat is refunded to the participant. The chain's bilateral settlement primitive bootstraps the chain's own genesis allocation.

Proceeds — the aggregate fiat raised at the close of the sign-up window — flow to the Genesis Bootstrap Fiduciary structure: held in the German Rechtsanwalt's Anderkonto pending Stiftung incorporation, then transferred to the Foundation upon successful registration in Zug 6–12 months post-genesis. The Rechtsanwalt is paid in FUNK at €1.00/FUNK conversion rate for professional services rendered. Detailed legal mechanics, sign-up window timing, KYC requirements, and operational specifics are documented in the separate Scriptum Genesis Economics specification published alongside this Whitepaper.

Genesis allocations sum to 16% of $S_{\text{max}} = 6,720,000$ Scriptum (5% TREASURE + 5% Funds + 5% RESERVE + 1% First-Buy). S_{net} at Phoenix 1 block 1 = 6,720,000. The remaining 84% is distributed to miners across the Phoenix lifespan via the emission formula $r = 1 \times (1 - S_{\text{net}} / 42,000,000)$, with genesis emission $r_1 = 0.84$ Scriptum/block, asymptotically approaching the 42,000,000 cap by Ragnarök.

Vesting Schedules

All three constitutional cohorts share a 12-month cliff, after which TREASURE vests linearly in monthly increments. Vesting durations track the relative weight of each cohort's constitutional responsibility: the WOC vests over the longest period (4 years total), the Founders over a medium period (3 years total), and the Curia over the shortest (2 years total).

WOC — Warden of the Chain (Allocation: 2% of total supply = 840,000 Scriptum, 1 person)

Cliff = 12 months (3,153,600 blocks)

Release = 1/36th of allocation per month after cliff

Duration = 4 years total (cliff + 36 monthly releases)

Notation = 12 / 36

Founders (Allocation: 2% of total supply = 840,000 Scriptum, distributed as 5 founders × 0.4% each = 168,000 Scriptum per founder)

Cliff = 12 months (3,153,600 blocks)

Release = 1/24th of allocation per month after cliff

Duration = 3 years total (cliff + 24 monthly releases)

Notation = 12 / 24

Curia (Allocation: 1% of total supply = 420,000 Scriptum, distributed as 9 members × ~0.111% each ≈ 46,667 Scriptum per Curia member)

Cliff = 12 months (3,153,600 blocks)

Release = 1/12th of allocation per month after cliff

Duration = 2 years total (cliff + 12 monthly releases)

Notation = 12 / 12

A constitutional position-holder who departs (voluntarily, through removal-for-cause, or through other circumstance) within the first 12 months forfeits all unvested TREASURE. The 12-month cliff is uniform across cohorts and ensures that no position-holder receives any liquid allocation without demonstrating at least one year of constitutional service.

All vesting is protocol-enforced via the VEST and RELEASE transactions. Bastet watches the schedule. RELEASE fires at each milestone. No manual action is required from the holder.

TREASURE is held in personal capacity by the position-holders, not by the Scriptum Foundation. The Foundation holds its own institutional Scriptum position separately, sourced from Curia-authorized disbursements out of Funds (the 32% Tithe slice) for operating purposes.

Subsequent Rebirths follow identical rules. Each new Phoenix begins at block height 0 with a fresh Block 0 derived from the preceding ARC manifest.

PART VI — LONGEVITY

The Hard Ceiling — Supply, Scarcity, and the Answer to Everything

Scriptum has a fixed maximum circulating supply of 42,000,000 Scriptum. This number is not arbitrary. It is the answer to the ultimate question of life, the universe, and everything — as established by Douglas Adams in *The Hitchhiker's Guide to the Galaxy*. Unlike Bitcoin's 21 million, whose origin remains debated, Scriptum's supply cap has a clear and deliberate meaning. It signals the character of the project and the people behind it.

The 42M ceiling is approached asymptotically within each Phoenix. No single Phoenix formally reaches 42M. The Scriptum system continues through successive Phoenix cycles — each beginning clean, each approaching 42M anew. Individual Phoenix instances end at Ragnarök. The system does not.

The Coin and its Unit

The coin is Scriptum. Its ticker is FUNK. The base unit is the Nugget: one Scriptum is divisible into 100,000,000 Nuggets, giving Scriptum eight decimal places of precision — identical to Bitcoin's divisibility model.

1 Scriptum = 100,000,000 Nuggets (ticker: FUNK)

Aurum displays balances in Scriptum by default. Nuggets are shown when the transaction amount falls below 0.001 Scriptum.

The River of Forgetting — Lethe and the Lost Coins

Every blockchain that has ever existed has ignored a fundamental truth: that value does not only move. It disappears. Not through burning, which is deliberate and recorded. Not through theft, which is traceable. But through the quiet, invisible, unstoppable entropy of human existence. Keys are lost. Owners die. Addresses are mistyped. The coins remain on chain, visible, countable, forever unreachable.

Scriptum is the first protocol to name this honestly.

In Greek mythology, Lethe is the river of oblivion. The dead drink from it and forget everything. Scriptum designates all permanently unreachable coins as Lethe — Lost Scriptum. Three categories:

N1 — The Forgotten Address: coins sent to an address that never existed. No key was ever generated. No human ever controlled this address.

N2 — The Dead Man's Wallet: the key existed, the holder is gone — through death, loss, or the destruction of the only record that mattered.

N3 — The Sleeper: the key exists, the holder is alive but silent. Indistinguishable from N2 at the protocol level. What cannot be proven is not protocol.

Two epistemic categories. The chain distinguishes what it can know in real time from what only Ragnarök reveals.

Known-Lethe (protocol-visible loss). Coins sent to provably-unreachable destinations — Gehenna burns, addresses with invalid bundle structure, malformed receive constructions — are recognisable from chain state alone. The chain accounts for these immediately: they enter S_{burned} (for burns) or are excluded from accounting (for malformed-address sends, which never enter S_{mined} to begin with). Known-Lethe is a real-time, deterministic ledger property.

Presumed-Lethe (private holdings of unknown status). Coins held at well-formed addresses where the chain has no signal of holder absence — categories N2 and N3 above — cannot be distinguished from active dormant holdings until Ragnarök. The chain counts these in S_{net} during the Phoenix lifetime exactly as it counts every other holder's value. Only at Ragnarök, when holders must actively board the Arc to migrate to the successor Phoenix, does presumed-Lethe become known: addresses that fail to board are revealed as truly lost. The successor Phoenix begins with the known-boarded supply; the unboarded portion stays on the previous chain as permanent record. This is the structural function of Ragnarök: it converts presumed-Lethe into known-Lethe through the explicit boarding action.

Lost Scriptum is **explicitly and permanently excluded from protocol calculations to the extent the chain can identify it**. Known-Lethe is excluded immediately; presumed-Lethe is identified at Ragnarök through non-boarding and excluded from the successor chain. $S_{\text{net}} = S_{\text{mined}} - S_{\text{burned}}$. Scriptum does not pretend to know what it cannot know in real time, and resolves the unknowable exactly once per century via the Arc procedure.

Based on observed loss rates across comparable proof-of-work chains, Scriptum estimates that presumed-Lethe will represent approximately 15% of mined supply at the twenty-year horizon. By Year 100, accumulated presumed-Lethe could represent 30–40% or more of total mined supply. This is precisely why Ragnarök exists.

The Sacred Fire — Burn Economics and Gehenna

Burning in Scriptum is not destruction. It is a self-healing mechanism. When Scriptum is burned, S_{burned} increases, S_{net} falls, and the block reward rises. This attracts miners, increases hash rate, strengthens network security, and restores equilibrium without intervention. Burning returns energy to the network. Burning is the network breathing.

Early chain burning is economically rational. When circulating supply is small, burning signals confidence and stabilises early price discovery. As supply matures, burning becomes progressively more expensive relative to its market impact and naturally self-limits. Rational economics prevents late-stage manipulation.

All burns are directed to Gehenna — the protocol-defined burn address, provably unspendable by mathematical construction. Gehenna takes its name from the Abrahamic tradition: the place of fire, of deliberate and permanent consumption. There is no key.

There is no reserve. There is no recovery. What enters Gehenna is gone from spendable circulation permanently.

BURN is always partial. The holder specifies amount x. Remainder stays ACTIVE. S_burned increases by x Scriptum only.

The Mayflower Parcel

The Mayflower Parcel is a Genesis marker assigned to founding participants of the Scriptum system. It represents participation in the founding event (ARC 0) and serves as a permanent, non-economic proof of origin.

Structure:

CLASS = MARKER

ROLE = MAYFLOWER

STATE = PERMA_LOCKED

ADDRESS = participant address

AMOUNT = 1 Nugget (display-only denomination, non-economic)

Behavioural rules:

Non-spendable — no UNLOCK, SPEND, LOCK, or BURN

Excluded from all economic calculations and from S_net

Always included in ARC transitions automatically

Re-instantiated unchanged at every Rebirth

Exactly one per founding participant

Cannot be created after ARC 0

Mayflower and Dana Parcels are the only records in the Scriptum system that survive every Ragnarök unchanged. They do not carry value. They carry identity. The amount of 1 Nugget is a display-only denomination — it exists to satisfy the Parcel data model, not to represent economic value. Founding participants and donors alike are permanently inscribed in every successor Phoenix, for as long as the protocol runs.

The Centennial Renewal — Ragnarök, the Arc, and the Successor Phoenix

In Norse mythology, Ragnarök is the twilight of the gods. The world ends. And then — the world is reborn. Green and clean. The cycle complete.

Scriptum has its own Ragnarök.

At Year 100 plus one day, the protocol triggers the centennial renewal. A new Phoenix is born. Every holder must actively board the Arc within a defined migration window. This is not automatic. It is not passive. It requires intent, action, and a living key.

TREASURE — Automatic Boarding

TREASURE boarding authority is defined in Section 6.6. In summary: all Parcels with CLASS = TREASURE board automatically. Parcels that have completed vesting and converted to CLASS = CIRCULATING must be enrolled manually.

Protocol Enforcement

Ragnarök is not a soft convention. It is a hard consensus rule.

RAGNAROK_HEIGHT = 315,360,000

No block with height > RAGNAROK_HEIGHT is valid under the Scriptum protocol

All nodes must reject blocks beyond this height

The value of RAGNAROK_HEIGHT is fixed in genesis parameters

It cannot be changed without departing from the Scriptum protocol

A chain that continues beyond RAGNAROK_HEIGHT is not Scriptum. It is a fork. The Scriptum protocol does not recognise it.

The Arc Procedure — Enrollment and Boarding

Enrollment

A holder initiates migration by declaring the intent to board. Through Aurum, the holder selects a specific boarding time prior to Ragnarök. Enrollment is an explicit act. Unconfirmed participation does not exist.

Scheduled Boarding

Boarding occurs at a predetermined moment chosen by the holder. At this moment, control passes from the user to the protocol. There is no dependency on network conditions or user interaction at the time of execution. Boarding is deterministic and guaranteed.

Cancellation

Enrollment may be cancelled until CANCEL_THRESHOLD. Once this threshold is passed, the decision is final. No further changes are possible.

CANCEL_THRESHOLD = 10 blocks before the scheduled boarding slot

At CANCEL_THRESHOLD, the STRAGGLER_WINDOW also closes. These two events are simultaneous and deliberate: the last moment a holder can cancel is also the last moment stragglers are evaluated. After this point the protocol has full control.

PACKING Phase

At approximately 8,640 blocks (~24 hours) before the selected boarding slot, the holder initiates PACK: all Parcels associated with the enrolled wallet — regardless of state — are consolidated into a single PACKING Parcel with $AMOUNT = \text{SUM}(\text{all Parcels})$. This is the last user-initiated action in the boarding procedure. From this point the protocol has full control.

Incoming Parcels received after PACKING begins are classified as stragglers.

STRAGGLER_WINDOW

The straggler evaluation window closes at CANCEL_THRESHOLD — 10 blocks before the boarding slot. At this point the protocol evaluates all pending stragglers:

STRAGGLER_WINDOW closes = 10 blocks before boarding slot

Aurum displays a countdown timer for the straggler window. At closure, the holder is presented with a confirmation:

Include stragglers? [YES / NO]

Default: NO (countdown expires with no response = NO)

YES confirmed: PACKING Parcel and stragglers consumed, new consolidated PACKING Parcel created

NO confirmed or timeout: stragglers remain on the originating Phoenix, permanently excluded from migration

Only value that was enrolled and confirmed boards the Arc. Nothing boards silently.

BOARDING

At the selected boarding slot, the protocol executes BOARDING: consumes the PACKING Parcel (and optionally accepted stragglers) and creates a single ARC output with $AMOUNT = \text{SUM}(\text{all boarded value})$. This transition is deterministic, irreversible, and final.

Before boarding, value is gathered. At boarding, it is committed.

ANOINTED and CONDEMNED

At CANCEL_THRESHOLD — 10 blocks before the scheduled boarding slot — enrollment closes permanently. At that moment, every wallet is in one of two states:

ANOINTED → enrolled, straggler decision made, boarding confirmed

CONDEMNED → not enrolled, permanently excluded from this Arc

There is no middle ground. No late enrollment. No appeals. The decision is final.

At the boarding slot — 10 blocks after CANCEL_THRESHOLD — the protocol executes BOARD for every ANOINTED wallet simultaneously. This is consensus-enforced: any block that omits a required BOARD output is invalid and rejected by all nodes. The miner cannot exclude it. There is no transaction to censor — BOARD is part of block validation itself, identical in nature to EJECT and EXPIRE. All protocol-driven multi-output events — including BOARD and DISEMBARK — use deterministic canonical ordering defined in the Scriptum Implementation Specification. Implementations that diverge from canonical ordering produce invalid blocks.

A miner who attempts to build a block without the required BOARD outputs produces an invalid block. Other nodes reject it. His chain is orphaned. His hashrate is wasted. There is no attack surface.

After Boarding

The originating Phoenix continues to operate until Ragnarök. Any value that arrives in a boarded wallet after boarding is automatically classified as condemned — it remains valid on the originating Phoenix but is permanently excluded from migration. If the holder does not transfer condemned coins out via SPEND before Ragnarök, they will become permanently inaccessible at the Ragnarök event.

Wallets that were never enrolled (CONDEMNED at CANCEL_THRESHOLD) continue to operate normally on the originating Phoenix until Ragnarök. Their value remains valid and spendable until RAGNAROK_HEIGHT. At Ragnarök, no new blocks are produced. Their value is permanently frozen on the originating chain — visible, verifiable, and permanently unreachable in the successor Phoenix.

*****My name is Ozymandias, king of kings: Look on my works, ye Mighty, and despair.*****
****The coins remain. The chain remembers. Nothing else does.*

Ragnarök and Departure

At Ragnarök, the ledger is finalized. All ARC-state value is fixed. No further boarding is possible. The Arc departs. All boarded value is carried forward into the successor Phoenix. All non-migrated value remains permanently on the originating Phoenix.

The old chain does not cease to exist. It continues, frozen — a permanent archaeological record of every transaction, every burn, every Lethe coin, every block from genesis to Ragnarök.

ARRIVAL — Disembarkation in the Successor Phoenix

ARRIVAL is the process by which ARC-state value becomes available in the new Phoenix. It does not transfer value. It validates proof and reconstructs ownership.

DISEMBARK is a protocol-driven procedure. There is no user-initiated claim transaction. At Block 0 of the successor Phoenix, the protocol reads the complete ARC manifest and instantiates all valid ARC Parcels directly as LOCKED outputs. Holders find their Scriptum already waiting in the new Phoenix, locked and ready to unlock. No action required.

The originating Phoenix is left behind — frozen, immutable, permanently readable. The successor Phoenix begins at block height 0 with no inherited history. Block 0 is its entire starting state.

Finality

Boarding occurs once. Boarding is irreversible. There is no second boarding event, no recovery phase, no extension. What is not aboard remains behind.

The Number That Matters

The number of Scriptum that board the Arc at Ragnarök is the single most important metric in Scriptum's history. It will be known on the day of departure. It will never change. It is a permanent, immutable, public record of how well humanity looked after its wealth across one century. If 42 million board — the system worked perfectly. Every coin that does not board is a story.

Canonical Continuity and Forks

Scriptum does not prevent alternative chains. Any group may choose to continue the ledger beyond Ragnarök by modifying the protocol. Such a continuation is not part of Scriptum as defined by Scriptum. Scriptum's identity is bound to its finite lifecycle and explicit renewal.

The Oldest Truth

Every human civilisation has named the place where things go and do not return. The Greeks called it Lethe. The Norse called the ending Ragnarök. The Hebrews called the fire Gehenna. The Sumerians, four thousand years ago, called the underworld Irkalla — the land of no return.

Scriptum does not invent this. It inherits it. Value has always been lost. Keys have always been forgotten. Owners have always died. What Scriptum does — for the first time in the history of digital assets — is name these things honestly, exclude them from calculations they would corrupt, and design a mechanism that, once per century, washes the chain clean.

Scriptum is not designed as a transactional cryptocurrency in the classical sense built for speculation, high-frequency action, and everyday small purchases. It is a centennial institution — designed to outlive everyone who builds it, everyone who mines it, and everyone who holds it, and to begin again for those who come after.

Ain Soph — System Philosophy

Scriptum is not:

A high-throughput payment network

A trading platform or speculative instrument

A computational layer or smart contract environment

A stablecoin or price-controlled instrument

Scriptum is:

A controlled-value system optimised for secure storage of high-value assets — not optimised for transactional cryptocurrency use cases

A deliberate value transfer mechanism — not optimised for transaction frequency

A centennial institution designed to outlast its creators

Post-quantum from birth

PART VII — SECURITY & REGULATORY

The Security Model

- Scriptum does not eliminate theft. It transforms theft from an instantaneous, invisible act into a sustained, public operation that must persist across time.

Structural Safety vs. Economic Safety

Scriptum is designed as a system, not as a snapshot of current network conditions. Every claim in this section concerns Scriptum's architecture — the properties that hold from block zero through a century of operation, across rising and falling hashrate, through the ingress of institutional adoption and the shocks of attack. Day-one comparisons against mature chains miss the point. Scriptum's security is structural, and structural properties do not depreciate as the network grows.

At low hashrate, all proof-of-work systems are vulnerable to chain reorganisation. In Bitcoin, reorganisation directly translates into the ability to reverse settlements and extract value — the architecture equates a valid signature with final settlement, and a reorg moves final settlements. Every proof-of-work chain has faced this vulnerability at birth. Bitcoin did, in 2009. Every altcoin does. Scriptum does. The question is not whether the vulnerability exists; the question is what it enables.

In Scriptum, reorganisation alone is insufficient. Value transfer requires three conditions simultaneously: control of the chain, survival across the full 8,640-block Safety Control Window, and possession of the legitimate holder's Dilithium3 key material. Any one of the three is fatal to an attempted theft; two of the three is not enough. This is the second independent constraint on settlement that Scriptum adds.

Scriptum does not replace hashrate security — it adds a second independent dimension. At maturity, Scriptum inherits the same hashrate dynamics as any proof-of-work system, but with an additional constraint on settlement that Bitcoin does not have.

Normalizing the axis. “Safer than” is meaningless in isolation. Safer against what, for whom, at what cost? Without a shared axis, every comparison is rhetorical. Normalize to the only axis that matters to money — the protection of holder value under attack — and every other metric becomes a means to that end, not an end in itself. A holder does not buy hashrate. A holder buys the assurance that their value survives.

The Bitcoin model. Bitcoin's architecture equates authorization with execution. A valid signature is a final settlement. There is no window between the two. A compromised key is a lost coin — instantly, irreversibly, with no point of intervention. Hashrate is the only thing standing between an attacker and other people's money. Therefore, hashrate must be enormous. At current scale, Bitcoin secures value by continuously expending tens of billions of dollars per year in energy and hardware.

This is not a critique of Bitcoin. It is a description of what Bitcoin must spend to make its architecture safe. The spend is real, recurring, and load-bearing. If the hashrate falls materially, holder value becomes materially less safe. The two are directly coupled.

The Scriptum model. Scriptum separates authorization from execution by protocol. A valid signature does not settle value — it starts a clock. The Safety Control Window opens, the action becomes visible on chain, the holder can cancel, and protocol-enforced EXPIRE returns any unspent ACTIVE Parcel to LOCKED. Holder value is protected by two mechanisms working together: Dilithium3 ownership (the cryptographic dimension) and the SCW (the temporal dimension). Hashrate in Scriptum performs the same role it performs in every proof-of-work chain: establishing chain ordering and making censorship expensive. Hashrate alone does not guarantee settlement safety in Scriptum because the SCW adds an independent temporal constraint — but hashrate remains necessary to ensure the SCW can function honestly under adversarial conditions.

An attacker with unlimited hashrate and no key steals nothing — but may still degrade liveness by censoring user-initiated transactions. Protocol-driven transactions (EJECT, EXPIRE) cannot be censored; honest nodes reject any block omitting them. User-initiated transactions (UNLOCK, CANCEL, SPEND) can be delayed by a majority attacker, but not falsified or silently dropped. An attacker with a stolen key and no hashrate is visible for twenty-four hours before the attack can complete and can be cancelled by the legitimate holder within that window. Hashrate remains necessary to ensure the Safety Control Window can function honestly; it is no longer sufficient on its own to enable theft. Theft requires both conditions simultaneously, sustained across the full observation window.

Bitcoin spends billions of dollars a year on hashrate to protect what Scriptum protects by design.

The honest concession. At genesis, Scriptum's hashrate is low. An attacker with rented GPU time can successfully reorg a Scriptum block in the first weeks of the chain. We concede this openly. A reorg by itself, without possession of a legitimate holder's Dilithium3 private key, extracts no value from anyone. Every Parcel in NEXUS dwell returns to LOCKED on CANCEL or proceeds to its destination on EJECT — the chain's protocol-driven transitions are consensus-enforced and cannot be censored. Every Parcel in ACTIVE reaches its expiry and is returned to LOCKED by protocol-enforced EXPIRE, which honest nodes will not allow any block to omit. Every LOCKED Parcel requires a fresh 8,640-block SCW before it can move — visible throughout, cancellable throughout, observed throughout. A reorg alone is insufficient for theft.

We must be precise about what the SCW does and does not prevent. The SCW does not make theft impossible regardless of attacker resources. An attacker who possesses BOTH a legitimate holder's Dilithium3 private key AND sufficient sustained hashrate to suppress the holder's CANCEL across a full 24-hour SCW can extract value. What the SCW does is elevate the attack from an instant, hidden, irreversible event to a sustained, detectable, public operation. The attacker must maintain dominance for 8,640 consecutive blocks in plain sight of the entire network while the legitimate holder attempts to cancel. Every honest node sees the UNLOCK. Every wallet can flag it. Every exchange can freeze downstream operations. The attacker cannot hide. Scriptum does not promise impossibility; it promises visibility and time — the preconditions for intervention that no instantly-final chain can offer.

SCW protection is conditional on the user's CANCEL transaction reaching a confirmed block before the observation window expires. In adversarial conditions where a majority mining coalition actively censors CANCEL, the SCW can be defeated by sustained suppression across the full 8,640-block window. The censorship must be continuous: if even one honest block lands in the window carrying CANCEL, the attack fails. This is the same economic assumption underlying every proof-of-work chain — that sufficient honest mining participation exists for user-initiated transactions to be included in reasonable time. Scriptum does not eliminate this assumption. What Scriptum adds is a 24-hour public window in which the assumption can be tested: the legitimate holder sees that their CANCEL is not being included, the community observes the same, exchanges and counterparties can freeze downstream operations, and honest hash has time to respond. The attack is expensive, detectable, and on the clock. It is not silent.

Adoption and the permanent asymmetry. As Scriptum adoption grows, hashrate accumulates the same way it does on every successful proof-of-work chain — through the economic incentive of mining a chain with real value. At parity of adoption, Scriptum has Bitcoin's hashrate protection AND the SCW. Bitcoin has only the first. The asymmetry is permanent. Bitcoin cannot retrofit the SCW onto fifteen years of running commerce; no soft fork can introduce a mandatory 24-hour observation window onto a chain whose entire tooling — wallets, exchanges, custodians, on-chain contracts — assumes instantaneous finality. The SCW is an architectural decision that must be made at block zero. Scriptum made it. Bitcoin did not. Both chains will continue. Only one of them has both layers.

The structural claim. Scriptum is not claimed to be “safer than” Bitcoin on all axes. Bitcoin has battle-testing, deep liquidity, ecosystem maturity, and a fifteen-year track record that Scriptum cannot match at genesis and may not match for many years. Those are real advantages, earned through operation at scale. What Scriptum claims is structural: on the axis that matters to a holder — does my value survive the attack — the answer is yes, by design, from block zero, independent of hashrate. At maturity, that protection stacks with full PoW hashrate. That claim is precise, testable, and does not depend on any comparison that cannot be made rigorous.

A 51% mining attack gives the attacker block production advantage. It does not bypass protocol. Honest nodes reject any block that omits a required protocol-driven output, contains invalid state transitions, or includes forged signatures. A 51% attacker is constrained to producing valid blocks — and valid blocks must follow all consensus rules. The following sections enumerate precisely what such an attacker can and cannot do.

Bitcoin asks: what does it cost to attack the chain. Scriptum asks: what does the holder lose when the attack succeeds. The answers are not on the same axis.

What a 51% Attacker Can Do

Reorg recent blocks (within their sustained advantage window)

Censor user-initiated transactions — UNLOCK, CANCEL, SPEND — by excluding them from blocks

Reverse recent payments within the 10-confirmation window

What a 51% Attacker Cannot Do

Forge a Dilithium3 signature — classically or quantum. No key, no spend.

Steal coins without the private key — Parcel ownership is cryptographically enforced

Censor protocol-driven transactions — EJECT, EXPIRE, RELEASE, BOARD are consensus-enforced. Any block omitting them is invalid and orphaned by honest nodes.

Reverse a finalised SPEND and then use the restored value. Reversing a confirmed SPEND requires a reorg past the 10-confirmation finality window (§32.2). Even after a successful reorg, the restored ACTIVE Parcel remains cryptographically owned by the original holder — the attacker holds a state, not a key. Without the key, the restored ACTIVE Parcel runs to its expiry and is returned to LOCKED by protocol-enforced EXPIRE. To actually move the restored value, the attacker needs the Dilithium3 key AND must complete a fresh 8,640-block SCW — visible and cancellable for the full duration.

The Three Attack Scenarios

Scenario 1 — Reverse NEXUS dwell:

Result: Parcel returns to LOCKED. Attacker gains nothing. Holder loses time only.

Scenario 2 — Reverse ACTIVE:

Result: Parcel returns to LOCKED via EXPIRE (protocol-driven, cannot be censored). Attacker gains nothing. Holder loses time only.

Scenario 3 — Reverse SPEND (within 10-confirmation window):

Result: ACTIVE Parcel restored with original timer. But attacker still needs the private key to move it. Without the key, the timer runs to zero, EXPIRE fires, coins return to LOCKED. To use the ACTIVE state, attacker needs the key AND must go through the full NEXUS dwell again — another 24 hours, visible to all.

In every scenario, without the private key, the coins cannot be moved. The SCW combined with protocol-driven mandatory transactions means the worst realistic outcome of a mining attack without key compromise is delay — not loss.

The bank can be broken into. The safe remains locked.

The attacker who possesses both the key and sustained hashrate is the one remaining threat. That attack is not architecturally prevented; it is architecturally exposed. A 24-hour public window of sustained attacker dominance against a specific holder is an extraordinary event — visible, recordable, cancellable, and costly to the attacker whether it succeeds or fails. Scriptum does not promise that theft cannot happen. It promises that theft cannot happen silently.

Scriptum is open source. Every line of protocol code, every consensus rule, and every parameter defined in this whitepaper will be publicly available for review, challenge, and improvement.

The community is invited to audit the protocol. A bug bounty program will be established at launch, rewarding verified vulnerability disclosures in Scriptum from the Reserve allocation.

A formal independent security audit will be commissioned from the Reserve allocation when the chain has sufficient value to fund it responsibly. Until then, open-source community review is the primary security mechanism — the same model used by Bitcoin for its first years of existence.

Key Theft Scenario

An attacker obtains a user's Dilithium3 private key. In Bitcoin, Ethereum, or any chain with immediate execution, this the consequence is immediate: the attacker can drain the wallet before any observer has time to react.

In Scriptum, key theft is the beginning of a race the attacker loses by default. To move value, the attacker must submit UNLOCK. UNLOCK opens the Safety Control Window. The victim — or anyone watching the chain on the victim's behalf — can CANCEL the unlock for the full window duration. CANCEL is free, has maximum mempool priority, and is prioritised by honest mining policy at the SCW boundary (see §Mempool Policy).

For the attacker to succeed, they must prevent the victim from observing the UNLOCK and submitting CANCEL for the full window. Monitoring tools, delegated audit keys, and trusted parties make this window difficult to bypass. The theft of a key no longer means theft of value. The theft of a key means an observable alert.

Oracle Manipulation Scenario

An attacker attempts to manipulate V_A by corrupting oracle data feeds or miner V_A proposals. The attacker's goal is to make large SPEND transactions cheap (inflated V_A) or to force fee-bound economic instability (deflated V_A).

The Urðarbrunnr Integrity System blocks this through three independent layers. First, per-block V_A movement is bounded — no single block can move V_A more than a fixed percentage. Second, rolling-window V_A movement is bounded — V_A cannot drift beyond a fixed percentage over the UIS window. Third, oracle data itself is cross-checked: values must pass both a cross-source consistency filter and a statistical outlier filter before they enter V_A computation at all.

Beyond these filters, two separate warning mechanisms detect different manipulation signatures. The Hugin and Munin CUSUM detection pair raises an informational tampering alarm when miner-proposed V_A consistently drifts from oracle-reference V_A — Hugin captures the per-block deviation, Munin accumulates it, and the alarm fires when the cumulative drift exceeds threshold. This is the signature of sustained manipulation. A separate fast-price-divergence warning fires when instantaneous oracle-vs- V_A gap

exceeds threshold — the signature of legitimate market events. Both are informational; neither affects execution or freezes the chain. Their purpose is transparency: making manipulation attempts economically irrational by making them publicly visible.

To manipulate V_A , an attacker must sustain distortion across multiple independent oracle sources, over time, while also controlling sufficient miner V_A proposals to avoid cross-checks. The required resource investment exceeds any plausible gain from the manipulation itself.

Timestamp Gaming Scenario

An attacker attempts to exploit the Menuha Window boundary by submitting outward-value transactions (SPEND, BURN, DONATE) with block timestamps that appear to fall just outside the restricted window.

Scriptum defends against timestamp gaming by using Median Time Past — the median of the last eleven block timestamps — rather than raw block timestamps to determine whether a block falls within Menuha. A single miner cannot shift MTP without controlling majority hashrate for multiple consecutive blocks. This is the same defense Bitcoin uses against timestamp manipulation, adapted to Menuha enforcement.

An attacker attempting to squeeze a SPEND in during Menuha-boundary blocks finds that MTP does not shift quickly enough to matter. The Menuha window is robust to single-miner timestamp games.

NIST Alignment Statement

Scriptum is designed to operate on a cryptographic foundation aligned with the post-quantum standards defined by the National Institute of Standards and Technology (NIST).

At its core, the system uses NIST-selected post-quantum primitives:

CRYSTALS-Dilithium3 (ML-DSA) — digital signatures. Active from genesis. Every user-authorized action on Scriptum is signed with Dilithium3 from genesis. Every address is secured against attacks from cryptographically relevant quantum computers. Protocol-driven transitions carry no signature — they are consensus-enforced and cannot be forged without breaking the consensus rules themselves.

CRYSTALS-Kyber (ML-KEM) — key encapsulation mechanism. Active from genesis. Enables stealth address derivation and holder-controlled audit keys.

Scriptum does not introduce proprietary cryptographic algorithms. It relies exclusively on publicly vetted, standardised, and peer-reviewed primitives implemented according to established best practices. Both Dilithium3 and Kyber were standardised by NIST in 2024 following years of open international review.

Crucially, Scriptum extends beyond cryptographic robustness by incorporating a protocol-level architecture that mitigates the impact of key compromise. Even in the event of key exposure, value transfer is subject to enforced temporal constraints — the Safety Control

Window — and deterministic validation processes. This provides an additional layer of systemic security independent of the underlying cryptographic assumptions.

This dual approach — standards-based post-quantum cryptography combined with protocol-level temporal safeguards — ensures both formal alignment with emerging global cryptographic standards and structural resilience against future threat models.

In plain terms: even if the cryptography is broken, the SCW gives the holder time to respond. Scriptum does not rely on any single layer of security. It was designed to survive failure at any layer.

Regulatory Position

Scriptum (FUNK) is a protocol-native asset required for network operation. It provides no yield, no entitlement, and no profit claim. It is the medium through which fees are paid and miners are compensated for securing the Scriptum network.

Statement of intent:

The Scriptum foundation will seek legal clarity in all relevant jurisdictions commencing at genesis. We acknowledge that the regulatory classification of digital assets varies by jurisdiction and evolves continuously. We will invite qualified legal counsel in the EU (MiCA), UK (FCA), and US (SEC/CFTC) as part of the mainnet preparation process.

Scriptum is a permissionless proof-of-work protocol with no issuer, no central operator, and no custodial intermediary. The protocol itself has no jurisdiction. Anyone, anywhere, can run a node, mine, hold, or transact, subject only to the laws that apply to them personally.

The Scriptum Foundation, once incorporated at the Bootstrap T1 milestone (target 6-12 months post-Genesis), will be responsible for the public-facing infrastructure: the dfqr.org website, the Aurum wallet software, and governance coordination. The Foundation operates in compliance with the laws of its jurisdiction of incorporation and any jurisdictions where it actively conducts business. The Foundation does not offer Scriptum for sale, does not provide custodial services, and does not operate a centralised exchange.

Until incorporation, public-facing infrastructure is operated by the Warden personally with Treuhänder support per Bootstrap §14

Holders in any jurisdiction are responsible for their own compliance with applicable local law, including tax reporting, anti-money-laundering requirements, and any licensing obligations that may apply to their own activities (for example, operating a commercial exchange or providing custodial services).

The protocol is neutral. The Foundation is compliant where it sits. The users are responsible for themselves. This is the only honest framing for a decentralised system.

This whitepaper does not constitute financial advice, an offer of securities, or an invitation to invest. It is a technical and philosophical description of a protocol.

Invariants and Assurance

The Scriptum corpus consists of nine documents: this Whitepaper, the Constitution and Governance documents, the System Parameters Extended specification, the Canonical Machine document, the Highlights document, the Invariants and Assurance document, the Economic and Adversarial Analysis, and the Aurum Technical Document. The IA codifies the mathematical properties the chain must always satisfy and the engineering practices that verify those properties continuously.

A chain intended to last a hundred years cannot rely on craftsmanship alone to catch consensus bugs. Three real bugs were caught during Scriptum's pre-genesis development by careful review of running code: a coinbase maturity bypass that would have allowed mined Scriptum to be spent before the 100-block confirmation discipline; a retarget direction inversion at sub-second block production that would have moved difficulty in the wrong direction during bootstrap and Phoenix renewal; and an arithmetic overflow in target adjustment that silently masked failure cases. Each was found before genesis. Each would have been a hard-fork problem to fix post-launch.

The IA document specifies five assurance practices:

Formal invariants. Mathematical properties the chain must always satisfy, stated as predicates over chain state. Examples: `s_mined` is monotonically non-decreasing; for every accepted block `N`, all transitions in `N` validate against state at height `N-1`; for any solution accepted at height `N`, the block timestamp satisfies both `block.timestamp > MTP(parent)` and `block.timestamp ≤ local_clock + FUTURE_TIMESTAMP_TOLERANCE_MS`; coinbase outputs match the protocol emission formula exactly. Each invariant has a precise statement; each statement is testable.

Property-based tests. For every formal invariant, a proptest-based test that exercises the invariant across randomised inputs. These run on every commit. The test suite must pass before any merge to the canonical branch.

Fuzzing harnesses. cargo-fuzz targets for every parser (message decoders), every state transition validator, every arithmetic operation involved in consensus (target adjustment, reward calculation, retargeting math). Continuous fuzzing runs across every parser surface.

Adversarial test scenarios. Simulated attacks with specified attacker capabilities, expected chain responses, and verified assertions. The current scenario catalogue includes timewarp manipulation, hashrate spike-and-leave, same-second timestamp flood under coarse-resolution emulation, mempool spam, reorg attempt at finality depth, and Sybil peer flooding. Each new attack-vector identified joins the catalogue and gains a regression test.

External audit. Pre-genesis, the chain undergoes at least one external security audit by a recognised firm. Audit scope, methodology, remediation requirements, and resulting reports are specified in IA. Audit findings that surface real consensus issues block genesis until resolved.

The three pre-genesis bugs caught by craftsmanship are documented in IA as retrospective case studies and have permanent regression tests in the assurance suite. Future bugs caught will join them. The chain's assurance posture is observable, not claimed.

Conclusion — A System Built for Failure

Every custody system in history has been built on the assumption that nothing will go wrong. Scriptum is built on the assumption that something will.

Keys will be compromised. Mistakes will be made. Attackers will obtain valid signing capability. People will die with unspent holdings. These are not edge cases — they are the normal operation of any system at scale across a century.

Scriptum does not prevent failure. It contains it. Every mechanism in this document — the SCW, the CANCEL window, Limbo, EXPIRE, the Arc, Ragnarok — exists because the designers assumed something would go wrong and built a time buffer around it.

Scriptum is a time-controlled custody system with on-chain enforcement. It is the first system designed not to eliminate risk, but to make risk survivable.

Final Statement

Scriptum does not move value quickly.

It ensures value does not move when it should not.

Appendix A. Privacy Architecture

This appendix describes the Scriptum privacy architecture. It is part of the protocol at genesis. Every mechanism described here is live in block zero; nothing awaits activation.

The Three-Layer Model

Scriptum provides three composable security layers: cryptographic (Dilithium3 signatures, post-quantum), temporal (the Safety Control Window), and privacy (stealth addressing plus holder-controlled audit).

The three layers are designed to coexist. Privacy does not weaken the SCW. The SCW does not break privacy. The holder observes their own SCW through their view key — privately. The world sees an unlock occurring. It cannot see whose, or how much.

Stealth Wallets — How They Work

Every Scriptum wallet generates the full key bundle:

Dilithium3 spend key — signs transactions. Authorises movement.

Kyber view key — enables post-quantum key exchange. Derives shared secrets for stealth operation.

Together these form a stealth address. The stealth address is published. It never changes. But every transaction directed to it lands at a different one-time address derived from the shared secret.

Sending to a Stealth Address

The sender:

Obtains the recipient stealth address (Dilithium3 + Kyber keys)

Kyber/ML-KEM-based key encapsulation is a send-side operation. Only the sender performs the encapsulation. The recipient scans the chain with their view key to detect incoming stealth outputs by decapsulating each SPEND's ephemeral material; no handshake or back-channel communication is involved.

Picks a random ephemeral value r

Uses the recipient Kyber key and r to derive a shared secret via Kyber/ML-KEM key encapsulation

Derives a unique one-time destination address from that secret

Sends to that address

Publishes r on chain (encrypted — only recipient can use it)

The world sees a transaction to an unknown one-time address. It cannot link that address to the recipient. It cannot link it to any prior transaction. It cannot link sender to receiver.

Receiving — Chain Scanning

The recipient wallet continuously scans the chain. For each published r value it:

Derives the shared secret using its Kyber view key

Derives the corresponding one-time address

Checks whether funds arrived at that address

If yes: derives the spend key from the shared secret and claims the output

No interaction between sender and recipient is required beyond the on-chain data.

The Audit Key

The holder may derive an audit key from their Kyber view key material. The recipient link — who sent, who received, how much — is visible only to the holder and to parties the holder explicitly authorises by sharing this key. No authority has inherent visibility. No regulator has automatic access. Disclosure is always and only the holder's decision.

The authority with the audit key can see:

All incoming transactions to the holder

Consumed inputs and output values

Transaction timing and amounts

The authority cannot see:

The real-world identity of the holder — pseudonymous addresses only

Outgoing transactions (spend key required — never shared)

Transactions of any other holder

The audit key grants read access only. It cannot sign, spend, unlock, or alter any transaction. The spend key never leaves the holder. An auditor with the audit key can see everything. They can do nothing. The mathematics enforces this separation — it is not a policy, it is a constraint.

Disclosure is voluntary. The holder decides who receives the audit key, when, and under what conditions. The protocol never compels or enables forced disclosure. Legal obligations are the holder's responsibility under their own jurisdiction. The chain does not know who you are. It cannot testify.

Identity is never exposed automatically by protocol. The chain knows addresses. It does not know people. Even the audit authority sees only what the mathematics allows — flows, not faces.

Protocol Integration

Stealth addressing is a live protocol feature from block zero. Every Scriptum address carries real Kyber key material. Every wallet performs Kyber/ML-KEM key encapsulation on send. Every recipient scans the chain with their view key. There is no activation event; the chain shipped complete.

Any future structural change to the privacy architecture requires the hard fork mechanism defined in the Governance Charter.

Appendix B. Aurum UX Reference

Aurum UX Reference

Implementation guidance for the Aurum reference wallet

This appendix defines Aurum-specific implementation requirements. These are not consensus rules. They are the canonical user experience model for the Scriptum reference wallet. Alternative wallet implementations may exist, but Aurum defines the standard against which user experience is measured.

Aurum displays a progressive warning system in parallel with all other wallet functions as Ragnarok approaches. Warnings are non-blocking. The wallet operates normally throughout. They are informational and time-sensitive.

Key heights:

ENROLLMENT_OPEN_HEIGHT = 288,873,600 (one year before Ragnarok)

RAGNAROK_HEIGHT = 315,360,000

A.1.1. Warning Levels

Level 1 — One Year Warning

Block 288,873,600. Enrollment opens.

THE ARC OPENS IN ONE YEAR

Enrollment is now available. Board before Ragnarok.

Level 2 — Six Month Warning

Block 297,116,800.

THE ARC DEPARTS IN SIX MONTHS

Enroll now. Boarding windows are filling.

Level 3 — Three Month Warning

Block 302,116,800.

THREE MONTHS TO RAGNAROK

Enroll immediately.

Level 4 — One Month Warning

Block 311,116,800.

RAGNAROK IN 30 DAYS

Final enrollment window. PACKING begins soon.

Level 5 — One Week Warning

Block 313,116,800.

SEVEN DAYS REMAINING

Enrollment closed. PACKING imminent.

Level 6 — PACKING Active

Triggered when wallet PACKING phase begins (~8,640 blocks before boarding slot).

PACKING IN PROGRESS

Your value is being consolidated. Do not send to this wallet.

Level 7 — Boarded

Triggered when BOARD transaction is confirmed.

YOU ARE ABOARD THE ARC

Value committed. Awaiting Ragnarok.

A.1.2. Warning Placement

All warning levels display as a persistent banner at the top of every Aurum screen. The banner is dismissible per session but reappears at each launch. It does not block wallet operation.

B.1. Boarding Status Display

Aurum maintains and displays a boarding status indicator at all times once ENROLLMENT_OPEN_HEIGHT is reached. Status transitions:

NOT ENROLLED -> enrollment available, no action taken

ENROLLED -> boarding slot selected, cancellation available

CANCELLED -> enrollment cancelled, re-enrollment available

PACKING -> consolidation in progress

STRAGGLER WINDOW -> 10 blocks before boarding, confirm or reject stragglers

ANOINTED -> enrollment confirmed, BOARD will execute automatically

CONDEMNED -> not enrolled, excluded from this Arc

BOARDED -> ARC state, awaiting Ragnarok

DEPARTED -> Ragnarok passed, value carried forward

The boarding status panel shows:

Current status

Selected boarding slot (block height and estimated date/time)

Countdown to boarding slot in blocks and estimated time

Countdown to Ragnarok

Total value enrolled in Scriptum and EUR

Straggler status if applicable

B.2. UIS Protected Mode

When the Urdarbrunnr Integrity System detects abnormal V_A conditions, Aurum activates Protected Mode:

PROTECTED MODE ACTIVE

Valuation anomaly detected. Fee estimates may be inaccurate.

Confirm to proceed or wait for V_A to normalize.

Protected Mode requires explicit user confirmation before any fee-bearing transaction (UNLOCK, SPEND) is submitted. CANCEL remains accessible without additional confirmation. Protected Mode deactivates automatically when UIS conditions normalize.

Extended Protected Mode (Prolonged Oracle Failure)

When oracle coverage remains below the minimum threshold for ORACLE_FAILURE_ESCALATION_BLOCKS (default ~7 days at 10-second blocks per SPX-8), Aurum enters Extended Protected Mode: a sustained banner warning that V_A is operating from a frozen historical value and that fees may not reflect current Scriptum-to-fiat market conditions. Aurum requires explicit re-confirmation for SPEND transactions above PROTECTED_MODE_SPEND_THRESHOLD (default €10,000 equivalent at frozen V_A). The Curia is notified via on-chain alert event and publishes a status assessment within 14 days. If oracle coverage does not recover within 30 days of escalation, the situation escalates to L4 constitutional governance for review of the oracle architecture itself. Throughout all stages of Extended Protected Mode, the chain continues operating with frozen V_A; consensus validation continues uninterrupted; the escalation rule is operational and governance-level, not consensus-level.

B.3. Parcel State Display

Aurum displays each Parcel with its full CLASS / ROLE / STATE attributes. Standard display format:

AMOUNT CLASS ROLE STATE

10.0000 CIRCULATING NONE LOCKED

5.0000 CIRCULATING NONE NEXUS [DWELL: 412 blocks until eject window]

2.5000 CIRCULATING NONE ACTIVE [Expires: 308 blocks]

50.0000 TREASURE FOUNDER VESTING [Release: block 1,440,000]

1 Nugget MARKER MAYFLOWER PERMA_LOCKED

Limbo state display:

5.0000 CIRCULATING NONE LIMBO – AWAITING EJECT

B.4. Fee Preview

Before any fee-bearing transaction, Aurum displays a full fee breakdown in both Scriptum and EUR:

UNLOCK 10.0000 Scriptum

f_U 0.0004 Scriptum = EUR 0.04

f_A 0.0001 Scriptum = EUR 0.01 (deducted at EJECT)

Total 0.0005 Scriptum = EUR 0.05

V_A EUR 100,000 / Scriptum (current)

If Protected Mode is active, fee preview includes:

! V_A anomaly detected. Fees shown may differ at execution.

Colophon

Prepared by DrFunkenstein.

Apache 2.0 licence on all referenced code and source files.

Cross-references in this document align with corpus v14.